

```
=====
=
const express = require('express');
const router = express.Router();
const userController = require('../controllers/userController');
const { verifyToken, authorizeRoles } =
require('../middlewares/authMiddleware');

// Rutas públicas
router.post('/create', userController.register);
router.post('/login', userController.login);

// Rutas protegidas
router.get('/', verifyToken, authorizeRoles(['admin', 'seller']),
userController.getAllUsers);
router.get('/:id', verifyToken, authorizeRoles(['admin', 'seller']),
userController.getUserById);
router.put('/:id', verifyToken, authorizeRoles(['admin', 'seller']),
userController.getUserUpdate);
router.delete('/delete/:id', verifyToken, authorizeRoles(['admin']),
userController.getUserDelete);

module.exports = router;
```

"; sé que tiene errores pero por ahora no vamos a modificar nada. En lo que quiero que me ayudes es a hacer una explicación del mismo tomando en cuenta la siguiente ruta para REACT: 1) Configuración del Entorno de Desarrollo (Qué necesito tener en mi computador para iniciarlo); 2) Iniciar el proyecto (todos los pasos para correr react); 3) Estructura de carpetas y archivos con su respectiva explicación; 4) Código del Proyecto (Desde el archivo más profundo hasta el más externo, de cara al usuario; además de explicar la función del archivo y su código paso a paso); 5) puesta en marcha (de React). Has lo mismo anterior, pero con la API. Explicame el primer punto hasta que escriba "siguiente" para pasar al segundo punto y así sucesivamente.

¡Excelente proyecto! Me alegra que estés trabajando con una arquitectura MVC bien estructurada. Voy a explicarte todo paso a paso, y esperaré tu "siguiente" para continuar con cada punto.

## 📌 PUNTO 1: Configuración del Entorno de Desarrollo (Frontend React)

Para ejecutar este frontend en React, necesitas tener instalado en tu computador lo siguiente:

### ✅ 1. Node.js y npm (o yarn)

- **Node.js** es el entorno de ejecución para JavaScript del lado del servidor. También incluye **npm** (Node Package Manager).
- **Versión recomendada:** Node.js 18.x o superior (LTS).
- **Verificar si ya los tienes:**

```
bash

node --version
npm --version
```

- Si no los tienes, descárgalos desde <https://nodejs.org>

## ✓ 2. Git (opcional pero recomendado)

- Para clonar repositorios o manejar versiones.
- Descargar desde <https://git-scm.com>

## ✓ 3. Un editor de código (IDE)

- **Visual Studio Code** (recomendado) con extensiones para React, ESLint, Prettier.
- O cualquier editor de tu preferencia (Sublime, Atom, WebStorm).

## ✓ 4. Navegador web actualizado

- **Chrome, Firefox o Edge** (con herramientas de desarrollador activas).

## ✓ 5. Conocimientos previos básicos

- HTML, CSS, JavaScript moderno (ES6+).
- Conceptos básicos de React: componentes, hooks (useState, useEffect), props, rutas.

---

### 📦 Qué tecnologías usa este proyecto (ya incluidas en package.json):

- **React 19** — Biblioteca principal.
- **React Router DOM 7** — Enrutamiento.
- **Vite 8** — Empaquetador y servidor de desarrollo rápido.
- **jwt-decode** — Para manejar tokens JWT en el cliente.
- **ESLint** — Para mantener la calidad del código.

---

### ⚠ Requisito adicional importante:

- El frontend consume una **API REST** que debe estar corriendo en `http://localhost:3000/api`.
- Asegúrate de que el backend esté funcionando **antes** de iniciar el frontend, o al menos configurado correctamente en `src/config/api.js` (ya está apuntando a `localhost:3000`).

---

### ¿Estás listo para el siguiente punto?

Escribe "siguiente" y te explicaré cómo iniciar el proyecto React desde cero. 🚀

siguiente

## 📌 PUNTO 2: Iniciar el proyecto React (Frontend)

Te voy a guiar paso a paso para que puedas correr este frontend en tu computador.

---

### 🚀 Paso 1: Ubicarte en la carpeta del frontend

Abre tu terminal (CMD, PowerShell, Git Bash o la terminal integrada de VS Code) y navega hasta la carpeta donde está el frontend:

```
bash

cd frontend_web/react_node_express
```

**Nota:** Asegúrate de que dentro de esta carpeta exista el archivo `package.json` que viste en el listado.

## Paso 2: Instalar las dependencias del proyecto

Ejecuta el siguiente comando para instalar todos los módulos necesarios (los que aparecen en `package.json` en `dependencies` y `devDependencies`):

```
bash

npm install
```

### ¿Qué hace este comando?

- Lee el archivo `package.json`.
- Descarga e instala todas las librerías en una carpeta llamada `node_modules`.
- Genera el archivo `package-lock.json` que fija las versiones exactas.

 **Tiempo estimado:** 1-2 minutos (dependiendo de tu conexión a internet).

### Posibles errores:

- Si ves `npm command not found`, es porque Node.js no está instalado correctamente.
- Si hay errores de permisos en Linux/Mac, usa `sudo npm install`.

## Paso 3: Verificar que todo se instaló correctamente

Después de la instalación, deberías ver:

- Una carpeta `node_modules` (con cientos de subcarpetas).
- El archivo `package-lock.json`.

Puedes verificar que React esté instalado con:

```
bash

npm list react
```

### Salida esperada:

```
text

sis-web-mvc@0.0.0 /ruta/de/tu/proyecto
├─ react@19.2.4
```

## Paso 4: Iniciar el servidor de desarrollo de Vite

Ejecuta el comando:

```
bash

npm run dev
```

### ¿Qué hace este comando?

- El archivo `package.json` tiene en la sección `"scripts"` la línea: `"dev": "vite"`
- Esto levanta un servidor de desarrollo rápido con Hot Module Replacement (HMR).
- Por defecto, corre en `http://localhost:5173`

### Salida esperada en consola:

```
text

VITE v8.0.4  ready in 300 ms

→ Local:   http://localhost:5173/
→ Network: http://192.168.x.x:5173/
→ press h + enter to show help
```

## Paso 5: Abrir el navegador

El servidor se abre automáticamente porque en `vite.config.js` tienes configurado:

```
javascript

server: {
  port: 5173,
  open: true // ← Abre automáticamente el navegador
}
```

Si no se abre solo, ve a tu navegador y escribe:

👉 `http://localhost:5173`

## Paso 6: Verificar que el frontend se ve correctamente

Deberías ver:

- La pantalla de **login** (porque la ruta raíz `/` redirige a `/login`).
- Los estilos CSS aplicados (gradiente morado de fondo).
- El formulario con campos de email y contraseña.

### Si ves errores en consola del navegador (F12 → Console):

- Revisa que el **backend** esté corriendo en `http://localhost:3000`.
- Verifica que la API esté respondiendo (puedes probar con `http://localhost:3000/api/users/login` en Postman).

## Comandos útiles adicionales

| Comando                      | Descripción                                                  |
|------------------------------|--------------------------------------------------------------|
| <code>npm run build</code>   | Genera la versión de producción (carpeta <code>dist</code> ) |
| <code>npm run preview</code> | Previsualiza la versión de producción localmente             |
| <code>npm run lint</code>    | Ejecuta ESLint para verificar calidad del código             |
| <code>Ctrl + C</code>        | Detiene el servidor de desarrollo                            |



## ⚠ Posibles problemas y soluciones

| Problema                             | Solución                                                                                                                |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Error <code>ECONNREFUSED 3000</code> | El backend no está corriendo. Inícialo antes.                                                                           |
| El puerto 5173 está ocupado          | Vite usará automáticamente el siguiente disponible (5174, etc.)                                                         |
| Los estilos no se ven                | Verifica que <code>import './App.css'</code> y <code>import './styles/common.css'</code> están en <code>main.jsx</code> |
| No carga el dashboard                | Usa <code>oliver@stone.com</code> y la contraseña que configuraste en el backend (debe existir en BD)                   |

### ✅ Conclusión del Punto 2:

Ya tienes el frontend corriendo en `http://localhost:5173`. Ahora puedes interactuar con la aplicación.

### ¿Listo para el siguiente punto?

Escribe "siguiente" y te explicaré en detalle la **estructura de carpetas y archivos** del proyecto React. 📁

siguiente

## 📌 PUNTO 3: Estructura de carpetas y archivos (Frontend React)

Te voy a explicar la organización del proyecto **de afuera hacia adentro**, para que entiendas qué hace cada carpeta y archivo.

### 📁 Estructura general del proyecto

text

```
frontend_web/react_node_express/
|
├── index.html                # Página principal HTML
├── package.json              # Dependencias y scripts del proyecto
├── vite.config.js            # Configuración de Vite (servidor, plugins)
├── eslint.config.js          # Reglas de ESLint para calidad de código
├── README.md                 # Documentación básica
├──
├── src/                      # TODO el código fuente de React
│   ├── main.jsx              # Punto de entrada (renderiza App)
│   ├── App.jsx               # Componente principal (rutas y layout)
│   ├── App.css               # Estilos globales básicos
│   ├── index.css             # Variables CSS y reset
│   ├──
│   ├── config/               # Configuraciones globales
│   │   ├── api.js            # URL base de la API y endpoints
│   │   └── routes.js         # Nombres de rutas (constantes)
│   ├──
│   ├── controllers/          # Lógica de negocio (MVC - Controladores)
│   │   ├── AuthController.js # Login, registro, logout
│   │   ├── UserController.js  # CRUD de usuarios con permisos
│   │   └── DashboardController.js # Estadísticas y actividades
```

|                     |                                              |
|---------------------|----------------------------------------------|
| models/             | # Comunicación con la API (MVC - Modelos)    |
| AuthModel.js        | # Peticiones de autenticación                |
| UserModel.js        | # Peticiones CRUD de usuarios                |
| DashboardModel.js   | # Datos simulados del dashboard              |
| services/           | # Servicios auxiliares                       |
| httpService.js      | # Cliente HTTP (fetch con token)             |
| storageService.js   | # Manejo de localStorage                     |
| jwtService.js       | # Decodificar y validar tokens JWT           |
| hooks/              | # Hooks personalizados de React              |
| useAuth.js          | # Estado de autenticación (login/logout)     |
| useUsers.js         | # Estado y operaciones de usuarios           |
| useDashboard.js     | # Estado del dashboard                       |
| views/              | # Componentes de vista (interfaz de usuario) |
| auth/               | # Pantallas de autenticación                 |
| LoginView.jsx       | # Formulario de login                        |
| RegisterView.jsx    | # Formulario de registro                     |
| dashboard/          | # Pantallas del panel principal              |
| DashboardView.jsx   | # Vista principal con tabs                   |
| DashboardHeader.jsx | # Header con tabs y logout                   |
| DashboardStats.jsx  | # Tarjetas de estadísticas                   |
| UsersView.jsx       | # Tabla de usuarios (CRUD)                   |
| UserForm.jsx        | # Modal para crear/editar usuarios           |
| UserDetails.jsx     | # Modal con detalles del usuario             |
| common/             | # Componentes reutilizables                  |
| Navbar.jsx          | # Barra de navegación pública                |
| AlertMessage.jsx    | # Alertas toast (éxito/error)                |
| LoadingSpinner.jsx  | # Indicador de carga                         |
| layouts/            | # (vacío) Para layouts complejos             |
| styles/             | # Archivos CSS específicos                   |
| common.css          | # Estilos base del dashboard                 |
| Login.css           | # Estilos pantalla de login                  |
| Register.css        | # Estilos pantalla de registro               |
| Navbar.css          | # Estilos barra de navegación                |
| Dashboard.css       | # Estilos tabs del dashboard                 |
| Users.css           | # Estilos tabla, modales, roles              |

## Explicación detallada de carpetas clave

### config/ - Configuraciones

- `api.js` → Define la URL base de la API ( `http://localhost:3000/api` ) y todos los endpoints ( `/login` , `/users` , etc.)
- `routes.js` → Constantes con los nombres de las rutas para evitar errores de escritura.

### controllers/ - Controladores (lógica de negocio)

- Reciben datos de las vistas, los validan, llaman a los modelos y devuelven respuestas.
- **Ejemplo:** `AuthController.handleLogin()` valida email, llama a `AuthModel.login()` y guarda el token.

### models/ - Modelos (comunicación con API)

- Solo se encargan de hacer peticiones HTTP y devolver datos crudos.
- **Ejemplo:** `UserModel.getAllUsers()` hace un `fetch` a `/api/users` y retorna la respuesta.

## 📁 `services/` - Servicios transversales

- `httpService.js` → Cliente HTTP que maneja el token automáticamente en los headers.
- `storageService.js` → Abstracción de `localStorage` (guardar/obtener token, usuario, rol).
- `jwtService.js` → Decodifica JWT y verifica expiración.

## 📁 `hooks/` - Hooks personalizados

- Son la **capa de estado** de React. Usan los controladores y exponen datos y funciones a las vistas.
- `useAuth` → Proporciona `login()`, `logout()`, `isAuthenticated`, `currentUser`.
- `useUsers` → Proporciona `users`, `createUser()`, `deleteUser()`, etc.

## 📁 `views/` - Componentes de UI

- `auth/` → Pantallas públicas (login, registro).
- `dashboard/` → Pantallas protegidas (solo usuarios autenticados).
- `common/` → Componentes reutilizables (Navbar, Alertas, Spinner).

## 📁 `styles/` - CSS modular

- Cada archivo CSS corresponde a un componente específico.
- No hay CSS-in-JS, se usan clases tradicionales.

## 💡 Flujo de datos típico

```
text

Usuario hace clic en "Iniciar Sesión"
    ↓
LoginView.jsx (captura el evento)
    ↓
Llama a useAuth.login()
    ↓
useAuth llama a AuthController.handleLogin()
    ↓
AuthController valida datos y llama a AuthModel.login()
    ↓
AuthModel hace fetch a la API real (httpService)
    ↓
API devuelve { success: true, data: { token, user } }
    ↓
AuthModel guarda token y usuario en storageService
    ↓
AuthController retorna el éxito a useAuth
    ↓
useAuth actualiza el estado (isAuthenticated = true)
    ↓
LoginView redirige al Dashboard
```

## 🏗️ Convenciones del proyecto

| Carpeta                   | Convención                                                                 |
|---------------------------|----------------------------------------------------------------------------|
| <code>controllers/</code> | Clases estáticas, métodos <code>handleXxx()</code>                         |
| <code>models/</code>      | Clases estáticas, métodos <code>getXxx()</code> , <code>createXxx()</code> |
| <code>services/</code>    | Clases con instancia única (singleton)                                     |

| Carpeta | Convención                                        |
|---------|---------------------------------------------------|
| hooks/  | Funciones que empiezan con <code>use</code>       |
| views/  | Componentes que terminan en <code>View.jsx</code> |
| styles/ | Mismo nombre que la vista + <code>.css</code>     |

### ✓ Conclusión del Punto 3:

El proyecto sigue una **arquitectura MVC** clara:

- **Vistas** = `views/` (interfaz)
- **Controladores** = `controllers/` (reglas de negocio)
- **Modelos** = `models/` (acceso a datos)

Además, usa **hooks personalizados** como puente entre el estado de React y los controladores.

### ¿Listo para el siguiente punto?

Escribe "siguiente" y te explicaré **código por código** desde el archivo más profundo hasta el más externo. 🔍

siguiente

## 📌 PUNTO 4: Código del Proyecto - Explicación paso a paso

Voy a explicarte el código desde el **archivo más profundo (servicios)** hasta el **más externo (App.jsx)**, entendiendo cómo se conectan todas las piezas.

## 🧩 CAPA 1: Servicios Base (La base técnica)

### 📄 `src/services/httpService.js` - Cliente HTTP

```
javascript

class HttpService {
  constructor() {
    this.baseUrl = API_CONFIG.BASE_URL // 'http://localhost:3000/api'
  }

  getToken() {
    return localStorage.getItem('auth_token')
  }

  getHeaders(includeAuth = true) {
    const headers = { 'Content-Type': 'application/json' }

    if (includeAuth) {
      const token = this.getToken()
      if (token) {
        headers['Authorization'] = `Bearer ${token}`
      }
    }

    return headers
  }
}
```

¿Qué hace?

- Es un cliente HTTP que envuelve `fetch`.
- Automáticamente agrega el token JWT a las peticiones si `includeAuth = true`.
- Centraliza la lógica de comunicación con la API.

#### Método POST:

```
javascript

async post(endpoint, data, includeAuth = true) {
  const url = `${this.baseUrl}${endpoint}`
  const response = await fetch(url, {
    method: 'POST',
    headers: this.getHeaders(includeAuth),
    body: JSON.stringify(data)
  })
  return await this.handleResponse(response)
}
```

#### Método handleResponse:

```
javascript

async handleResponse(response) {
  let data = await response.json()

  if (!response.ok) {
    throw new Error(data.message || `Error HTTP: ${response.status}`)
  }

  return data // Retorna toda la respuesta { success, data, message }
}
```

### **src/services/storageService.js - Manejo de almacenamiento**

```
javascript

class StorageService {
  constructor() {
    this.tokenKey = 'auth_token'
    this.userKey = 'user_data'
    this.roleKey = 'user_role'
  }

  setToken(token) {
    localStorage.setItem(this.tokenKey, token)
  }

  getToken() {
    return localStorage.getItem(this.tokenKey)
  }

  setUser(user) {
    localStorage.setItem(this.userKey, JSON.stringify(user))
  }

  getUser() {
    const user = localStorage.getItem(this.userKey)
    return user ? JSON.parse(user) : null
  }

  clearSession() {
    this.removeToken()
    this.removeUser()
    this.removeUserRole()
  }
}
```

## ¿Qué hace?

- Abstrae el `localStorage` para guardar/leer:
  - `auth_token` → Token JWT
  - `user_data` → Objeto con id, email, name, role
  - `user_role` → Rol del usuario (para permisos rápidos)

### `src/services/jwtService.js` - Manejo de JWT

```
javascript

class JWTService {
  verifyToken(token) {
    try {
      const parts = token.split('.')
      if (parts.length !== 3) return false

      const payload = JSON.parse(atob(parts[1]))

      // Verificar expiración con tolerancia de 5 minutos
      if (payload.exp) {
        const now = Date.now()
        const expTime = payload.exp * 1000
        const tolerance = 5 * 60 * 1000 // 5 minutos

        if (expTime + tolerance < now) {
          return false // Token expirado
        }
      }
      return true
    } catch (error) {
      return false
    }
  }

  decodeToken(token) {
    const parts = token.split('.')
    return JSON.parse(atob(parts[1]))
  }
}
```

## ¿Qué hace?

- No genera tokens reales (el backend lo hace con JWT).
- Solo verifica que el token no esté expirado.
- Decodifica el payload para leer datos (como el rol).

## CAPA 2: Modelos (Comunicación con la API)

### `src/models/AuthModel.js` - Modelo de autenticación

```
javascript

static async login(credentials) {
  try {
    // Llamada a la API real
    const response = await httpService.post(
      API_CONFIG.ENDPOINTS.LOGIN, // '/users/login'
      {
        email: credentials.email,
        password: credentials.password
      },
    )
  }
}
```

```

    false // No requiere token (es público)
  )

  // Extraer datos de la respuesta
  const userDataFromApi = response.data
  const sessionToken = userDataFromApi.session_token // 'JWT eyJhbG...'

  // Limpiar el token (quitar 'JWT ' si existe)
  let token = sessionToken
  if (sessionToken && sessionToken.startsWith('JWT ')) {
    token = sessionToken.substring(4)
  }

  // Guardar en storage
  storageService.setToken(token)

  const userData = {
    id: userDataFromApi.id,
    email: userDataFromApi.email,
    name: userDataFromApi.name,
    lastname: userDataFromApi.lastname,
    role: userDataFromApi.role,
    phone: userDataFromApi.phone,
    image: userDataFromApi.image
  }

  storageService.setUser(userData)

  return {
    success: true,
    token: token,
    user: userData
  }
} catch (error) {
  return { success: false, error: error.message }
}
}

```

### ¿Qué hace?

- Envía email/contraseña al endpoint `/users/login`.
- Recibe `{ success: true, data: { session_token, id, email, name, role } }`
- Extrae el token, lo limpia y lo guarda en `localStorage`.
- Guarda los datos del usuario para acceso rápido.

### `src/models/UserModel.js` - Modelo de usuarios

```

javascript

static async getAllUsers() {
  try {
    // GET a /api/users (requiere token)
    const response = await httpService.get(API_CONFIG.ENDPOINTS.USERS, true)

    // La API devuelve { success: true, data: [...] }
    let usersArray = []
    if (response && response.data && Array.isArray(response.data)) {
      usersArray = response.data
    }

    return { success: true, data: usersArray }
  } catch (error) {
    return { success: false, error: error.message }
  }
}

static async deleteUser(id) {

```

```

try {
  const endpoint = API_CONFIG.ENDPOINTS.USER_DELETE.replace(':id', id)
  const response = await httpService.delete(endpoint, true)
  return { success: true, message: 'Usuario eliminado' }
} catch (error) {
  return { success: false, error: error.message }
}
}

```

¿Qué hace?

- `getAllUsers()` → Obtiene lista de usuarios (solo admin/seller).
- `deleteUser()` → Elimina usuario por ID (solo admin).
- **Importante:** Los modelos no verifican permisos, eso lo hace el **controlador**.

## ✖ CAPA 3: Controladores (Lógica de negocio)

### 📄 `src/controllers/AuthController.js` - Controlador de autenticación

```

javascript

class AuthController {
  static async handleLogin(credentials, onSuccess, onError) {
    try {
      // 1. Validaciones de negocio
      if (!credentials.email || !credentials.password) {
        onError('Por favor complete todos los campos')
        return
      }

      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
      if (!emailRegex.test(credentials.email)) {
        onError('Por favor ingrese un email válido')
        return
      }

      // 2. Llamar al modelo
      const result = await AuthModel.login(credentials)

      // 3. Procesar respuesta
      if (result.success) {
        onSuccess(result.user, result.token)
      } else {
        onError(result.error || 'Credenciales incorrectas')
      }
    } catch (error) {
      onError('Error al conectar con el servidor')
    }
  }

  static getUserRole() {
    const user = AuthModel.getCurrentUser()
    return user?.role || null
  }

  static hasRole(requiredRole) {
    const userRole = this.getUserRole()
    return userRole === requiredRole
  }
}

```

¿Qué hace?

- **Valida** los datos antes de enviarlos al modelo.



- Llama al modelo y procesa la respuesta.
- Provee métodos auxiliares como `hasRole()` para verificar permisos.

#### `src/controllers/UserController.js` - Controlador de usuarios

```
javascript

static async deleteUser(id, onSuccess, onError) {
  try {
    // Verificar permisos (SOLO admin puede eliminar)
    if (!AuthController.hasRole('admin')) {
      onError('No tiene permisos para eliminar usuarios. Se requiere rol de administrador.')
      return
    }

    // No permitir que un admin se elimine a sí mismo
    const currentUser = AuthController.getAuthState().currentUser
    if (currentUser && currentUser.id === parseInt(id)) {
      onError('No puede eliminar su propio usuario')
      return
    }

    const result = await UserModel.deleteUser(id)
    if (result.success) {
      onSuccess()
    } else {
      onError(result.error)
    }
  } catch (error) {
    onError('Error al eliminar usuario')
  }
}
```

#### ¿Qué hace?

- Verifica roles antes de ejecutar acciones sensibles.
- Reglas de negocio: Un admin no puede eliminarse a sí mismo.
- Centraliza toda la lógica de permisos.

## CAPA 4: Hooks Personalizados (Estado de React)

#### `src/hooks/useAuth.js` - Estado de autenticación

```
javascript

const useAuth = () => {
  const [isAuthenticated, setIsAuthenticated] = useState(false)
  const [currentUser, setCurrentUser] = useState(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    checkAuth() // Verificar si hay sesión al cargar
  }, [])

  const checkAuth = async () => {
    setLoading(true)
    const authState = AuthController.getAuthState()
    setIsAuthenticated(authState.isAuthenticated)
    setCurrentUser(authState.currentUser)
    setLoading(false)
  }
}
```

```

const login = async (credentials) => {
  return new Promise((resolve, reject) => {
    AuthController.handleLogin(
      credentials,
      (user, token) => {
        setIsAuthenticated(true)
        setCurrentUser(user)
        resolve({ success: true, user, token })
      },
      (error) => reject({ success: false, error })
    )
  })
}

const logout = async () => {
  return new Promise((resolve, reject) => {
    AuthController.handleLogout(
      () => {
        setIsAuthenticated(false)
        setCurrentUser(null)
        resolve({ success: true })
      },
      (error) => reject({ success: false, error })
    )
  })
}

return { isAuthenticated, currentUser, loading, login, logout }
}

```

### ¿Qué hace?

- **Mantiene el estado global** de autenticación.
- **Expone funciones** como `login()`, `logout()` a los componentes.
- **Se suscribe a cambios** en `localStorage` (multi-tab).

### `src/hooks/useUsers.js` - Estado de usuarios

javascript

```

const useUsers = () => {
  const [users, setUsers] = useState([])
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState(null)

  const loadUsers = async () => {
    setLoading(true)
    UserController.getAllUsers(
      (data) => {
        setUsers(data)
        setLoading(false)
      },
      (err) => {
        setError(err)
        setLoading(false)
      }
    )
  }

  const deleteUser = async (id) => {
    return new Promise((resolve, reject) => {
      UserController.deleteUser(
        id,
        () => {
          loadUsers() // Recargar lista después de eliminar
          resolve({ success: true })
        },

```

```

      (err) => reject({ success: false, error: err })
    )
  })
}

useEffect(() => {
  loadUsers() // Cargar usuarios al montar el hook
}, [])

return { users, loading, error, loadUsers, deleteUser, createUser, updateUser }
}

```

¿Qué hace?

- Carga automática de usuarios al usarse.
- Recarga la lista después de crear, editar o eliminar.
- Maneja estados de carga y error.

## 🌱 CAPA 5: Vistas (Interfaz de usuario)

### 📄 src/views/auth/LoginView.jsx - Pantalla de login

```

javascript

const LoginView = () => {
  const [credentials, setCredentials] = useState({ email: '', password: '' })
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')

  const { login } = useAuth()
  const navigate = useNavigate()

  const handleSubmit = async (e) => {
    e.preventDefault()

    setLoading(true)
    try {
      await login(credentials)
      navigate('/dashboard') // Redirigir al dashboard
    } catch (err) {
      setError(err.error || 'Error al iniciar sesión')
    } finally {
      setLoading(false)
    }
  }

  return (
    <div className="login-container">
      <div className="login-card">
        <form onSubmit={handleSubmit}>
          <input
            type="email"
            value={credentials.email}
            onChange={(e) => setCredentials({...credentials, email: e.target.valu
e})}
          />
          <input
            type="password"
            value={credentials.password}
            onChange={(e) => setCredentials({...credentials, password: e.target.va
lue})}
          />
          <button type="submit" disabled={loading}>
            {loading ? 'Iniciando...' : 'Iniciar Sesión'}
          </button>
        </form>
      </div>
    </div>
  )
}

```

```

    </div>
  </div>
)
}

```

¿Qué hace?

- Muestra formulario de email/contraseña.
- Usa el hook `useAuth()` para llamar a `login()`.
- Maneja estados locales de carga y error.
- Redirige a `/dashboard` si el login es exitoso.

## src/views/dashboard/UsersView.jsx - Tabla de usuarios

javascript

```

const UsersView = () => {
  const { users, loading, deleteUser, loadUsers } = useUsers()
  const { currentUser } = useAuth()
  const [searchTerm, setSearchTerm] = useState('')

  // Verificar permisos
  const canDelete = (user) => {
    return currentUser?.role === 'admin' && currentUser.id !== user.id
  }

  const handleDelete = async (user) => {
    if (window.confirm(`¿Eliminar a ${user.email}?`)) {
      await deleteUser(user.id)
    }
  }

  // Filtrar usuarios por búsqueda
  const filteredUsers = users.filter(user =>
    user.email.toLowerCase().includes(searchTerm.toLowerCase()) ||
    user.name.toLowerCase().includes(searchTerm.toLowerCase())
  )

  return (
    <div>
      <input
        type="text"
        placeholder="Buscar..."
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
      />

      <table>
        <thead>
          <tr><th>Email</th><th>Nombre</th><th>Rol</th><th>Acciones</th></tr>
        </thead>
        <tbody>
          {filteredUsers.map(user => (
            <tr key={user.id}>
              <td>{user.email}</td>
              <td>{user.name}</td>
              <td>{user.role}</td>
              <td>
                <button onClick={() => handleDelete(user)} disabled={!canDelete(us
er)}>

```

```
    </div>
  )
}
```

¿Qué hace?

- Usa `useUsers()` para obtener la lista de usuarios.
- Usa `useAuth()` para saber el rol del usuario actual.
- Filtra usuarios en tiempo real (búsqueda).
- Deshabilita botones según permisos (solo admin puede eliminar).

## ✚ CAPA 6: Componente Principal y Enrutamiento

### src/App.jsx - Componente raíz

```
javascript

import { Routes, Route, Navigate } from 'react-router-dom'
import useAuth from './hooks/useAuth'
import LoginView from './views/auth/LoginView'
import DashboardView from './views/dashboard/DashboardView'
import LoadingSpinner from './views/common/LoadingSpinner'

// Componente para proteger rutas
const ProtectedRoute = ({ children }) => {
  const { isAuthenticated, loading } = useAuth()

  if (loading) return <LoadingSpinner />
  if (!isAuthenticated) return <Navigate to="/login" replace />

  return children
}

function App() {
  return (
    <>
      <Navbar />
      <Routes>
        <Route path="/" element={<Navigate to="/login" />} />
        <Route path="/login" element={<LoginView />} />
        <Route path="/register" element={<RegisterView />} />
        <Route
          path="/dashboard/*"
          element={
            <ProtectedRoute>
              <DashboardView />
            </ProtectedRoute>
          }
        />
      </Routes>
    </>
  )
}
```

¿Qué hace?

- Define las rutas de la aplicación.
- Protege rutas sensibles con `ProtectedRoute`.
- Muestra `LoadingSpinner` mientras verifica autenticación.
- Redirige automáticamente al login si no hay sesión.

## src/main.jsx - Punto de entrada

```
javascript

import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './App.css'
import './styles/common.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)
```

### ¿Qué hace?

- Renderiza la aplicación en el DOM.
- Envuelve todo con `BrowserRouter` para el enrutamiento.
- Importa estilos globales.
- Usa `React.StrictMode` para detectar problemas.

## Flujo completo desde el usuario hasta la API

```
text

1. Usuario escribe email/contraseña y hace clic en "Iniciar Sesión"
   ↓
2. LoginView.jsx → handleSubmit() → useAuth.login()
   ↓
3. useAuth.js → AuthController.handleLogin()
   ↓
4. AuthController.js → valida datos → AuthModel.login()
   ↓
5. AuthModel.js → httpService.post('/users/login', {email, pass})
   ↓
6. httpService.js → fetch() a http://localhost:3000/api/users/login
   ↓
7. API (backend) → responde con { success: true, data: { session_token, user } }
   ↓
8. AuthModel.js → storageService.setToken(token) y storageService.setUser(user)
   ↓
9. AuthController.js → onSuccess(user, token)
   ↓
10. useAuth.js → setIsAuthenticated(true), setCurrentUser(user)
   ↓
11. LoginView.jsx → navigate('/dashboard')
   ↓
12. App.jsx → ProtectedRoute permite el acceso
   ↓
13. DashboardView.jsx → se renderiza
```

### ✅ Conclusión del Punto 4:

El código sigue una **arquitectura limpia en capas**:

- **Servicios** → Comunicación base y almacenamiento.
- **Modelos** → Peticiones a la API.
- **Controladores** → Lógica de negocio y permisos.

- **Hooks** → Estado de React.
- **Vistas** → Interfaz de usuario.

**Principio clave:** Las vistas **nunca** llaman directamente a los modelos, siempre pasan por controladores y hooks.

¿Listo para el siguiente punto?

Escribe "**siguiente**" y te explicaré **cómo poner en marcha todo el sistema (frontend + backend)** y qué esperar al ejecutarlo. 🚀

Antes de continuar, explicame ahora todo el código desde el main.tsx hasta los servicios. Muéstrame la responsabilidad main.tsx, luego me explicas todo el código hasta que escriba "siguiente" y así con el próximo archivo. Cuando terminemos, me explicas la puesta en marcha

Perfecto! Vamos a hacerlo **archivo por archivo**, desde el más externo hasta el más profundo. Tú me dices "**siguiente**" cuando quieras avanzar al siguiente archivo.

## 📄 ARCHIVO 1: `src/main.jsx` (Punto de entrada principal)

```
javascript

// src/main.jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
// Importar estilos globales
import './App.css'
import './styles/common.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)
```

## 🔗 Responsabilidad de `main.jsx`

Este es el **archivo más importante de entrada** de toda la aplicación React. Su responsabilidad es:

1. **Crear el punto de anclaje en el DOM** → Busca el elemento `<div id="root"></div>` en `index.html` y monta toda la aplicación ahí.
2. **Configurar el entorno global** → Envuelve la app con:
  - `<React.StrictMode>` → Detecta problemas potenciales (ciclos de vida obsoletos, efectos secundarios, etc.)
  - `<BrowserRouter>` → Habilita el enrutamiento (React Router DOM) para manejar las URL como `/login`, `/dashboard`
3. **Cargar estilos globales** → Importa `App.css` (reset básico) y `common.css` (estilos del dashboard) para que estén disponibles en toda la app.
4. **Montar el componente raíz** `<App />` → Todo lo demás vive dentro de `App`.

## 🔍 Explicación línea por línea

| Línea | Código                                                                       | Explicación                                                                        |
|-------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 1-3   | <code>import React from 'react' etc.</code>                                  | Importa las bibliotecas necesarias                                                 |
| 4     | <code>import { BrowserRouter } from 'react-router-dom'</code>                | Importa el enrutador para manejar navegación                                       |
| 5     | <code>import App from './App.jsx'</code>                                     | Importa el componente principal de la app                                          |
| 7-8   | <code>import './App.css'</code><br><code>import './styles/common.css'</code> | Estilos globales (afectan a toda la app)                                           |
| 10    | <code>ReactDOM.createRoot(document.getElementById('root'))</code>            | Encuentra <code>&lt;div id="root"&gt;</code> en HTML y crea un "root" de React 18+ |
| 11-15 | <code>.render(...)</code>                                                    | Renderiza todo el contenido dentro del root                                        |
| 12    | <code>&lt;React.StrictMode&gt;</code>                                        | Activa verificaciones adicionales (solo desarrollo)                                |
| 13    | <code>&lt;BrowserRouter&gt;</code>                                           | Provee contexto de enrutamiento a toda la app                                      |
| 14    | <code>&lt;App /&gt;</code>                                                   | El componente principal que contiene todas las pantallas                           |
| 16    | <code>);</code>                                                              | Cierra la función                                                                  |

## Relación con otros archivos

```

text

main.jsx
├─ index.html (proporciona <div id="root">)
├─ App.jsx (componente principal)
├─ App.css (estilos base)
└─ styles/common.css (estilos del dashboard)

```

## Puntos clave a recordar

- **No hay lógica de negocio aquí** → Solo configuración de arranque.
- **El orden de imports importa** → Los estilos globales deben ir ANTES que los estilos específicos de componentes para no sobrescribirse accidentalmente.
- **BrowserRouter vs HashRouter** → Usa `BrowserRouter` para URLs limpias (sin `#`), pero requiere configuración en el servidor para manejar rutas (Vite lo maneja automáticamente).

## Flujo de ejecución al cargar la página

```

text

1. El navegador carga index.html
2. index.html tiene <script type="module" src="/src/main.jsx">
3. main.jsx se ejecuta
4. Importa React, ReactDOM, BrowserRouter, App, estilos
5. Busca <div id="root"> en el DOM
6. Crea un root de React

```



7. Renderiza: StrictMode → BrowserRouter → App
8. ¡La app aparece en pantalla!

## 💡 Analogía para entenderlo

Imagina que `main.jsx` es como el **director de orquesta**:

- No toca ningún instrumento (no hay UI directa aquí).
- Se asegura de que todos los músicos estén en su lugar (componentes).
- Marca el ritmo inicial (monta la app en el DOM).
- Configura el escenario (estilos globales, enrutamiento).

¿Entendiste la responsabilidad de `main.jsx` ?

Escribe "siguiente" y te explicaré el **ARCHIVO 2:** `App.jsx` (el componente raíz con las rutas). 🚀

siguiente

## 📄 ARCHIVO 2: `src/App.jsx` (Componente raíz con enrutamiento)

```
javascript

// src/App.jsx
import { Routes, Route, Navigate } from 'react-router-dom'
import { ROUTES } from '../config/routes'
import LoginView from '../views/auth/LoginView'
import RegisterView from '../views/auth/RegisterView'
import DashboardView from '../views/dashboard/DashboardView'
import Navbar from '../views/common/Navbar'
import useAuth from '../hooks/useAuth'
import LoadingSpinner from '../views/common/LoadingSpinner'

const ProtectedRoute = ({ children }) => {
  const { isAuthenticated, loading } = useAuth()

  if (loading) {
    return <LoadingSpinner />
  }

  if (!isAuthenticated) {
    return <Navigate to={ROUTES.LOGIN} replace />
  }

  return children
}

function App() {
  return (
    <>
      <Navbar />
      <Routes>
        <Route path="/" element={<Navigate to="/login" />} />
        <Route path="/login" element={<LoginView />} />
        <Route path="/register" element={<RegisterView />} />
        <Route
          path="/dashboard/*"
          element={
            <ProtectedRoute>
              <DashboardView />
            </ProtectedRoute>
          }
        />
      </Routes>
    </>
  )
}
```

```
</>
)
}

export default App
```

## Responsabilidad de App.jsx

Este es el **componente principal** que organiza toda la interfaz de usuario. Su responsabilidad es:

1. **Definir el sistema de rutas** → Qué pantalla se muestra según la URL actual.
2. **Proteger rutas sensibles** → Solo usuarios autenticados pueden ver `/dashboard`.
3. **Mostrar la barra de navegación** → El `<Navbar />` aparece en todas las páginas (excepto dentro del dashboard por diseño).
4. **Manejar el estado de carga** → Muestra un spinner mientras verifica si hay sesión activa.
5. **Redirigir automáticamente** → Si alguien intenta entrar a una ruta protegida sin login, lo envía al login.

## Explicación línea por línea

### Imports (líneas 1-8)

| Línea | Código                                                                 | Explicación                                                                 |
|-------|------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1     | <pre>import { Routes, Route, Navigate } from 'react-router-dom'</pre>  | Componentes de React Router para manejar rutas y redirecciones              |
| 2     | <pre>import { ROUTES } from './config/routes'</pre>                    | Constantes con nombres de rutas (ej: <code>ROUTES.LOGIN = '/login'</code> ) |
| 3-5   | <pre>import LoginView, RegisterView, DashboardView</pre>               | Las pantallas/componentes principales de la app                             |
| 6     | <pre>import Navbar from './views/common/Navbar'</pre>                  | Barra de navegación que aparece en modo público                             |
| 7     | <pre>import useAuth from './hooks/useAuth'</pre>                       | Hook personalizado para saber si el usuario está logueado                   |
| 8     | <pre>import LoadingSpinner from './views/common/LoadingSpinner',</pre> | Componente visual de "cargando..."                                          |

### ProtectedRoute (líneas 10-20)

```
javascript

const ProtectedRoute = ({ children }) => {
  const { isAuthenticated, loading } = useAuth()

  if (loading) {
    return <LoadingSpinner />
  }
}
```

```

    if (!isAuthenticated) {
      return <Navigate to={ROUTES.LOGIN} replace />
    }

    return children
  }
}

```

### ¿Qué hace?

Es un **componente wrapper** que verifica autenticación antes de mostrar su contenido.

| Línea | Código                                                                                 | Explicación                                                                                       |
|-------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| 10    | <pre>const ProtectedRoute = ({   children }) =&gt; {</pre>                             | Recibe <code>children</code> (el contenido a proteger, ej: <code>&lt;DashboardView /&gt;</code> ) |
| 11    | <pre>const { isAuthenticated,   loading } = useAuth()</pre>                            | Obtiene estado de autenticación y si está verificando                                             |
| 13-15 | <pre>if (loading) return &lt;LoadingSpinner /&gt;</pre>                                | Mientras verifica, muestra un spinner (evita parpadeos)                                           |
| 17-19 | <pre>if (!isAuthenticated) return &lt;Navigate to={ROUTES.LOGIN}   replace /&gt;</pre> | Si NO está autenticado, redirige al login ( <code>replace</code> evita volver atrás)              |
| 21    | <pre>return children</pre>                                                             | Si está autenticado, muestra el contenido protegido                                               |

### Ejemplo de uso:

```

javascript

<ProtectedRoute>
  <DashboardView /> { /* Solo se muestra si hay sesión */}
</ProtectedRoute>

```

### Función App (líneas 23-39)

```

javascript

function App() {
  return (
    <>
      <Navbar />
      <Routes>
        <Route path="/" element={<Navigate to="/login" />} />
        <Route path="/login" element={<LoginView />} />
        <Route path="/register" element={<RegisterView />} />
        <Route
          path="/dashboard/*"
          element={
            <ProtectedRoute>
              <DashboardView />
            </ProtectedRoute>
          }
        />
      </Routes>
    </>
  )
}

```

| Línea | Código                                                                            | Explicación                                                                                |
|-------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 23    | <code>function App() {</code>                                                     | Declara el componente principal                                                            |
| 25    | <code>&lt;&gt; ... &lt;/&gt;</code>                                               | Fragmento React (no añade divs extras al DOM)                                              |
| 26    | <code>&lt;Navbar /&gt;</code>                                                     | Muestra la barra de navegación en todas las rutas públicas                                 |
| 27    | <code>&lt;Routes&gt;</code>                                                       | Contenedor de React Router que evalúa las rutas                                            |
| 28    | <code>&lt;Route path="/" element={&lt;Navigate to="/login" /&gt;} /&gt;</code>    | La raíz <code>/</code> redirige automáticamente a <code>/login</code>                      |
| 29    | <code>&lt;Route path="/login" element={&lt;LoginView /&gt;} /&gt;</code>          | URL <code>/login</code> muestra la pantalla de inicio de sesión                            |
| 30    | <code>&lt;Route path="/register" element={&lt;RegisterView /&gt;} /&gt;</code>    | URL <code>/register</code> muestra la pantalla de registro                                 |
| 31-37 | <code>&lt;Route path="/dashboard/*" ... /&gt;</code>                              | URL <code>/dashboard</code> o <code>/dashboard/cualquier/cosa</code> está <b>protegida</b> |
| 33-36 | <code>&lt;ProtectedRoute&gt;&lt;DashboardView /&gt;&lt;/ProtectedRoute&gt;</code> | Envuelve el dashboard con el protector de autenticación                                    |
| 39    | <code>export default App</code>                                                   | Permite que <code>main.jsx</code> importe este componente                                  |

🗺️ Mapa de rutas de la aplicación

| URL                               | Componente                 | ¿Protegida? | ¿Qué hace?                                  |
|-----------------------------------|----------------------------|-------------|---------------------------------------------|
| <code>/</code>                    | <code>Navigate</code>      | No          | Redirige a <code>/login</code>              |
| <code>/login</code>               | <code>LoginView</code>     | No          | Formulario de inicio de sesión              |
| <code>/register</code>            | <code>RegisterView</code>  | No          | Formulario de registro                      |
| <code>/dashboard</code>           | <code>DashboardView</code> | ✅ Sí        | Panel principal con gestión de usuarios     |
| <code>/dashboard/usuarios</code>  | <code>DashboardView</code> | ✅ Sí        | Sigue siendo el mismo componente (tiene tal |
| <code>/cualquier/otra/cosa</code> | (ninguna)                  | -           | React Router muestra página en blanco (404  |

🔄 Flujo de navegación

text

Usuario abre la app → URL: localhost:5173/

↓

`<Navigate to="/login" />` → Redirige a `/login`

↓

`<Route path="/login">` → Muestra `LoginView`

↓

Usuario ingresa credenciales correctas

↓

`useAuth.login()` cambia `isAuthenticated = true`

↓

```
navigate('/dashboard') programático
↓
<Route path="/dashboard/*"> → Evalúa ProtectedRoute
↓
ProtectedRoute: isAuthenticated = true → Muestra DashboardView
↓
¡Dashboard visible!
```

## 💡 Analogía para entenderlo

Imagina que `App.jsx` es como el **portero de un edificio**:

- `<Navbar />` → Es el cartel exterior que todos ven (en modo público).
- `<Routes>` → Es el mapa que dice qué piso visitar según qué botón presiones.
- `<Route path="/login">` → Es la recepción (acceso libre).
- `<ProtectedRoute>` → Es el guardia de seguridad que solo deja pasar a los que tienen credencial.
- `<DashboardView />` → Es la oficina privada (solo empleados).

## ⚠️ Puntos clave a recordar

| Concepto                                | Explicación                                                                                                                                    |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>path="/dashboard/*"</code>        | El <code>*</code> significa que cualquier subruta ( <code>/dashboard/usuarios</code> , <code>/dashboard/config</code> ) también está protegida |
| <code>&lt;Navigate replace /&gt;</code> | <code>replace</code> evita que el usuario presione "Atrás" y vuelva a la ruta protegida                                                        |
| <code>useAuth().loading</code>          | Es crucial para evitar un "flash" de redirección mientras se verifica el token                                                                 |
| <code>ROUTES</code> constante           | Usar <code>ROUTES.LOGIN</code> en lugar de <code>"/login"</code> escrito a mano evita errores de tipeo                                         |

## 📁 Relación con otros archivos

text

```
App.jsx
├─ useAuth.js (hook para saber si hay sesión)
├─ LoadingSpinner.jsx (muestra mientras verifica)
├─ Navbar.jsx (barra superior en modo público)
├─ LoginView.jsx (pantalla de login)
├─ RegisterView.jsx (pantalla de registro)
├─ DashboardView.jsx (pantalla principal protegida)
└─ config/routes.js (constantes de rutas)
```

¿Entendiste la responsabilidad de `App.jsx` y cómo protege las rutas?

Escribe "siguiente" y te explicaré el **ARCHIVO 3:** `config/routes.js` (las constantes de rutas). 📄

siguiente

## ARCHIVO 3: `src/config/routes.js` (Constantes de rutas)

```
javascript

// src/config/routes.js
export const ROUTES = {
  LOGIN: '/login',
  DASHBOARD: '/dashboard',
  HOME: '/'
}

export const PROTECTED_ROUTES = [ROUTES.DASHBOARD]
export const PUBLIC_ROUTES = [ROUTES.LOGIN, ROUTES.HOME]
```

### Responsabilidad de `routes.js`

Este archivo es pequeño pero poderoso. Su responsabilidad es:

1. **Centralizar todas las rutas de la aplicación** → Evita tener URLs escritas "a mano" en múltiples archivos.
2. **Clasificar rutas por tipo** → Separa qué rutas son públicas y cuáles requieren autenticación.
3. **Servir como fuente única de verdad** → Si una URL cambia, solo se modifica aquí, no en 10 archivos diferentes.
4. **Mejorar mantenibilidad** → Usar `ROUTES.LOGIN` es más seguro que escribir `"/login"` cada vez.

### Explicación línea por línea

| Línea | Código                                                                | Explicación                                         |
|-------|-----------------------------------------------------------------------|-----------------------------------------------------|
| 1     | <code>// src/config/routes.js</code>                                  | Comentario indicando la ubicación del archivo       |
| 2     | <code>export const ROUTES = {</code>                                  | Exporta un objeto constante con todas las rutas     |
| 3     | <code>  LOGIN: '/login',</code>                                       | Define la ruta para la pantalla de inicio de sesión |
| 4     | <code>  DASHBOARD: '/dashboard',</code>                               | Define la ruta para el panel principal (protegido)  |
| 5     | <code>  HOME: '/',</code>                                             | Define la ruta raíz (redirige al login)             |
| 6     | <code>}</code>                                                        | Cierra el objeto ROUTES                             |
| 8     | <code>export const PROTECTED_ROUTES = [ROUTES.DASHBOARD]</code>       | Array con rutas que requieren autenticación         |
| 9     | <code>export const PUBLIC_ROUTES = [ROUTES.LOGIN, ROUTES.HOME]</code> | Array con rutas de acceso público                   |

### ¿Por qué es importante centralizar las rutas?

## ❌ Mala práctica (rutas escritas a mano):

```
javascript

// En App.jsx
<Route path="/login" element={<LoginView />} />

// En Navbar.jsx
<Link to="/login">Iniciar Sesión</Link>

// En LoginView.jsx
navigate('/dashboard')

// En algún otro lugar
if (path === '/login') { ... }
```

**Problema:** Si decides cambiar `/login` a `/iniciar-sesion`, tienes que buscar y reemplazar en todos los archivos.

## ✅ Buena práctica (usando ROUTES):

```
javascript

// En App.jsx
import { ROUTES } from './config/routes'
<Route path={ROUTES.LOGIN} element={<LoginView />} />

// En Navbar.jsx
import { ROUTES } from './config/routes'
<Link to={ROUTES.LOGIN}>Iniciar Sesión</Link>

// En LoginView.jsx
import { ROUTES } from './config/routes'
navigate(ROUTES.DASHBOARD)
```

**Ventaja:** Solo cambias la ruta en un solo lugar (`routes.js`) y todo sigue funcionando.

## 📁 Cómo se usa en otros archivos

En `App.jsx` (enrutamiento principal):

```
javascript

import { ROUTES, PROTECTED_ROUTES } from './config/routes'

// En las rutas
<Route path={ROUTES.LOGIN} element={<LoginView />} />
<Route path={ROUTES.DASHBOARD} element={<ProtectedRoute><DashboardView /></ProtectedRoute>} />

// En redirecciones
if (!isAuthenticated) {
  return <Navigate to={ROUTES.LOGIN} replace />
}
```

En `Navbar.jsx` (enlaces de navegación):

```
javascript

import { ROUTES } from './config/routes'

<Link to={ROUTES.LOGIN} className="nav-link">
  Iniciar Sesión
</Link>
<Link to={ROUTES.REGISTER} className="nav-link">
```

```
Registrarse
</Link>
```

En `LoginView.jsx` (redirección post-login):

```
javascript

import { ROUTES } from '../config/routes'
import { useNavigate } from 'react-router-dom'

const navigate = useNavigate()

// Después de Login exitoso
navigate(ROUTES.DASHBOARD)
```

## Clasificación de rutas

| Ruta             | Valor      | Tipo      | ¿Requiere autenticación? |
|------------------|------------|-----------|--------------------------|
| ROUTES.HOME      | /          | Pública   | No (redirige al login)   |
| ROUTES.LOGIN     | /login     | Pública   | No                       |
| ROUTES.DASHBOARD | /dashboard | Protegida | ✓ Sí                     |

Arrays de clasificación:

```
javascript

PROTECTED_ROUTES = ['/dashboard']
PUBLIC_ROUTES = ['/login', '/']
```

Estos arrays podrían usarse en un futuro para:

- Mostrar/ocultar enlaces según autenticación.
- Prevenir que usuarios autenticados vayan al login.
- Manejar redirecciones inteligentes.

## Flujo típico usando ROUTES

```
text

1. Desarrollador decide agregar nueva ruta '/profile'
  ↓
2. Agrega a ROUTES: PROFILE: '/profile'
  ↓
3. Decide si es pública o protegida
  ↓
4. La agrega al array correspondiente
  ↓
5. En App.jsx: <Route path={ROUTES.PROFILE} element={...} />
  ↓
6. En Navbar.jsx: <Link to={ROUTES.PROFILE}>Mi Perfil</Link>
  ↓
7. ¡Todo funciona sin errores de tipeo!
```

## Analogía para entenderlo

Imagina que `routes.js` es como un **directorio telefónico de la empresa**:



- `ROUTES` → Es la guía que dice "Departamento de Ventas = extensión 101".
- `PROTECTED_ROUTES` → Es la lista de oficinas que requieren tarjeta de acceso.
- `PUBLIC_ROUTES` → Es la lista de áreas de acceso libre (recepción, cafetería).

Cuando alguien necesita llamar a ventas, no recuerda "101", sino que busca en el directorio. Si cambian la extensión, solo actualizas el directorio, no todos los teléfonos.

### ⚠️ Puntos clave a recordar

| Concepto              | Explicación                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------------|
| Centralización        | Todas las rutas viven en un solo archivo                                                        |
| Reusabilidad          | Cualquier componente puede importar <code>ROUTES</code>                                         |
| Mantenibilidad        | Cambiar una ruta = modificar 1 línea                                                            |
| Prevención de errores | El autocompletado del IDE ayuda a escribir <code>ROUTES.LOGIN</code> correctamente              |
| Escalabilidad         | Puedes agregar más rutas fácilmente:<br><code>PROFILE: '/profile', SETTINGS: '/settings'</code> |

### 🚀 Posibles mejoras futuras

```
javascript
// Podrías agregar más información sobre cada ruta
export const ROUTES = {
  LOGIN: { path: '/login', title: 'Iniciar Sesión', public: true },
  DASHBOARD: { path: '/dashboard', title: 'Dashboard', public: false, roles: ['admin', 'seller'] },
  HOME: { path: '/', title: 'Inicio', public: true }
}

// 0 rutas dinámicas
export const ROUTES = {
  USER_PROFILE: (id) => `/users/${id}/profile`,
  USER_EDIT: (id) => `/users/${id}/edit`
}
```

### 📁 Relación con otros archivos

```
text
config/routes.js
├─ App.jsx (usa ROUTES y PROTECTED_ROUTES para definir rutas)
├─ Navbar.jsx (usa ROUTES para enlaces)
├─ LoginView.jsx (usa ROUTES.DASHBOARD para redirigir)
├─ RegisterView.jsx (usa ROUTES.LOGIN para redirigir)
└─ Cualquier componente que necesite navegar (usa useNavigate + ROUTES)
```

¿Entendiste por qué es importante centralizar las rutas y cómo se usa?

Escribe "siguiente" y te explicaré el ARCHIVO 4: `config/api.js` (configuración de la API). 🌐

Me gustaría que me explicaras el código del archivo de acuerdo a la ejecución

¡Excelente pregunta! Tienes toda la razón. Vamos a explicar **cómo se ejecuta el código en orden cronológico**, desde que el usuario abre la app hasta que puede ver el dashboard. Así entenderás qué archivos se "activan" en cada momento.

## FLUJO DE EJECUCIÓN COMPLETO (Cronológico)

Imaginemos que un usuario abre la aplicación por primera vez, inicia sesión, y ve el dashboard.

## MOMENTO 0: El navegador carga la aplicación

### Paso 0.1: Se carga `index.html`

```
html

<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title>Sistema de Información Web</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

#### ¿Qué pasa aquí?

- El navegador encuentra `<div id="root"></div>` → será el contenedor de toda la app.
- El navegador ve `<script type="module" src="/src/main.jsx">` → descarga y ejecuta `main.jsx`.

## MOMENTO 1: Se ejecuta `main.jsx`

```
javascript

// main.jsx se ejecuta COMPLETAMENTE ahora
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './App.css'
import './styles/common.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)
```

Orden de ejecución línea por línea:

| Orden | Línea                                                | ¿Qué hace?                                                    |
|-------|------------------------------------------------------|---------------------------------------------------------------|
| 1     | <code>import React from 'react'</code>               | Carga la biblioteca React                                     |
| 2     | <code>import ReactDOM from 'react-dom/client'</code> | Carga ReactDOM para renderizar                                |
| 3     | <code>import { BrowserRouter }</code>                | Carga el enrutador                                            |
| 4     | <code>import App from './App.jsx'</code>             | <b>¡OJO!</b> Esto NO ejecuta App.jsx todavía, solo la importa |
| 5-6   | <code>import './App.css'</code>                      | Aplica estilos CSS globales                                   |
| 7     | <code>ReactDOM.createRoot(...).render(...)</code>    | <b>Aquí es donde TODO comienza</b>                            |

En el momento de `render()` :

1. Crea un "root" de React 18+.
2. Renderiza `<React.StrictMode>`.
3. Dentro, renderiza `<BrowserRouter>`.
4. Dentro, **finalmente ejecuta** `<App />`.

## 🕒 MOMENTO 2: Se ejecuta `App.jsx`

```
javascript
// App.jsx se ejecuta AHORA
import { Routes, Route, Navigate } from 'react-router-dom'
import { ROUTES } from './config/routes'
import LoginView from './views/auth/LoginView'
import RegisterView from './views/auth/RegisterView'
import DashboardView from './views/dashboard/DashboardView'
import Navbar from './views/common/Navbar'
import useAuth from './hooks/useAuth'
import LoadingSpinner from './views/common/LoadingSpinner'

const ProtectedRoute = ({ children }) => {
  // Esta función NO se ejecuta todavía, solo se define
  const { isAuthenticated, loading } = useAuth() // ⚠️ useAuth se ejecutará cuando se use ProtectedRoute
  // ...
}

function App() {
  // Esto se ejecuta AHORA
  return (
    <>
      <Navbar /> // ← Se ejecuta Navbar.jsx
      <Routes> // ← React Router empieza a evaluar rutas
        <Route path="/" element={<Navigate to="/login" />} />
        <Route path="/login" element={<LoginView />} />
        <Route path="/register" element={<RegisterView />} />
        <Route
          path="/dashboard/*"
          element={
            <ProtectedRoute> // ← ProtectedRoute se define, pero NO se ejecuta todavía (porque la URL no es /dashboard)
              <DashboardView />
            </ProtectedRoute>
          }
        />
      </Routes>
    </>
  )
}
```

```
)  
}
```

En este momento:

1. Se ejecuta `Navbar.jsx` (porque está fuera de las rutas, siempre visible).
2. React Router analiza la URL actual (ej: `http://localhost:5173/`).
3. Como la URL es `/`, encuentra  

```
<Route path="/" element={<Navigate to="/login" />}>.
```
4. Ejecuta el `Navigate` → cambia la URL a `/login`.

---

### MOMENTO 3: Se ejecuta `LoginView.jsx` (usuario ve el formulario)

```
javascript  
  
// LoginView.jsx  
import { useState } from 'react'  
import { useNavigate } from 'react-router-dom'  
import useAuth from '../../hooks/useAuth' // ← Importa el hook  
import AlertMessage from '../common/AlertMessage'  
  
const LoginView = () => {  
  // 🟢 ESTO SE EJECUTA AHORA (cuando React renderiza LoginView)  
  const [credentials, setCredentials] = useState({ email: '', password: '' })  
  const [loading, setLoading] = useState(false)  
  const [error, setError] = useState('')  
  
  const { login } = useAuth() // ← SE EJECUTA useAuth() AQUÍ  
  const navigate = useNavigate()  
  
  const handleSubmit = async (e) => {  
    // Esta función NO se ejecuta ahora, solo se define  
    // Se ejecutará cuando el usuario haga clic en el botón  
    e.preventDefault()  
    setLoading(true)  
    try {  
      await login(credentials) // ← Aquí se ejecuta login() del hook  
      navigate('/dashboard')  
    } catch (err) {  
      setError(err.error)  
    } finally {  
      setLoading(false)  
    }  
  }  
  
  return (  
    // Renderiza el formulario HTML  
    <div className="login-container">  
      <form onSubmit={handleSubmit}>  
        <input type="email" onChange={...} />  
        <input type="password" onChange={...} />  
        <button type="submit">Iniciar Sesión</button>  
      </form>  
    </div>  
  )  
}
```

En este momento, la pantalla muestra:

- Formulario de login (campos email, contraseña, botón)
  - El usuario **NO** ha hecho clic todavía.
-

## MOMENTO 4: Usuario escribe email y contraseña

```
javascript

// En LoginView.jsx, cada vez que el usuario escribe:
<input
  onChange={(e) => setCredentials({...credentials, email: e.target.value})}
/>

// setCredentials() hace que React:
// 1. Actualice el estado 'credentials'
// 2. RE-RENDERICE LoginView con el nuevo valor en el input
```

No hay llamadas a la API todavía.

## MOMENTO 5: Usuario hace clic en "Iniciar Sesión"

```
javascript

// Se ejecuta handleSubmit()
const handleSubmit = async (e) => {
  e.preventDefault() // Evita que el navegador recargue la página

  setLoading(true) // Cambia el estado → botón muestra "Cargando..."

  // 🚀 Llamada CRÍTICA: ejecuta login() del hook useAuth
  const result = await login(credentials)

  // Si llega aquí, el login fue exitoso
  navigate('/dashboard') // Cambia la URL a /dashboard
}
```

Dentro de `login(credentials)` (desde `useAuth.js`):

```
javascript

// useAuth.js
const login = async (credentials) => {
  return new Promise((resolve, reject) => {
    // Llama al controlador
    AuthController.handleLogin(
      credentials,
      (user, token) => {
        // Esto se ejecuta si el login es exitoso
        setIsAuthenticated(true)
        setCurrentUser(user)
        resolve({ success: true, user, token })
      },
      (error) => {
        // Esto se ejecuta si hay error
        reject({ success: false, error })
      }
    )
  })
}
```

## MOMENTO 6: Se ejecuta `AuthController.handleLogin()`

```
javascript

// AuthController.js
static async handleLogin(credentials, onSuccess, onError) {
  try {
    // 1. Validar que los campos no estén vacíos
```

```

if (!credentials.email || !credentials.password) {
  onError('Por favor complete todos los campos')
  return
}

// 2. Validar formato de email
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
if (!emailRegex.test(credentials.email)) {
  onError('Por favor ingrese un email válido')
  return
}

// 3. Llamada al MODELO para hacer la petición HTTP
const result = await AuthModel.login(credentials)

// 4. Procesar la respuesta
if (result.success) {
  onSuccess(result.user, result.token) // ← Esto vuelve a useAuth
} else {
  onError(result.error || 'Credenciales incorrectas')
}
} catch (error) {
  onError('Error al conectar con el servidor')
}
}

```

## MOMENTO 7: Se ejecuta `AuthModel.login()` (la petición HTTP real)

```

javascript

// AuthModel.js
static async login(credentials) {
  try {
    // Hace la petición POST a http://localhost:3000/api/users/Login
    const response = await httpService.post(
      API_CONFIG.ENDPOINTS.LOGIN, // '/users/Login'
      {
        email: credentials.email,
        password: credentials.password
      },
      false // No requiere token porque es público
    )

    // response = { success: true, data: { id, email, name, role, session_token } }
  }

  const userDataFromApi = response.data
  const sessionToken = userDataFromApi.session_token // "JWT eyJhbGciOiJIUzI1NiIs...
  // Limpiar el token (quitar "JWT " del inicio)
  let token = sessionToken
  if (sessionToken && sessionToken.startsWith('JWT ')) {
    token = sessionToken.substring(4)
  }

  // 📁 Guardar en LocalStorage (usando storageService)
  storageService.setToken(token)

  const userData = {
    id: userDataFromApi.id,
    email: userDataFromApi.email,
    name: userDataFromApi.name,
    lastname: userDataFromApi.lastname,
    role: userDataFromApi.role,
    phone: userDataFromApi.phone,
    image: userDataFromApi.image
  }
}

```

```

    }

    storageService.setUser(userData) // Guarda usuario en LocalStorage

    // Retorna el éxito al controlador
    return {
      success: true,
      token: token,
      user: userData
    }

  } catch (error) {
    return {
      success: false,
      error: error.message || 'Error de conexión con el servidor'
    }
  }
}
}

```

En este momento:

- Se hizo `fetch()` a la API real.
- El backend verificó email/contraseña.
- El backend generó un JWT.
- Se guardó el token y usuario en `localStorage`.

## MOMENTO 8: El controlador vuelve al hook, y el hook actualiza el estado

```

javascript

// useAuth.js (continuación)
// Dentro de Login(), el callback onSuccess se ejecuta:
(user, token) => {
  setIsAuthenticated(true) // ← Esto hace que React RE-RENDERICE componentes que
  // usen useAuth
  setCurrentUser(user) // ← Guarda el usuario en el estado
  resolve({ success: true, user, token })
}

// LoginView.jsx recibe el resolve:
await login(credentials) // Esto termina exitosamente
navigate('/dashboard') // ← Cambia la URL a /dashboard

```

## MOMENTO 9: La URL cambia a `/dashboard`, React Router re-evalúa las rutas

```

javascript

// App.jsx se RE-EJECUTA (parcialmente) porque cambió la URL
function App() {
  return (
    <>
      <Navbar /> // Navbar se vuelve a renderizar
      <Routes>
        // ... otras rutas
        <Route
          path="/dashboard/*"
          element={
            <ProtectedRoute> // ← AHORA SÍ, la URL es /dashboard, así que ESTO S
E EJECUTA
              <DashboardView />
            </ProtectedRoute>

```

```

    }
  />
</Routes>
</>
)
}

```

## MOMENTO 10: Se ejecuta ProtectedRoute

javascript

```

const ProtectedRoute = ({ children }) => {
  // 🔥 useAuth se ejecuta AHORA
  const { isAuthenticated, loading } = useAuth()

  // isAuthenticated = true (porque el Login lo cambió)
  // loading = false (ya terminó de verificar)

  if (loading) {
    return <LoadingSpinner /> // No entra aquí porque loading = false
  }

  if (!isAuthenticated) {
    return <Navigate to="/login" replace /> // No entra aquí porque isAuthenticated = true
  }

  return children // ← Retorna <DashboardView />
}

```

## 🕒 MOMENTO 11: Se ejecuta DashboardView.jsx

javascript

```

// DashboardView.jsx
import { useState } from 'react'
import useAuth from '../hooks/useAuth'
import UsersView from './UsersView'

const DashboardView = () => {
  const { currentUser } = useAuth() // Obtiene el usuario logueado
  const [activeTab, setActiveTab] = useState('dashboard')

  // Renderiza el dashboard con tabs
  return (
    <div>
      <DashboardHeader user={currentUser} />
      {activeTab === 'dashboard' ? <DashboardStats /> : <UsersView />}
    </div>
  )
}

```

En este momento, el usuario ve el dashboard.

Si hace clic en la pestaña "Usuarios":

javascript

```

// Se ejecuta setActiveTab('users')
// React re-renderiza DashboardView
// Ahora muestra <UsersView /> en lugar de <DashboardStats />

```



 **MOMENTO 12: Se ejecuta `UsersView.jsx` (cuando se cambia a la pestaña Usuarios)**

```
javascript

// UsersView.jsx
import useUsers from '../hooks/useUsers'

const UsersView = () => {
  // 🕒 useUsers se ejecuta AHORA
  const { users, loading, deleteUser, loadUsers } = useUsers()

  // El hook useUsers, en su useEffect, llama a loadUsers() automáticamente
  // loadUsers() hace: UserController.getAllUsers() → UserModel.getAllUsers() → GET a /api/users

  // Muestra la tabla con los usuarios
  return (
    <table>
      {users.map(user => ( ... ))}
    </table>
  )
}
```

 **RESUMEN DEL ORDEN DE EJECUCIÓN**

| Orden | Archivo           | ¿Cuándo se ejecuta?                                    |
|-------|-------------------|--------------------------------------------------------|
| 1     | index.html        | El navegador lo carga primero                          |
| 2     | main.jsx          | Inmediatamente después (por el script tag)             |
| 3     | App.jsx           | Cuando main.jsx hace render(<App />)                   |
| 4     | Navbar.jsx        | Se ejecuta porque App.jsx lo renderiza siempre         |
| 5     | routes.js         | Se importa en App.jsx, se usa para definir rutas       |
| 6     | LoginView.jsx     | Cuando la URL es / o /login                            |
| 7     | useAuth.js        | Cuando LoginView ejecuta const { login } = useAuth()   |
| 8     | AuthController.js | Cuando LoginView ejecuta login(credentials)            |
| 9     | AuthModel.js      | Cuando AuthController llama a AuthModel.login()        |
| 10    | httpService.js    | Cuando AuthModel llama a httpService.post()            |
| 11    | storageService.js | Cuando AuthModel guarda el token/usuario               |
| 12    | DashboardView.jsx | Después del login exitoso, cuando la URL es /dashboard |
| 13    | useUsers.js       | Cuando DashboardView monta <UsersView />               |
| 14    | UserController.js | Cuando useUsers ejecuta loadUsers()                    |

| Orden | Archivo      | ¿Cuándo se ejecuta?                                      |
|-------|--------------|----------------------------------------------------------|
| 15    | UserModel.js | Cuando UserController llama a<br>UserModel.getAllUsers() |

## 🎯 CONCLUSIÓN CLAVE

Los archivos NO se ejecutan todos al mismo tiempo.

- Algunos se ejecutan al cargar la app ( `main.jsx` , `App.jsx` ).
- Otros se ejecutan solo cuando el usuario interactúa ( `handleSubmit` , `login()` ).
- Otros se ejecutan solo cuando se renderiza un componente específico ( `UsersView.jsx` solo si se abre la pestaña Usuarios).

### ¿Entendiste el flujo de ejecución cronológica?

Escribe "siguiente" y te explicaré la puesta en marcha del frontend (comandos, configuración, qué esperar). 🚀

olvida el último prompt. Sigue la explicación de los archivos como la traíamos pero en el orden de ejecución por favor

¡Perfecto! Reiniciamos. Ahora voy a explicarte los archivos en el orden en que se ejecutan cronológicamente, desde que el navegador carga la app hasta que el usuario ve el dashboard.

## 🚀 ORDEN DE EJECUCIÓN - FRONTEND REACT

### 📄 ARCHIVO 1: `index.html` (PRIMERO en ejecutarse)

```
html
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Sistema de Información Web con React + MVC + JWT" />
    <title>Sistema de Información Web</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

## 🎯 Responsabilidad en la ejecución

Este es el primer archivo que el navegador carga. Su responsabilidad es:

1. Proveer el contenedor principal → `<div id="root"></div>` donde React montará toda la aplicación.
2. Cargar el punto de entrada JavaScript → `<script type="module" src="/src/main.jsx">` le dice al navegador: "descarga y

ejecuta `main.jsx` ".

3. **Configurar metadata** → Título de la página, descripción, viewport (responsive design).

## Orden de ejecución línea por línea

| Orden | Código                                                        | ¿Qué hace el navegador?                                |
|-------|---------------------------------------------------------------|--------------------------------------------------------|
| 1     | <code>&lt;!DOCTYPE html&gt;</code>                            | Declara que es HTML5                                   |
| 2     | <code>&lt;html lang="es"&gt;</code>                           | Abre el documento HTML en español                      |
| 3     | <code>&lt;head&gt;</code>                                     | Comienza la sección de metadatos                       |
| 4-8   | <code>&lt;meta&gt;</code> tags                                | Configura caracteres, viewport, descripción            |
| 9     | <code>&lt;title&gt;</code>                                    | Cambia el título de la pestaña del navegador           |
| 10    | <code>&lt;/head&gt;</code>                                    | Cierra la sección de metadatos                         |
| 11    | <code>&lt;body&gt;</code>                                     | Comienza el cuerpo visible de la página                |
| 12    | <code>&lt;div id="root"&gt;&lt;/div&gt;</code>                | Crea un contenedor VACÍO (React lo llenará después)    |
| 13    | <code>&lt;script type="module" src="/src/main.jsx"&gt;</code> | <b>EJECUTA INMEDIATAMENTE</b><br><code>main.jsx</code> |
| 14-15 | <code>&lt;/body&gt;&lt;/html&gt;</code>                       | Cierra el documento                                    |

## Lo que NO pasa aquí

- No hay estilos CSS aplicados todavía (solo los que vienen por defecto del navegador).
- No hay componentes de React visibles.
- El usuario ve una página en blanco por milisegundos.

## Flujo en este momento

```
text

Navegador carga index.html
↓
Lee línea por línea
↓
Llega a <script src="/src/main.jsx">
↓
DESCARGA main.jsx (si no está en caché)
↓
EJECUTA main.jsx inmediatamente
↓
[CONTINÚA en Archivo 2]
```

¿Entendiste por qué `index.html` es el PRIMERO en ejecutarse?

Escribe "siguiente" y pasamos al ARCHIVO 2: `main.jsx` (el punto de entrada de

ARCHIVO 2: `src/main.jsx` (SEGUNDO en ejecutarse)

```
javascript

// src/main.jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
// Importar estilos globales
import './App.css'
import './styles/common.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)
```

Responsabilidad en la ejecución

Este archivo se ejecuta inmediatamente después de `index.html` porque el `<script>` lo invoca. Su responsabilidad es:

- 1. Importar todas las dependencias necesarias (React, ReactDOM, Router, componente App, estilos).
- 2. Encontrar el contenedor `<div id="root">` en el DOM.
- 3. Crear el "root" de React 18+ y renderizar la aplicación dentro de él.
- 4. Configurar el enrutador `<BrowserRouter>` para que toda la app pueda navegar entre páginas.
- 5. Aplicar estilos globales antes de que se vea cualquier componente.

Orden de ejecución línea por línea

| Orden | Línea                                                                          | ¿Qué hace JavaScript?                                                            |
|-------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| 1     | <code>import React from 'react'</code>                                         | Carga la biblioteca React en memoria                                             |
| 2     | <code>import ReactDOM from 'react-dom/client'</code>                           | Carga ReactDOM (para renderizar en el DOM real)                                  |
| 3     | <code>import { BrowserRouter } from 'react-router-dom'</code>                  | Carga el enrutador para manejar URLs                                             |
| 4     | <code>import App from './App.jsx'</code>                                       | NO ejecuta App.jsx todavía, solo la guarda para usarla después                   |
| 5-6   | <code>import './App.css' y</code><br><code>import './styles/common.css'</code> | Aplica estilos CSS globales al DOM (inyecta <code>&lt;style&gt;</code> o enlaza) |
| 7     | <code>ReactDOM.createRoot(document.getElementById('root'))</code>              | Busca <code>&lt;div id="root"&gt;</code> en el DOM real (el de index.html)       |

| Orden | Línea                                   | ¿Qué hace JavaScript?                                                                                                            |
|-------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 8     | <code>.render(</code>                   | Comienza a renderizar contenido dentro de ese root                                                                               |
| 9     | <code>&lt;React.StrictMode&gt;</code>   | Activa modo estricto (detecta problemas, solo desarrollo)                                                                        |
| 10    | <code>&lt;BrowserRouter&gt;</code>      | Crea el contexto de enrutamiento para toda la app                                                                                |
| 11    | <code>&lt;App /&gt;</code>              |  <b>AQUÍ se ejecuta App.jsx por primera vez</b> |
| 12-14 | Cierres <code>)</code> y <code>;</code> | Finaliza la llamada a render                                                                                                     |

 **Lo que pasa en este momento**

```
javascript

// En tiempo de ejecución REAL:

1. JavaScript descarga e importa React (toma unos milisegundos)

2. Aplica los CSS:
  - Lee App.css y common.css
  - Los inyecta en el <head> del documento
  - El usuario empieza a ver estilos aplicados

3. Busca <div id="root"> en el DOM:
  - Lo encuentra (porque index.html lo creó)
  - Prepara ese div para que React lo controle

4. Crea el "root" de React:
  - Es un objeto interno que React usa para saber qué renderizar

5. Renderiza StrictMode → BrowserRouter → App:
  - ¡App.jsx se ejecuta por primera vez!
```

 **Estado del DOM después de main.jsx**

Antes de main.jsx:

```
html

<body>
  <div id="root"></div>  <!-- VACÍO -->
</body>
```

Después de main.jsx:

```
html

<body>
  <div id="root">
    <!-- React ha inyectado todo el contenido de <App /> aquí -->
    <div>...</div>  <!-- Estructura interna de React -->
  </div>
</body>
```

 **Relación con otros archivos en este momento**

```
text
```

```
main.jsx (ejecutándose)
├─ index.html (ya cargado, provee <div id="root">)
├─ App.css (aplica estilos)
├─ common.css (aplica estilos)
└─ App.jsx (SE EJECUTA a continuación)
```

## 💡 Analogía para entenderlo

Imagina que `main.jsx` es como un **director de cine**:

- **Paso 1:** Trae a todos los actores (importa React, Router, etc.)
- **Paso 2:** Prepara el escenario (busca `<div id="root">`)
- **Paso 3:** Enciende las luces (aplica estilos CSS)
- **Paso 4:** Grita "¡Acción!" (ejecuta `.render()`)
- **Paso 5:** El actor principal `App.jsx` sale al escenario

## ⚠️ Puntos clave para recordar

| Concepto         | Explicación                                                                                                                                 |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Orden de imports | Los imports se resuelven ANTES de ejecutar el código. React, ReactDOM, y App.jsx ya están cargados cuando se llega a <code>.render()</code> |
| Estilos globales | Se importan UNA SOLA VEZ aquí. Los estilos de componentes (Login.css, Users.css) se importan en sus respectivos componentes                 |
| BrowserRouter    | Sin esto, no podrías usar <code>useNavigate()</code> , <code>&lt;Link&gt;</code> , o <code>useParams()</code> en ningún componente          |
| StrictMode       | Solo afecta desarrollo. En producción no existe. Hace que ciertos efectos se ejecuten dos veces para detectar problemas                     |

## 🐛 Posibles errores en este punto

| Error                                 | Causa                                                      | Solución                                       |
|---------------------------------------|------------------------------------------------------------|------------------------------------------------|
| Cannot find module './App.jsx'        | App.jsx no existe o la ruta es incorrecta                  | Verifica que App.jsx esté en <code>src/</code> |
| Target container is not a DOM element | No existe <code>&lt;div id="root"&gt;</code> en index.html | Agrega el div al HTML                          |
| Los estilos no se ven                 | Los archivos CSS no existen o la ruta es incorrecta        | Verifica que App.css y common.css existan      |

¿Entendiste por qué `main.jsx` es el SEGUNDO en ejecutarse y qué hace?

Escribe "siguiente" y pasamos al ARCHIVO 3: `App.jsx` (el componente principal que organiza las rutas). 📁

[siguiente](#)

## 📄 ARCHIVO 3: `src/App.jsx` (TERCERO en ejecutarse)

```

javascript

// src/App.jsx
import { Routes, Route, Navigate } from 'react-router-dom'
import { ROUTES } from './config/routes'
import LoginView from './views/auth/LoginView'
import RegisterView from './views/auth/RegisterView'
import DashboardView from './views/dashboard/DashboardView'
import Navbar from './views/common/Navbar'
import useAuth from './hooks/useAuth'
import LoadingSpinner from './views/common/LoadingSpinner'

const ProtectedRoute = ({ children }) => {
  const { isAuthenticated, loading } = useAuth()

  if (loading) {
    return <LoadingSpinner />
  }

  if (!isAuthenticated) {
    return <Navigate to={ROUTES.LOGIN} replace />
  }

  return children
}

function App() {
  return (
    <>
      <Navbar />
      <Routes>
        <Route path="/" element={<Navigate to="/login" />} />
        <Route path="/login" element={<LoginView />} />
        <Route path="/register" element={<RegisterView />} />
        <Route
          path="/dashboard/*"
          element={
            <ProtectedRoute>
              <DashboardView />
            </ProtectedRoute>
          }
        />
      </Routes>
    </>
  )
}

export default App

```

## Responsabilidad en la ejecución

Este archivo se ejecuta cuando `main.jsx` hace `<App />` dentro de `.render()`. Su responsabilidad es:

1. Definir el sistema de rutas de toda la aplicación.
2. Mostrar la barra de navegación `<Navbar />` en todas las páginas públicas.
3. Proteger rutas sensibles (solo accesibles con sesión activa).
4. Redirigir automáticamente según el estado de autenticación.
5. Manejar el estado de carga mientras se verifica la sesión.

## Orden de ejecución línea por línea

| Orden | Línea                                                                           | ¿Qué pasa en tiempo de ejecución?                                                                                                                                         |
|-------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-8   | <code>import ...</code>                                                         | Carga todos los componentes y hooks necesarios (NO se ejecutan todavía)                                                                                                   |
| 10    | <code>const ProtectedRoute = ({ children }) =&gt; {</code>                      | Define la función, pero NO la ejecuta (solo la guarda en memoria)                                                                                                         |
| 11    | <code>const { isAuthenticated, loading } = useAuth()</code>                     |  <b>SE EJECUTA</b> <code>useAuth()</code> cuando <code>ProtectedRoute</code> sea llamado |
| 12-19 | <code>if (loading)... if (!isAuthenticated)...</code>                           | Condicionales que se evaluarán cuando <code>ProtectedRoute</code> se ejecute                                                                                              |
| 21    | <code>function App() {</code>                                                   | Define el componente principal                                                                                                                                            |
| 22    | <code>return (</code>                                                           | Comienza a retornar lo que se va a renderizar                                                                                                                             |
| 23    | <code>&lt;&gt;</code>                                                           | Fragmento React (no crea nodo DOM extra)                                                                                                                                  |
| 24    | <code>&lt;Navbar /&gt;</code>                                                   |  <b>EJECUTA</b> <code>Navbar.jsx</code> inmediatamente                                   |
| 25    | <code>&lt;Routes&gt;</code>                                                     | Componente de React Router que evalúa la URL actual                                                                                                                       |
| 26    | <code>&lt;Route path="/" element={&lt;Navigate to="/login" /&gt; } /&gt;</code> | Si URL es <code>/</code> , ejecuta <code>Navigate</code> y cambia a <code>/login</code>                                                                                   |
| 27    | <code>&lt;Route path="/login" element={&lt;LoginView /&gt; } /&gt;</code>       | Si URL es <code>/login</code> ,  <b>EJECUTA</b> <code>LoginView.jsx</code>             |
| 28    | <code>&lt;Route path="/register" element={&lt;RegisterView /&gt; } /&gt;</code> | Si URL es <code>/register</code> , ejecuta <code>RegisterView</code>                                                                                                      |
| 29-35 | <code>&lt;Route path="/dashboard/*" element={&lt;ProtectedRoute&gt;...}</code>  | Si URL empieza con <code>/dashboard</code> , <b>EJECUTA</b> <code>ProtectedRoute</code>                                                                                   |
| 36-37 | <code>&lt;/&gt;&lt;/Routes&gt;</code>                                           | Cierra componentes                                                                                                                                                        |
| 38    | <code>}</code>                                                                  | Cierra función <code>App</code>                                                                                                                                           |
| 39    | <code>export default App</code>                                                 | Hace que <code>App</code> esté disponible para <code>main.jsx</code>                                                                                                      |

 **Escenario real: Usuario abre la app por primera vez (URL = `/`)**

javascript

// Flujo de ejecución EN VIVO:

1. `App()` se ejecuta
2. Renderiza `<Navbar />` → `Navbar.jsx` se ejecuta
3. Renderiza `<Routes>`
4. React Router mira la URL actual: `"http://localhost:5173/"`
5. Recorre las rutas en orden:
  - ¿`path="/"`?  **Sí**, coincide



- element={<Navigate to="/login" />} → **EJECUTA** Navigate

6. Navigate cambia la URL a "/login"
7. React Router detecta el cambio de URL
8. Vuelve a evaluar las rutas:
  - {path="/login"?} ☒ **Sí**, coincide
  - element={<LoginView />} → **EJECUTA** LoginView.jsx
9. El usuario ve el formulario de login

## ⚡ Escenario real: Usuario autenticado intenta ir a /dashboard

javascript

1. App() se ejecuta
2. Renderiza <Navbar />
3. Renderiza <Routes>
4. React Router mira URL: "http://localhost:5173/dashboard"
5. Recorre rutas:
  - path="/" → **No** (solo coincide exactamente "/")
  - path="/login" → **No**
  - path="/register" → **No**
  - path="/dashboard/\*" → ☒ **Sí** (el \* significa cualquier subruta)
6. element={<ProtectedRoute><DashboardView /></ProtectedRoute>}
7. **\*\*EJECUTA** ProtectedRoute({ children: <DashboardView /> })\*\*
  - Dentro de ProtectedRoute: llama a useAuth()
  - useAuth() lee localStorage y descubre que hay token válido
  - isAuthenticated = true, loading = false
  - No entra en if(loading)
  - No entra en if(!isAuthenticated)
  - return children → **EJECUTA** DashboardView.jsx
8. El usuario ve el dashboard

## 📊 Estado de ejecución en este momento

| Componente     | ¿Se ejecutó?                                  | ¿Por qué?                                    |
|----------------|-----------------------------------------------|----------------------------------------------|
| Navbar         | <input checked="" type="checkbox"/> <b>Sí</b> | Está fuera de <Routes>, siempre se renderiza |
| ProtectedRoute | Depende                                       | Solo si la URL es /dashboard/*               |
| LoginView      | Depende                                       | Solo si la URL es /login                     |
| RegisterView   | Depende                                       | Solo si la URL es /register                  |
| DashboardView  | Depende                                       | Solo si la URL es /dashboard Y hay sesión    |

## 🔄 Relación con otros archivos en este momento

text

App.jsx (ejecutándose)

- ├─ Navbar.jsx → SE EJECUTA (siempre)
- ├─ useAuth.js → SE EJECUTA (dentro de ProtectedRoute o donde se use)
- ├─ LoginView.jsx → SE EJECUTA (si URL = /login)
- ├─ RegisterView.jsx → SE EJECUTA (si URL = /register)
- ├─ DashboardView.jsx → SE EJECUTA (si URL = /dashboard Y autenticado)
- ├─ LoadingSpinner.jsx → SE EJECUTA (mientras loading = true)
- └─ config/routes.js → YA SE IMPORTÓ (las constantes están disponibles)

## 💡 Analogía para entenderlo

Imagina que App.jsx es un recepcionista de un edificio:

- <Navbar /> → Es el letrero que siempre está visible.
- <Routes> → Es el mapa que dice qué oficina está en cada piso.

- `<Route path="/login">` → Es la oficina de "Recepción" (acceso libre).
- `<ProtectedRoute>` → Es el guardia de seguridad que solo deja pasar a quienes tienen credencial.
- `<Navigate to="/login">` → Es el cartel que dice "Si no sabes a dónde ir, ve a Recepción".

Cuando alguien entra al edificio (abre la app), el recepcionista (App.jsx):

1. Muestra el letrero general (Navbar).
2. Mira en qué piso quiere ir el visitante (URL).
3. Lo guía a la oficina correspondiente (componente específico).

 **Puntos clave para recordar**

| Concepto                                   | Explicación                                                                                                                                        |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| El orden de las rutas importa              | React Router evalúa en orden. Pon las rutas más específicas PRIMERO (ej: <code>/dashboard/*</code> antes que <code>/dashboard</code> si existiera) |
| ProtectedRoute se ejecuta DENTRO de Routes | No es una ruta en sí misma, es un wrapper que decide si mostrar su contenido                                                                       |
| Navigate NO muestra nada visual            | Solo cambia la URL. Es como un <code>window.location.href</code> pero controlado por React                                                         |
| El asterisco <code>*</code> en path        | Significa "cualquier subruta".<br><code>/dashboard/usuarios</code> , <code>/dashboard/config</code> , etc., todas coinciden                        |

¿Entendiste por qué `App.jsx` es el TERCERO en ejecutarse y cómo decide qué mostrar según la URL?

Escribe "siguiente" y pasamos al ARCHIVO 4: `Navbar.jsx` (se ejecuta inmediatamente después de App.jsx). 

siguiente

 **ARCHIVO 4: `src/views/common/Navbar.jsx` (CUARTO en ejecutarse)**

```
javascript
// src/views/common/Navbar.jsx
import { Link, useNavigate, useLocation } from 'react-router-dom'
import useAuth from '../hooks/useAuth'
import '../styles/Navbar.css'

const Navbar = () => {
  const { isAuthenticated, logout, currentUser } = useAuth()
  const navigate = useNavigate()
  const location = useLocation()

  const handleLogout = async () => {
    await logout()
    navigate('/login')
  }

  // No mostrar navbar en dashboard
```

```

if (location.pathname.includes('/dashboard')) {
  return null
}

return (
  <nav className="navbar">
    <div className="navbar-container">
      <div className="navbar-logo">
        <Link to="/">
          <h2>Sistema Web</h2>
        </Link>
      </div>

      <div className="navbar-menu">
        {!isAuthenticated ? (
          <>
            <Link to="/login" className="nav-link">
              Iniciar Sesión
            </Link>
            <Link to="/register" className="nav-link btn-primary">
              Registrarse
            </Link>
          </>
        ) : (
          <>
            <span className="user-welcome">
              Hola, {currentUser?.name || currentUser?.email}
            </span>
            <Link to="/dashboard" className="nav-link">
              Dashboard
            </Link>
            <button onClick={handleLogout} className="nav-link btn-logout">
              Cerrar Sesión
            </button>
          </>
        )}
      </div>
    </div>
  </nav>
)
}

export default Navbar

```

## 🔧 Responsabilidad en la ejecución

`Navbar.jsx` se ejecuta inmediatamente después de que `App.jsx` lo renderiza (porque está fuera de `<Routes>`). Su responsabilidad es:

1. **Mostrar una barra de navegación diferente** según si el usuario está autenticado o no.
2. **Ocultarse automáticamente** cuando el usuario está en el dashboard (para evitar duplicación, porque el dashboard tiene su propio header).
3. **Proveer enlaces de navegación** (login, registro, dashboard).
4. **Manejar el cierre de sesión** con el botón "Cerrar Sesión".
5. **Mostrar el nombre del usuario** cuando está autenticado.

## 🔍 Orden de ejecución línea por línea

| Orden | Línea                                                                         | ¿Qué pasa en tiempo de ejecución?  |
|-------|-------------------------------------------------------------------------------|------------------------------------|
| 1-3   | <pre>import { Link, useNavigate, useLocation } from 'react- router-dom'</pre> | Importa herramientas de navegación |

| Orden | Línea                                                                           | ¿Qué pasa en tiempo de ejecución?                                                                                                                            |
|-------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4     | <code>import useAuth from<br/>'../../hooks/useAuth'</code>                      | Importa el hook de autenticación                                                                                                                             |
| 5     | <code>import<br/>'../../styles/Navbar.css'</code>                               | Aplica estilos específicos del navbar                                                                                                                        |
| 7     | <code>const Navbar = () =&gt; {</code>                                          | Define el componente                                                                                                                                         |
| 8     | <code>const { isAuthenticated,<br/>logout, currentUser } =<br/>useAuth()</code> |  <b>EJECUTA</b> <code>useAuth()</code> para obtener estado de autenticación |
| 9     | <code>const navigate = useNavigate()</code>                                     | Obtiene función para cambiar URL programáticamente                                                                                                           |
| 10    | <code>const location = useLocation()</code>                                     | Obtiene la URL actual para saber dónde está el usuario                                                                                                       |
| 12    | <code>const handleLogout = async ()<br/>=&gt; {</code>                          | Define función (NO se ejecuta todavía)                                                                                                                       |
| 13    | <code>await logout()</code>                                                     | Se ejecutará SOLO cuando el usuario haga clic                                                                                                                |
| 14    | <code>navigate('/login')</code>                                                 | Se ejecutará después del logout                                                                                                                              |
| 17    | <code>if<br/>(location.pathname.includes('/<br/>dashboard')) {</code>           | <b>EVALÚA AHORA</b> si la URL contiene '/dashboard'                                                                                                          |
| 18    | <code>return null</code>                                                        | Si está en dashboard, NO RENDERIZA NADA (se oculta)                                                                                                          |
| 21    | <code>return (</code>                                                           | Si NO está en dashboard, renderiza el navbar                                                                                                                 |
| 22-55 | Estructura HTML del navbar                                                      | Se muestra al usuario                                                                                                                                        |
| 24-26 | <code>&lt;Link to="/"&gt;Sistema<br/>Web&lt;/Link&gt;</code>                    | Enlace al inicio (redirige a <code>/</code> que luego va a <code>/login</code> )                                                                             |
| 33-49 | Renderizado condicional                                                         | Muestra botones según <code>isAuthenticated</code>                                                                                                           |

### Escenario real: Usuario NO autenticado, en la página `/login`

javascript

// Flujo de ejecución EN VIVO:

- Navbar se ejecuta
- `useAuth()` → `{ isAuthenticated: false, currentUser: null }`
- `location.pathname = '/login'`
- `pathname.includes('/dashboard')?` → `false`
- No entra en `if` → continúa
- Renderiza `return (...)`
- `isAuthenticated = false` → entra en `{!isAuthenticated ? (...) : (...)}`
- Muestra:
  - "Iniciar Sesión" (Link a `/login`)
  - "Registrarse" (Link a `/register`)
- El usuario ve estos botones en la barra superior

Lo que el usuario ve:

text

|             |                |             |
|-------------|----------------|-------------|
| Sistema Web | Iniciar Sesión | Registrarse |
|-------------|----------------|-------------|

## ⚡ Escenario real: Usuario autenticado, en la página `/login` (raro, pero posible)

javascript

1. Navbar se ejecuta
2. `useAuth()` → `{ isAuthenticated: true, currentUser: { name: "Oliver", email: "oliver@stone.com" } }`
3. `location.pathname = '/login'`
4. `isAuthenticated = true` → NO entra en `{!isAuthenticated}`
5. Entra en el `else {...}`
6. Muestra:
  - "Hola, Oliver"
  - "Dashboard" (Link a `/dashboard`)
  - "Cerrar Sesión" (botón)

Lo que el usuario ve:

text

|             |              |           |               |
|-------------|--------------|-----------|---------------|
| Sistema Web | Hola, Oliver | Dashboard | Cerrar Sesión |
|-------------|--------------|-----------|---------------|

## ⚡ Escenario real: Usuario autenticado, en `/dashboard`

javascript

1. Navbar se ejecuta
2. `useAuth()` → `{ isAuthenticated: true }`
3. `location.pathname = '/dashboard'`
4. `pathname.includes('/dashboard')? → true` ✓
5. `return null` // 🔥 EL NAVBAR NO SE RENDERIZA
6. El usuario NO ve el navbar (porque el dashboard tiene su propio header)

## 🔄 Flujo de `handleLogout` cuando el usuario hace clic

javascript

// El usuario hace clic en "Cerrar Sesión"

1. Se ejecuta `handleLogout()`
2. `await logout()` → Llama al hook `useAuth().logout()`
  - `useAuth().logout()` llama a `AuthController.handleLogout()`
  - `AuthController.handleLogout()` llama a `AuthModel.logout()`
  - `AuthModel.logout()` llama a `storageService.clearSession()`
  - Se borra token y usuario de `localStorage`
  - El estado `isAuthenticated` cambia a `false`
3. `navigate('/login')` → Cambia la URL a `/login`
4. React Router detecta cambio de URL
5. App.jsx re-evalúa rutas → muestra `LoginView`
6. Navbar se re-renderiza con `isAuthenticated = false`

## 🎨 Renderizado condicional explicado

javascript

```

{!isAuthenticated ? (
  // Usuario NO Logueado: muestra botones de acceso
  <>
    <Link to="/login">Iniciar Sesión</Link>
    <Link to="/register">Registrarse</Link>
  </>
) : (
  // Usuario Logueado: muestra bienvenida y botón de Logout
  <>
    <span>Hola, {currentUser?.name}</span>
    <Link to="/dashboard">Dashboard</Link>
    <button onClick={handleLogout}>Cerrar Sesión</button>
  </>
)}

```

| isAuthenticated | Lo que se muestra                        |
|-----------------|------------------------------------------|
| false           | Iniciar Sesión, Registrarse              |
| true            | "Hola, nombre", Dashboard, Cerrar Sesión |

## 💡 Analogía para entenderlo

Imagina que `Navbar.jsx` es un vigilante que cambia de uniforme según quién llega:

- **Si llega un visitante (no autenticado):**  
El vigilante muestra un cartel que dice "Regístrese" y "Inicie Sesión".
- **Si llega un empleado (autenticado):**  
El vigilante dice "Bienvenido, [nombre]", y muestra las opciones "Ir a la oficina" (Dashboard) y "Salir" (Cerrar Sesión).
- **Si el empleado ya está dentro de la oficina (dashboard):**  
El vigilante se esconde (return null) para no estorbar, porque dentro ya hay otro recepcionista (DashboardHeader).

## ⚠️ Puntos clave para recordar

| Concepto                                 | Explicación                                                                                                                                     |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>useLocation()</code>               | Hook que permite saber en qué URL está el usuario. Se actualiza automáticamente cuando cambia la ruta                                           |
| <code>return null</code>                 | Forma de React de decir "no renderices nada". El componente existe pero no produce HTML                                                         |
| <code>currentUser?.name</code>           | El <code>?</code> es optional chaining. Si <code>currentUser</code> es null, no intenta leer <code>.name</code> y no da error                   |
| <b>El navbar NO se ve en dashboard</b>   | Es intencional. El dashboard tiene <code>DashboardHeader</code> que cumple la misma función con un diseño diferente                             |
| <code>Link</code> vs <code>button</code> | <code>Link</code> es para navegación entre páginas.<br><code>button</code> es para acciones (como logout que ejecuta lógica antes de redirigir) |

## 🐛 Posibles errores en este punto

| Error                               | Causa                                                                               | Solución                                                         |
|-------------------------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Cannot read property 'name' of null | <code>currentUser</code> es null pero se intenta leer <code>currentUser.name</code> | Usa <code>currentUser?.name</code> (optional chaining)           |
| El navbar no se ve nunca            | <code>location.pathname.includes('/dashboard')</code> siempre es true               | Verifica que no estés siempre en rutas que contengan "dashboard" |
| El logout no redirige               | El <code>navigate</code> no se ejecuta porque <code>logout()</code> lanzó error     | Revisa la consola para ver el error real                         |

¿Entendiste por qué `Navbar.jsx` se ejecuta justo después de `App.jsx` y cómo decide qué mostrar?

Escribe "siguiente" y pasamos al ARCHIVO 5: `useAuth.js` (se ejecuta DENTRO de `Navbar` y `ProtectedRoute`). 🗝️

siguiente

## 📄 ARCHIVO 5: `src/hooks/useAuth.js` (QUINTO en ejecutarse)

```
javascript
// src/hooks/useAuth.js
import { useState, useEffect } from 'react'
import AuthController from '../controllers/AuthController'
import storageService from '../services/storageService'

const useAuth = () => {
  const [isAuthenticated, setIsAuthenticated] = useState(false)
  const [currentUser, setCurrentUser] = useState(null)
  const [loading, setLoading] = useState(true)
  const [token, setToken] = useState(null)

  useEffect(() => {
    checkAuth()

    const handleStorageChange = (e) => {
      if (e.key === 'auth_token') {
        checkAuth()
      }
    }

    window.addEventListener('storage', handleStorageChange)
    return () => window.removeEventListener('storage', handleStorageChange)
  }, [])

  const checkAuth = async () => {
    setLoading(true)
    const authState = AuthController.getAuthState()
    setIsAuthenticated(authState.isAuthenticated)
    setCurrentUser(authState.currentUser)
    setToken(authState.currentToken)
    setLoading(false)
  }

  const login = async (credentials) => {
    return new Promise((resolve, reject) => {
      AuthController.handleLogin(
        credentials,
        (user, authToken) => {
          setIsAuthenticated(true)
        }
      )
    })
  }
}
```

```

        setCurrentUser(user)
        setToken(authToken)
        resolve({ success: true, user, token: authToken })
      },
      (error) => {
        reject({ success: false, error })
      }
    )
  })
}

const register = async (userData) => {
  return new Promise((resolve, reject) => {
    AuthController.handleRegister(
      userData,
      (user) => {
        resolve({ success: true, user })
      },
      (error) => {
        reject({ success: false, error })
      }
    )
  })
}

const logout = async () => {
  return new Promise((resolve, reject) => {
    AuthController.handleLogout(
      () => {
        setIsAuthenticated(false)
        setCurrentUser(null)
        setToken(null)
        resolve({ success: true })
      },
      (error) => {
        reject({ success: false, error })
      }
    )
  })
}

return {
  isAuthenticated,
  currentUser,
  token,
  loading,
  login,
  register,
  logout,
  checkAuth
}
}

export default useAuth

```

## Responsabilidad en la ejecución

`useAuth.js` es un Hook personalizado de React que se ejecuta cada vez que un componente lo llama (Navbar, ProtectedRoute, LoginView, etc.). Su responsabilidad es:

1. **Mantener el estado global de autenticación** (¿hay sesión? ¿quién es el usuario? ¿está cargando?).
2. **Proveer funciones** para login, registro y logout que cualquier componente puede usar.
3. **Sincronizar el estado con localStorage** (si el usuario recarga la página, mantiene la sesión).



4. **Escuchar cambios en localStorage** (si abres otra pestaña y cierras sesión, esta pestaña se entera).
5. **Centralizar la lógica de autenticación** para que todos los componentes usen la misma fuente de verdad.

### Orden de ejecución línea por línea

| Orden | Línea                                                                    | ¿Qué pasa en tiempo de ejecución?                                                                                                     |
|-------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1     | <pre>import { useState, useEffect } from 'react'</pre>                   | Importa hooks de React                                                                                                                |
| 2     | <pre>import AuthController from '../controllers/AuthController'</pre>    | Importa el controlador (lógica de negocio)                                                                                            |
| 3     | <pre>import storageService from '../services/storageService'</pre>       | Importa servicio de almacenamiento                                                                                                    |
| 5     | <pre>const useAuth = () =&gt; {</pre>                                    | Define el hook personalizado                                                                                                          |
| 6-9   | <pre>const [isAuthenticated, setIsAuthenticated] = useState(false)</pre> | Crea 4 estados locales con valores iniciales                                                                                          |
| 11    | <pre>useEffect(() =&gt; {</pre>                                          | <b>SE EJECUTA después del primer renderizado</b>                                                                                      |
| 12    | <pre>checkAuth()</pre>                                                   | Llama a la función que verifica sesión                                                                                                |
| 14-17 | <pre>const handleStorageChange = ...</pre>                               | Define manejador para cambios en localStorage                                                                                         |
| 19-20 | <pre>window.addEventListener('storage', handleStorageChange)</pre>       | Escucha cambios de localStorage en OTRAS pestañas                                                                                     |
| 21    | <pre>return () =&gt; window.removeEventListener(... )</pre>              | Limpia el evento cuando el hook se desmonta                                                                                           |
| 22    | <pre>}, [])</pre>                                                        | Array vacío → useEffect se ejecuta UNA SOLA VEZ                                                                                       |
| 24    | <pre>const checkAuth = async () =&gt; {</pre>                            | Define función (se ejecuta al montar y cuando cambia storage)                                                                         |
| 25    | <pre>setLoading(true)</pre>                                              | Cambia estado a "cargando"                                                                                                            |
| 26    | <pre>const authState = AuthController.getAuthState()</pre>               |  <b>LLAMA AL CONTROLADOR</b> para verificar sesión |
| 27-29 | <pre>setIsAuthenticated(...)</pre>                                       | Actualiza los 3 estados con los valores del controlador                                                                               |
| 30    | <pre>setLoading(false)</pre>                                             | Termina estado de carga                                                                                                               |
| 32    | <pre>const login = async (credentials) =&gt; {</pre>                     | Define función (se ejecuta cuando el usuario hace clic en login)                                                                      |

| Orden | Línea                                                                                                    | ¿Qué pasa en tiempo de ejecución?                                               |
|-------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 33    | <code>return new Promise((resolve, reject) =&gt; {</code>                                                | Retorna una promesa para que LoginView pueda usar <code>await</code>            |
| 34    | <code>AuthController.handleLogin(...)</code>                                                             | 🔥 <b>LLAMA AL CONTROLADOR</b> con callbacks                                     |
| 36-38 | <code>(user, authToken) =&gt; { ... }</code>                                                             | <b>Callback de éxito</b> (se ejecuta cuando el login funciona)                  |
| 37    | <code>setIsAuthenticated(true)</code>                                                                    | Actualiza estado → React re-renderiza componentes que usan <code>useAuth</code> |
| 38    | <code>setCurrentUser(user)</code>                                                                        | Guarda usuario en estado                                                        |
| 39    | <code>setToken(authToken)</code>                                                                         | Guarda token en estado                                                          |
| 40    | <code>resolve({ success: true, user, token: authToken })</code>                                          | Resuelve la promesa → LoginView continúa                                        |
| 42-44 | <code>(error) =&gt; { reject(...) }</code>                                                               | <b>Callback de error</b> (se ejecuta si falla)                                  |
| 47-61 | <code>register</code> y <code>logout</code>                                                              | Similares a login, con su propia lógica                                         |
| 63    | <code>return { isAuthenticated, currentUser, token, loading, login, register, logout, checkAuth }</code> | 🔥 <b>EXPORTA</b> todo para que los componentes lo usen                          |

## ⚡ Escenario real: Un componente llama a `useAuth()` por primera vez

javascript

// Ejemplo: Navbar.jsx ejecuta `const { isAuthenticated, ... } = useAuth()`

1. `useAuth()` se ejecuta
2. Se crean los estados: `isAuthenticated = false`, `loading = true`
3. React registra el `useEffect` (pero NO lo ejecuta todavía)
4. `useAuth()` retorna `{ isAuthenticated: false, loading: true, ... }`
5. El **componente** (Navbar) recibe estos valores
6. React termina de renderizar Navbar por primera vez
7. **\*\*DESPUÉS** del renderizado\*\*, React ejecuta el `useEffect`
8. Dentro de `useEffect`: `checkAuth()` se ejecuta
9. `checkAuth()` llama a `AuthController.getAuthState()`
10. `AuthController` lee `localStorage` y determina si hay sesión
11. Se actualizan los estados: `setIsAuthenticated(true/false)`, `setLoading(false)`
12. React detecta que el estado cambió → **RE-RENDERIZA** Navbar con los nuevos valores

## 📺 Evolución del estado en el tiempo

### Momento 1: Primera ejecución de `useAuth()`

javascript

```
isAuthenticated = false
currentUser = null
loading = true
token = null
```

## Momento 2: Después de checkAuth() (sin sesión)

```
javascript

isAuthenticated = false
currentUser = null
loading = false // ← cambió
token = null
```

## Momento 3: Después de checkAuth() (CON sesión guardada)

```
javascript

isAuthenticated = true // ← cambió
currentUser = { id: 1, email: "user@test.com", name: "User" }
loading = false // ← cambió
token = "eyJhbGciOiJIUzI1NiIs..."
```

## Momento 4: Después de login() exitoso

```
javascript

isAuthenticated = true // ← cambió de false a true
currentUser = { id: 5, email: "oliver@stone.com", name: "Oliver" }
loading = false
token = "nuevo_token_del_backend"
```

## Flujo completo de login()

```
javascript

// En LoginView.jsx, el usuario hace clic:
await login({ email: "oliver@stone.com", password: "123456" })

// Dentro de useAuth.Login():
1. Crea una nueva Promise
2. Llama a AuthController.handleLogin(credentials, onSuccess, onError)
3. AuthController valida datos y llama a AuthModel.login()
4. AuthModel hace fetch a la API real
5. La API responde con token y usuario
6. Se ejecuta el callback onSuccess(user, token)
   - setIsAuthenticated(true) → Estado cambia
   - setCurrentUser(user) → Estado cambia
   - setToken(token) → Estado cambia
   - resolve() → La promesa se resuelve
7. LoginView recibe el resolve y ejecuta navigate('/dashboard')
```

## Analogía para entenderlo

Imagina que `useAuth` es un **centro de control de seguridad de un edificio**:

- **Estados** (`isAuthenticated`, `currentUser`, `loading`) → Son las pantallas que muestran:
  - "¿Hay alguien dentro?" (`isAuthenticated`)
  - "¿Quién está dentro?" (`currentUser`)
  - "¿Estamos revisando las credenciales?" (`loading`)
- `checkAuth()` → Es el guardia que cada mañana revisa la lista de empleados registrados (`localStorage`) para saber quién tiene acceso.
- `login(credentials)` → Es el proceso en la recepción: tomas tus datos, los verifican, y si son correctos, te dan una tarjeta de acceso (`token`) y te dejan entrar.

- `logout()` → Es cuando entregas tu tarjeta al salir: te dan de baja del sistema y ya no tienes acceso.
- El `useEffect` → Es como tener un walkie-talkie que te avisa si en otra entrada (otra pestaña) alguien cerró sesión, para que tú también salgas.

 **Comunicación entre componentes usando useAuth**

```
javascript

// Componente A (Navbar)
const { isAuthenticated, currentUser } = useAuth()
// Muestra "Hola, Oliver" si isAuthenticated = true

// Componente B (ProtectedRoute)
const { isAuthenticated, loading } = useAuth()
// Deja pasar a /dashboard solo si isAuthenticated = true

// Componente C (LoginView)
const { login } = useAuth()
// Cuando login() cambia isAuthenticated a true,
// NAVBAR Y PROTECTEDROUTE SE RE-RENDERIZAN AUTOMÁTICAMENTE
```

 **Puntos clave para recordar**

| Concepto                                                    | Explicación                                                                                                                                                                   |
|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>useEffect</code> con <code>[]</code> vacío            | El efecto se ejecuta UNA SOLA VEZ después del primer renderizado. Es perfecto para cargar datos iniciales (verificar sesión)                                                  |
| <code>window.addEventListener('storage')</code>             | Escucha cambios de <code>localStorage</code> en OTRAS pestañas del mismo dominio. Si cierras sesión en pestaña A, la pestaña B se entera y actualiza su UI                    |
| <code>new Promise</code>                                    | Permite que <code>LoginView</code> use <code>await login()</code> en lugar de callbacks anidados. Hace el código más legible                                                  |
| Los callbacks <code>onSuccess</code> y <code>onError</code> | El controlador no retorna valores directamente, sino que ejecuta estos callbacks cuando termina. Es un patrón asíncrono clásico                                               |
| <code>loading</code> es crucial                             | Sin <code>loading</code> , <code>ProtectedRoute</code> podría redirigir al login ANTES de que <code>checkAuth()</code> termine, causando un "flash" de redirección incorrecta |

 **Posibles errores en este punto**

| Error                                             | Causa                                                | Solución                                                                                            |
|---------------------------------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>loading</code> siempre es <code>true</code> | <code>checkAuth()</code> nunca termina o tiene error | Verifica que <code>AuthController.getAuthState()</code> no lance excepción                          |
| El estado no se actualiza después de login        | El callback <code>onSuccess</code> no se ejecuta     | Verifica que <code>AuthController.handleLogin()</code> llame a <code>onSuccess</code> correctamente |

| Error                                                | Causa                                                                     | Solución                                                                                                            |
|------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>isAuthenticated</code> no persiste al recargar | <code>checkAuth()</code> no se ejecuta o no lee <code>localStorage</code> | Verifica que <code>useEffect</code> se ejecute y que <code>storageService</code> tenga los datos                    |
| El componente se re-renderiza muchas veces           | Algo está cambiando el estado en un bucle infinito                        | Revisa dependencias de <code>useEffect</code> y que no llames a <code>setState</code> dentro de <code>render</code> |

¿Entendiste por qué `useAuth.js` se ejecuta desde múltiples componentes y cómo mantiene el estado sincronizado?

Escribe "siguiente" y pasamos al ARCHIVO 6: `AuthController.js` (se ejecuta cuando `useAuth` llama a sus métodos). 🐼

siguiente

## 📄 ARCHIVO 6: `src/controllers/AuthController.js` (SEXTO en ejecutarse)

```
javascript

// src/controllers/AuthController.js
import AuthModel from '../models/AuthModel'

class AuthController {
  // Manejar Login
  static async handleLogin(credentials, onSuccess, onError) {
    try {
      console.log('🐼 AuthController: Procesando login para:', credentials.email)

      // Validar que los campos no estén vacíos
      if (!credentials.email || !credentials.password) {
        onError('Por favor complete todos los campos')
        return
      }

      // Validar formato de email
      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
      if (!emailRegex.test(credentials.email)) {
        onError('Por favor ingrese un email válido')
        return
      }

      // Intentar Login con la API real
      const result = await AuthModel.login(credentials)

      console.log('📄 Resultado del login:', result)

      if (result.success) {
        // result.user ahora incluye: id, email, name, lastname, role, phone, image
        // result.token es el JWT real firmado por el backend
        onSuccess(result.user, result.token)
      } else {
        onError(result.error || 'Credenciales incorrectas')
      }
    } catch (error) {
      console.error('❌ Error en handleLogin:', error)
      onError('Error al conectar con el servidor')
    }
  }
}
```

```

// Manejar registro de usuario
static async handleRegister(userData, onSuccess, onError) {
  try {
    console.log('🔑 AuthController: Procesando registro para:', userData.email)

    // Validaciones
    if (!userData.email || !userData.password || !userData.name || !userData.lastName) {
      onError('Por favor complete todos los campos obligatorios (nombre, apellido, email, contraseña)')
      return
    }

    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
    if (!emailRegex.test(userData.email)) {
      onError('Por favor ingrese un email válido')
      return
    }

    if (userData.password.length < 5) {
      onError('La contraseña debe tener al menos 6 caracteres')
      return
    }

    const result = await AuthModel.register(userData)

    console.log('📋 Resultado del registro:', result)

    if (result.success) {
      onSuccess(result.user)
    } else {
      onError(result.error)
    }
  } catch (error) {
    console.error('❌ Error en handleRegister:', error)
    onError('Error al registrar usuario')
  }
}

// Manejar Logout
static async handleLogout(onSuccess, onError) {
  try {
    const result = await AuthModel.logout()
    if (result.success) {
      onSuccess()
    } else {
      onError(result.error || 'Error al cerrar sesión')
    }
  } catch (error) {
    onError('Error al cerrar sesión')
  }
}

// Obtener estado de autenticación
static getAuthState() {
  return {
    isAuthenticated: AuthModel.isAuthenticated(),
    currentUser: AuthModel.getCurrentUser(),
    currentToken: AuthModel.getCurrentToken()
  }
}

// Verificar autenticación (útil para rutas protegidas)
static async checkAuth() {
  const isValid = AuthModel.isAuthenticated()
  if (!isValid) {
    await AuthModel.logout()
  }
  return isValid
}

```

```

// Obtener el rol del usuario actual
static getUserRole() {
  const user = AuthModel.getCurrentUser()
  return user?.role || null
}

// Verificar si el usuario tiene un rol específico
static hasRole(requiredRole) {
  const userRole = this.getUserRole()
  return userRole === requiredRole
}

// Verificar si el usuario tiene alguno de los roles permitidos
static hasAnyRole(allowedRoles) {
  const userRole = this.getUserRole()
  return allowedRoles.includes(userRole)
}
}

export default AuthController

```

## Responsabilidad en la ejecución

`AuthController.js` es un **Controlador** que se ejecuta **cuando** `useAuth.js` **llama a sus métodos** (login, register, logout, getAuthState). Su responsabilidad es:

1. **Validar los datos del usuario** antes de enviarlos al modelo (email válido, contraseña no vacía, etc.).
2. **Orquestrar la comunicación** entre el hook (`useAuth`) y el modelo (`AuthModel`).
3. **Procesar las respuestas** del modelo y ejecutar los callbacks correspondientes (`onSuccess` o `onError`).
4. **Proveer métodos auxiliares** como `getUserRole()`, `hasRole()`, `hasAnyRole()` para verificar permisos.
5. **Centralizar la lógica de negocio** relacionada con autenticación.

## Orden de ejecución línea por línea (método `handleLogin`)

| Orden | Línea                                                                              | ¿Qué pasa en tiempo de ejecución?                            |
|-------|------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 1     | <code>import AuthModel from '../models/AuthModel'</code>                           | Importa el modelo (NO se ejecuta todavía)                    |
| 3     | <code>class AuthController {</code>                                                | Define la clase (sirve como contenedor de métodos estáticos) |
| 5     | <code>static async handleLogin(credentials, onSuccess, onError) {</code>           | Define método estático (se ejecuta cuando alguien lo llama)  |
| 6     | <code>try {</code>                                                                 | Inicia bloque de manejo de errores                           |
| 7     | <code>console.log(...)</code>                                                      | Muestra en consola que se está procesando el login           |
| 9-12  | <code>if (!credentials.email    !credentials.password) {<br/>onError(...) }</code> | <b>VALIDA</b> que los campos no estén vacíos                 |

| Orden | Línea                                                                                               | ¿Qué pasa en tiempo de ejecución?                                                                                           |
|-------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 14-17 | <pre>const emailRegex = ... if (!emailRegex.test(...)) {   onError(...) }</pre>                     | <b>VALIDA</b> formato de email                                                                                              |
| 19    | <pre>const result = await AuthModel.login(credentials)</pre>                                        |  <b>LLAMA AL MODELO</b> y espera respuesta |
| 21    | <pre>console.log('📄 Resultado del login:', result)</pre>                                            | Muestra resultado en consola                                                                                                |
| 23-28 | <pre>if (result.success) {   onSuccess(result.user,   result.token) } else {   onError(...) }</pre> | <b>EJECUTA CALLBACK</b> según éxito o error                                                                                 |
| 29-32 | <pre>catch (error) { onError('Error al conectar con el servidor') }</pre>                           | Captura errores de red o excepciones                                                                                        |

### ⚡ Escenario real: `useAuth.login()` llama a `AuthController.handleLogin()`

```
javascript

// Desde useAuth.js:
AuthController.handleLogin(
  credentials,                // { email: "oliver@stone.com", password: "123456" }
  (user, token) => {          // onSuccess - se ejecuta si todo sale bien
    setIsAuthenticated(true)
    setCurrentUser(user)
    resolve({ success: true, user, token })
  },
  (error) => {                // onError - se ejecuta si algo falla
    reject({ success: false, error })
  }
)

// DENTRO DE AuthController.handleLogin():

Paso 1: Validar campos
- credentials.email = "oliver@stone.com" ✓
- credentials.password = "123456" ✓

Paso 2: Validar formato email
- "oliver@stone.com" ✓ pasa la regex

Paso 3: Llamar al modelo
- await AuthModel.login(credentials)
- ⌚ Espera respuesta de la API (puede tomar 200-500ms)

Paso 4: Recibe resultado del modelo
- result = { success: true, user: {...}, token: "eyJhbG..." }

Paso 5: Ejecuta callback según resultado
- result.success = true → onSuccess(user, token)
- Esto vuelve a useAuth.js y ejecuta el callback que actualiza el estado
```

### 📄 Flujo de validaciones en `handleLogin`

```
javascript

// Escenario 1: Email vacío
credentials = { email: "", password: "123456" }
```



```

→ if (!credentials.email) → onError('Por favor complete todos los campos')
→ NO llega a llamar a AuthModel

// Escenario 2: Password vacío
credentials = { email: "test@test.com", password: "" }
→ if (!credentials.password) → onError('Por favor complete todos los campos')
→ NO llega a llamar a AuthModel

// Escenario 3: Email inválido
credentials = { email: "correo-invalido", password: "123456" }
→ emailRegex.test() = false → onError('Por favor ingrese un email válido')
→ NO llega a llamar a AuthModel

// Escenario 4: Todo válido
credentials = { email: "oliver@stone.com", password: "123456" }
→ Pasa todas las validaciones
→ Llama a AuthModel.login(credentials)
→ Espera respuesta de la API

```

## Método `getAuthState()` - el más usado

```

javascript

// Desde useAuth.checkAuth():
const authState = AuthController.getAuthState()

// Dentro de AuthController:
static getAuthState() {
  return {
    isAuthenticated: AuthModel.isAuthenticated(), // ← Lee LocalStorage
    currentUser: AuthModel.getCurrentUser(),       // ← Lee LocalStorage
    currentToken: AuthModel.getCurrentToken()      // ← Lee LocalStorage
  }
}

```

### ¿Por qué es importante?

Este método NO hace llamadas a la API, solo lee `localStorage`. Es **síncrono y rápido**, perfecto para verificar sesión al cargar la página.

## Métodos auxiliares para roles

```

javascript

// En cualquier componente, puedes hacer:
import AuthController from '../controllers/AuthController'

// Verificar si el usuario es admin
if (AuthController.hasRole('admin')) {
  // Mostrar botón de eliminar usuario
}

// Verificar si tiene alguno de estos roles
if (AuthController.hasAnyRole(['admin', 'seller'])) {
  // Mostrar panel de gestión de usuarios
}

```

## Responsabilidades por capa

| Capa                         | Responsabilidad                    | ¿Valida datos? | ¿Llama a API?          | ¿Maneja... |
|------------------------------|------------------------------------|----------------|------------------------|------------|
| Vista (LoginView)            | Mostrar UI, capturar eventos       | ✗ No           | ✗ No                   | ✗ No       |
| Hook (useAuth)               | Mantener estado, exponer funciones | ✗ No           | ✗ No                   | ✗ No       |
| Controlador (AuthController) | Validar, orquestar, procesar       | ✓ Sí           | ✗ No (llama al modelo) | ✗ No       |

| Capa               | Responsabilidad                | ¿Valida datos? | ¿Llama a API? | ¿Maneja errores? |
|--------------------|--------------------------------|----------------|---------------|------------------|
| Modelo (AuthModel) | Comunicación con API y storage | ❌ No           | ✅ Sí          | ✅ Sí             |

⚠️ **Puntos clave para recordar**

| Concepto               | Explicación                                                                                                                                                                                            |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Métodos estáticos      | No necesitas crear una instancia de <code>AuthController</code> . Llamas directamente <code>AuthController.handleLogin()</code>                                                                        |
| Patrón de callbacks    | En lugar de <code>return</code> un valor, el controlador recibe <code>onSuccess</code> y <code>onError</code> como parámetros. Esto es común en código asíncrono "callback hell" pero aquí se usa bien |
| Validación doble       | El frontend valida (email, campos vacíos) para evitar llamadas innecesarias a la API. El backend también debe validar por seguridad                                                                    |
| <code>try/catch</code> | Captura errores de red (servidor caído, timeout, etc.) y los convierte en mensajes amigables para el usuario                                                                                           |
| Separación de concerns | El controlador NO sabe cómo se guarda el token (eso es responsabilidad del modelo). Solo valida y orquesta                                                                                             |

🐛 **Posibles errores en este punto**

| Error                                                   | Causa                                                                 | Solución                                                                    |
|---------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>AuthModel.login is not a function</code>          | El modelo no exporta correctamente el método                          | Verifica que <code>AuthModel</code> tenga <code>static async login()</code> |
| El callback <code>onSuccess</code> no se ejecuta        | <code>result.success</code> es false o hay error                      | Revisa <code>console.log(result)</code> para ver qué devuelve el modelo     |
| Las validaciones rechazan emails válidos                | La regex es muy estricta                                              | Prueba con diferentes formatos de email                                     |
| <code>userData.password.length &lt; 5</code> (mínimo 6) | El mensaje dice "al menos 6" pero la condición es <code>&lt; 5</code> | Corregir mensaje o cambiar condición a <code>&lt; 6</code>                  |

¿Entendiste por qué `AuthController.js` actúa como un puente entre el hook y el modelo, validando datos y ejecutando callbacks?

Escribe "siguiente" y pasamos al ARCHIVO 7: `AuthModel.js` (se ejecuta cuando el controlador lo llama). 🦄

siguiente

📄 **ARCHIVO 7: `src/models/AuthModel.js` (SÉPTIMO en ejecutarse)**

```
javascript
```

```

// src/models/AuthModel.js
import httpService from '../services/httpService'
import storageService from '../services/storageService'
import jwtService from '../services/jwtService'
import API_CONFIG from '../config/api'

class AuthModel {
  static async login(credentials) {
    try {
      console.log('📧 Enviando login a API real:', credentials.email)
      console.log('🔑 URL completa:', `${API_CONFIG.BASE_URL}${API_CONFIG.ENDPOINT
S.LOGIN}`)

      const response = await httpService.post(
        API_CONFIG.ENDPOINTS.LOGIN,
        {
          email: credentials.email,
          password: credentials.password
        },
        false
      )

      console.log('📦 Respuesta COMPLETA del login:', JSON.stringify(response, nul
l, 2))

      // Verificar si la respuesta es exitosa
      if (!response.success) {
        console.error('❌ La API respondió con success=false:', response.message)
        return {
          success: false,
          error: response.message || 'Error en el servidor'
        }
      }

      // Los datos del usuario están en response.data
      const userDataFromApi = response.data

      if (!userDataFromApi) {
        console.error('❌ No se encontró response.data')
        return {
          success: false,
          error: 'Error en la respuesta del servidor: no hay datos'
        }
      }

      console.log('👤 userDataFromApi:', userDataFromApi)

      const sessionToken = userDataFromApi.session_token

      if (!sessionToken) {
        console.error('❌ No se encontró session_token')
        return {
          success: false,
          error: 'Error al obtener token de autenticación'
        }
      }

      let token = sessionToken
      if (sessionToken && sessionToken.startsWith('JWT ')) {
        token = sessionToken.substring(4)
      }

      console.log('✅ Token extraído correctamente')

      storageService.setToken(token)

      const userData = {
        id: userDataFromApi.id,
        email: userDataFromApi.email,
        name: userDataFromApi.name || userDataFromApi.email.split('@')[0],
        lastname: userDataFromApi.lastname || '',

```

```

        role: userDataFromApi.role || 'user',
        phone: userDataFromApi.phone || '',
        image: userDataFromApi.image || ''
    }

    storageService.setUser(userData)

    if (userData.role) {
        storageService.setUserRole(userData.role)
    }

    console.log('✅ Login exitoso, usuario guardado:', userData.email)

    return {
        success: true,
        token: token,
        user: userData
    }

} catch (error) {
    console.error('❌ Error en login:', error)
    return {
        success: false,
        error: error.message || 'Error de conexión con el servidor'
    }
}
}

static async register(userData) {
    try {
        console.log('📝 Registrando usuario:', userData.email)

        const userToCreate = {
            name: userData.name,
            lastname: userData.lastname || '',
            email: userData.email,
            password: userData.password,
            phone: userData.phone || '',
            image: userData.image || '',
            role: userData.role || 'user'
        }

        const response = await httpService.post(
            API_CONFIG.ENDPOINTS.REGISTER,
            userToCreate,
            false
        )

        console.log('📄 Respuesta registro:', response)

        if (!response.success) {
            return {
                success: false,
                error: response.message || 'Error al registrar usuario'
            }
        }

        const createdUser = response.data || response

        return {
            success: true,
            user: createdUser,
            message: 'Usuario registrado exitosamente'
        }

    } catch (error) {
        console.error('❌ Error en registro:', error)
        return {
            success: false,
            error: error.message || 'Error al registrar usuario'
        }
    }
}

```

```

    }
  }

  static async logout() {
    try {
      storageService.clearSession()
      return { success: true }
    } catch (error) {
      console.error('Error en logout:', error)
      return { success: false, error: error.message }
    }
  }

  static isAuthenticated() {
    const token = storageService.getToken()
    if (!token) return false

    const isValid = jwtService.verifyToken(token)

    if (!isValid) {
      storageService.clearSession()
      return false
    }

    return true
  }

  static getCurrentUser() {
    return storageService.getUser()
  }

  static getCurrentToken() {
    return storageService.getToken()
  }
}

export default AuthModel

```

## Responsabilidad en la ejecución

`AuthModel.js` es el **Modelo** que se ejecuta **cuando** `AuthController.js` **llama a sus métodos**. Su responsabilidad es:

1. **Comunicarse con la API real** a través de `httpService`.
2. **Procesar las respuestas** de la API y transformarlas en un formato estándar ( `{ success, data, error }` ).
3. **Guardar y leer datos** del `localStorage` usando `storageService`.
4. **Verificar la validez del token** usando `jwtService`.
5. **Aislar la lógica de persistencia** para que el controlador no sepa dónde se guardan los datos.

## Orden de ejecución línea por línea (método `login` )

| Orden | Línea                                          | ¿Qué pasa en tiempo de ejecución? |
|-------|------------------------------------------------|-----------------------------------|
| 1-4   | <code>import ...</code>                        | Importa servicios y configuración |
| 6     | <code>class AuthModel {</code>                 | Define la clase contenedora       |
| 8     | <code>static async login(credentials) {</code> | Define método estático asíncrono  |

| Orden | Línea                                                                                 | ¿Qué pasa en tiempo de ejecución?                                                                                                  |
|-------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 9     | <code>try {</code>                                                                    | Inicia bloque de manejo de errores                                                                                                 |
| 10-11 | <code>console.log(...)</code>                                                         | Logs de depuración                                                                                                                 |
| 13-19 | <code>const response = await<br/>httpService.post(...)</code>                         |  <b>LLAMA A LA API REAL</b><br>(espera respuesta) |
| 21    | <code>console.log(...)</code>                                                         | Muestra respuesta completa en consola                                                                                              |
| 23-29 | <code>if (!response.success) {<br/>return { success: false,<br/>error: ... } }</code> | Si la API devuelve error, retorna error formateado                                                                                 |
| 31-38 | <code>const userDataFromApi =<br/>response.data</code>                                | Extrae datos del usuario de la respuesta                                                                                           |
| 40-46 | <code>const sessionToken =<br/>userDataFromApi.session_token</code>                   | Extrae el token JWT de la respuesta                                                                                                |
| 48-52 | <code>let token = sessionToken</code>                                                 | Limpia el token (quita "JWT " si existe)                                                                                           |
| 54    | <code>storageService.setToken(token)</code>                                           |  <b>GUARDA TOKEN EN LOCALSTORAGE</b>              |
| 56-65 | <code>const userData = { ... }</code>                                                 | Construye objeto de usuario normalizado                                                                                            |
| 67    | <code>storageService.setUser(userData)</code>                                         |  <b>GUARDA USUARIO EN LOCALSTORAGE</b>          |
| 69-71 | <code>if (userData.role) {<br/>storageService.setUserRole(userData.role) }</code>     | Guarda rol por separado (para acceso rápido)                                                                                       |
| 75-79 | <code>return { success: true, token:<br/>token, user: userData }</code>               |  <b>RETORNA ÉXITO AL CONTROLADOR</b>            |
| 81-86 | <code>catch (error) { return {<br/>success: false, error: ... } }</code>              | Captura errores de red/excepciones                                                                                                 |

### **Escenario real: `AuthController.handleLogin()` llama a `AuthModel.login()`**

```

javascript

// Desde AuthController.js:
const result = await AuthModel.login(credentials)

// DENTRO DE AuthModel.Login():

Paso 1: Hacer petición HTTP a la API
- httpService.post('/users/login', { email, password }, false)
- URL: http://localhost:3000/api/users/login
- Método: POST
- Body: { "email": "oliver@stone.com", "password": "123456" }

Paso 2: Recibir respuesta de la API
// Respuesta exitosa del backend:
{
  "success": true,

```

```

    "message": "Usuario autenticado",
    "data": {
      "id": 5,
      "email": "oliver@stone.com",
      "name": "Oliver",
      "lastname": "Stone",
      "role": "admin",
      "phone": "123456789",
      "image": "",
      "session_token": "JWT_eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
    }
  }
}

```

Paso 3: Procesar respuesta

```

- response.success = true ✓
- userDataFromApi = response.data
- sessionToken = "JWT_eyJhbGciOiJIUzI1NiIs..."

```

Paso 4: Limpiar token

```

- token = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." (sin "JWT ")

```

Paso 5: Guardar en localStorage

```

- localStorage.setItem('auth_token', token)
- localStorage.setItem('user_data', JSON.stringify(userData))
- localStorage.setItem('user_role', 'admin')

```

Paso 6: Retornar al controlador

```

return {
  success: true,
  token: "eyJhbGciOiJIUzI1NiIs...",
  user: { id: 5, email: "oliver@stone.com", name: "Oliver", role: "admin", ... }
}

```

## Formato de la respuesta de la API (backend esperado)

javascript

```

// El backend DEBE responder con esta estructura:

// Éxito (200 OK):
{
  "success": true,
  "message": "Usuario autenticado",
  "data": {
    "id": 5,
    "email": "usuario@ejemplo.com",
    "name": "Nombre",
    "lastname": "Apellido",
    "role": "admin|seller|customer|user",
    "phone": "123456789",
    "image": "",
    "session_token": "JWT_eyJhbGciOiJIUzI1NiIs..."
  }
}

// Error (401 Unauthorized):
{
  "success": false,
  "message": "Contraseña o correo incorrecto"
}

```

## Método isAuthenticated() - verificación de sesión

javascript

```

static isAuthenticated() {
  // 1. Obtener token de LocalStorage
  const token = storageService.getToken()
}

```

```

if (!token) return false // No hay token → no autenticado

// 2. Verificar si el token es válido (no expirado)
const isValid = jwtService.verifyToken(token)

// 3. Si el token expiró, limpiar sesión automáticamente
if (!isValid) {
  storageService.clearSession()
  return false
}

return true
}

```

### ¿Cuándo se ejecuta?

- Cuando la app carga ( `useAuth.checkAuth()` )
- Cada vez que se necesita verificar autenticación

## Estructura de localStorage después del login

```

javascript

// Después de un Login exitoso, LocalStorage contiene:

localStorage = {
  "auth_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NSwiZW1haWwiOiJvbG12ZXJAc3RvbmUuY29tIiwicm9sZSI6ImFkbWluIiwiaXNjaXNzIjoxNzMzMjU0NDAwfQ...",
  "user_data": "{\\"id\\":5,\\"email\\":\\"oliver@stone.com\\",\\"name\\":\\"Oliver\\",\\"lastname\\":\\"Stone\\",\\"role\\":\\"admin\\",\\"phone\\":\\"123456789\\",\\"image\\":\\"\\\"}",
  "user_role": "admin"
}

```

## Puntos clave para recordar

| Concepto               | Explicación                                                                                                                                                  |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| El modelo NO valida    | Las validaciones (email vacío, formato, etc.) las hace el controlador. El modelo solo procesa la respuesta de la API                                         |
| Normalización de datos | El modelo transforma la respuesta de la API en un formato consistente (ej: si no viene <code>name</code> , usa la parte del email antes del <code>@</code> ) |
| Limpieza del token     | El backend puede devolver <code>"JWT eyJhbG..."</code> o solo <code>"eyJhbG..."</code> . El modelo lo normaliza a solo el token                              |
| Almacenamiento triple  | Guarda token, usuario completo, y rol por separado para acceso rápido                                                                                        |
| Manejo de errores      | Si la API no responde (servidor caído), el <code>catch</code> captura y retorna error amigable                                                               |

## Posibles errores en este punto

| Error                                                 | Causa                                                             | Solución                                                                       |
|-------------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>response.data.session_token is undefined</code> | El backend no devuelve <code>session_token</code> en la respuesta | Verifica el controlador del backend (debe incluir <code>session_token</code> ) |



| Error                                                                | Causa                                                     | Solución                                                          |
|----------------------------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------|
| <code>response.data</code> es null                                   | La API devolvió <code>success: true</code> pero sin datos | Revisa la respuesta real del backend en la consola                |
| El token se guarda pero <code>isAuthenticated()</code> retorna false | <code>jwtService.verifyToken()</code> falla               | Verifica que el token tenga formato JWT válido y no esté expirado |
| <code>storageService.setToken</code> falla                           | <code>localStorage</code> está lleno o deshabilitado      | Captura el error y muestra mensaje al usuario                     |

¿Entendiste por qué `AuthModel.js` es el encargado de comunicarse con la API y guardar datos en `localStorage`?

Escribe "siguiente" y pasamos al ARCHIVO 8: `httpService.js` (el cliente HTTP que hace las peticiones reales). 🌐

siguiente

## 📄 ARCHIVO 8: `src/services/httpService.js` (OCTAVO en ejecutarse)

```
javascript
// src/services/httpService.js
import API_CONFIG from '../config/api'

class HttpService {
  constructor() {
    this.baseUrl = API_CONFIG.BASE_URL
  }

  getToken() {
    return localStorage.getItem('auth_token')
  }

  getHeaders(includeAuth = true) {
    const headers = {
      'Content-Type': 'application/json'
    }

    if (includeAuth) {
      const token = this.getToken()
      if (token) {
        headers['Authorization'] = `Bearer ${token}`
      }
    }

    return headers
  }

  async handleResponse(response) {
    console.log('📡 Response status:', response.status)

    // Obtener el cuerpo de la respuesta
    let data = {}
    try {
      data = await response.json()
    } catch (e) {
      console.error('Error al parsear JSON:', e)
    }

    console.log('📡 Response data completa:', data)
```

```

    if (!response.ok) {
        const errorMessage = data.message || data.error || `Error HTTP: ${response.status}`
        throw new Error(errorMessage)
    }

    // Retornamos toda la respuesta para que el Modelo pueda acceder a data.session_token
    return data
}

async post(endpoint, data, includeAuth = true) {
    try {
        const url = `${this.baseURL}${endpoint}`
        console.log(`📡 POST a: ${url}`)
        console.log(`📡 Datos enviados:`, data)

        const response = await fetch(url, {
            method: 'POST',
            headers: this.getHeaders(includeAuth),
            body: JSON.stringify(data)
        })

        console.log(`📡 Status:`, response.status)
        return await this.handleResponse(response)
    } catch (error) {
        console.error(`❌ POST ${endpoint} error:`, error)
        throw error
    }
}

async get(endpoint, includeAuth = true) {
    try {
        const url = `${this.baseURL}${endpoint}`
        console.log(`📡 GET: ${url}`)

        const response = await fetch(url, {
            method: 'GET',
            headers: this.getHeaders(includeAuth)
        })

        return await this.handleResponse(response)
    } catch (error) {
        console.error(`GET ${endpoint} error:`, error)
        throw error
    }
}

async put(endpoint, data, includeAuth = true) {
    try {
        const url = `${this.baseURL}${endpoint}`
        console.log(`📡 PUT a: ${url}`)

        const response = await fetch(url, {
            method: 'PUT',
            headers: this.getHeaders(includeAuth),
            body: JSON.stringify(data)
        })

        return await this.handleResponse(response)
    } catch (error) {
        console.error(`PUT ${endpoint} error:`, error)
        throw error
    }
}

async delete(endpoint, includeAuth = true) {
    try {
        const url = `${this.baseURL}${endpoint}`
        console.log(`📡 DELETE a: ${url}`)

        const response = await fetch(url, {
            method: 'DELETE',
            headers: this.getHeaders(includeAuth)
        })

        return await this.handleResponse(response)
    }
}

```

```

    } catch (error) {
      console.error(`DELETE ${endpoint} error:`, error)
      throw error
    }
  }
}

export default new HttpService()

```

## Responsabilidad en la ejecución

`httpService.js` es un **servicio singleton** que se ejecuta **cuando** `AuthModel.js` (u otros modelos) llaman a sus métodos (`post`, `get`, `put`, `delete`). Su responsabilidad es:

1. Realizar peticiones HTTP a la API usando `fetch` nativo.
2. Agregar automáticamente el token JWT en los headers (`Authorization: Bearer <token>`) cuando `includeAuth = true`.
3. Centralizar la configuración de la URL base (`http://localhost:3000/api`).
4. Manejar respuestas (convertir JSON, verificar errores HTTP).
5. Proveer logs detallados para depuración.

## Explicación del patrón Singleton

```

javascript

// Al final del archivo:
export default new HttpService()

```

### ¿Qué significa?

En lugar de exportar la **clase**, exportamos una **instancia única** de la clase. Esto asegura que toda la aplicación use el mismo objeto `httpService` con la misma configuración.

### Diferencia:

```

javascript

// ❌ Si exportaras la clase:
import { HttpService } from './httpService'
const miHttp = new HttpService() // Cada quien crea su propia instancia

// ✅ Como está ahora:
import httpService from './httpService'
// httpService ya es una instancia lista para usar, no necesitas `new`

```

## Orden de ejecución línea por línea (método `post`)

| Orden | Línea                                                             | ¿Qué pasa en tiempo de ejecución?                                    |
|-------|-------------------------------------------------------------------|----------------------------------------------------------------------|
| 1     | <code>import API_CONFIG from '../config/api'</code>               | Carga configuración (URL base, endpoints)                            |
| 3     | <code>class HttpService {</code>                                  | Define la clase                                                      |
| 4-6   | <code>constructor() { this.baseUrl = API_CONFIG.BASE_URL }</code> | Al crear la instancia, guarda <code>http://localhost:3000/api</code> |

| Orden | Línea                                                                                       | ¿Qué pasa en tiempo de ejecución?                                                                           |
|-------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 8-10  | <code>getToken() { return localStorage.getItem('auth_token') }</code>                       | Función para leer token de localStorage                                                                     |
| 12-24 | <code>getHeaders(includeAuth)</code>                                                        | Construye headers HTTP (Content-Type, Authorization)                                                        |
| 26-38 | <code>async handleResponse(response)</code>                                                 | Procesa la respuesta del servidor                                                                           |
| 40-56 | <code>async post(endpoint, data, includeAuth)</code>                                        |  <b>MÉTODO PRINCIPAL</b>   |
| 41    | <code>try {</code>                                                                          | Inicia bloque de manejo de errores                                                                          |
| 42    | <code>const url = this.baseURL {endpoint}`</code>                                           | Construye URL completa:<br><code>http://localhost:3000/api/users/login</code>                               |
| 43-44 | <code>console.log(...)</code>                                                               | Logs de depuración                                                                                          |
| 46-51 | <code>const response = await fetch(url, { method: 'POST', headers: ..., body: ... })</code> |  <b>PETICIÓN HTTP REAL</b> |
| 53    | <code>console.log('📄 Status:', response.status)</code>                                      | Muestra código de respuesta (200, 401, 500, etc.)                                                           |
| 54    | <code>return await this.handleResponse(response)</code>                                     | Procesa la respuesta y la retorna                                                                           |
| 55-56 | <code>catch (error) { throw error }</code>                                                  | Re-lanza el error para que el modelo lo capture                                                             |

### Escenario real: `AuthModel.login()` llama a `httpService.post()`

```

javascript

// Desde AuthModel.js:
const response = await httpService.post(
  API_CONFIG.ENDPOINTS.LOGIN, // '/users/login'
  {
    email: "oliver@stone.com",
    password: "123456"
  },
  false // includeAuth = false (no necesita token)
)

// DENTRO DE httpService.post():

Paso 1: Construir URL
- this.baseURL = "http://localhost:3000/api"
- endpoint = "/users/login"
- url = "http://localhost:3000/api/users/login"

Paso 2: Construir headers
- includeAuth = false → NO agrega token
- headers = { "Content-Type": "application/json" }

Paso 3: Hacer fetch
- fetch("http://localhost:3000/api/users/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },

```

```
body: JSON.stringify({ email: "oliver@stone.com", password: "123456" })
})
```

Paso 4: El servidor responde

- response.status = 200 (OK)
- response.body = { "success": true, "data": { ... } }

Paso 5: `handleResponse()` procesa

- data = `await response.json()`
- `if (!response.ok) → no entra (status 200 es OK)`
- `return data // { success: true, data: {...} }`

Paso 6: Retorna al modelo

- response = { success: true, data: { id, email, session\_token, ... } }

## Ejemplo de petición GET con autenticación

javascript

```
// Para obtener lista de usuarios (requiere token):
const users = await httpService.get('/users', true)
```

```
// DENTRO DE httpService.get():
```

Paso 1: Construir URL: `http://localhost:3000/api/users`

Paso 2: Construir headers:

- includeAuth = true → llama a `getToken()`
- token = "eyJhbGciOiJIUzI1NiIs..."
- headers = {  
 "Content-Type": "application/json",  
 "Authorization": "Bearer eyJhbGciOiJIUzI1NiIs..."  
}

Paso 3: `fetch()` con GET y headers con token

Paso 4: El backend verifica el token y responde con la lista

## Método `handleResponse()` en detalle

javascript

```
async handleResponse(response) {
  console.log('🔥 Response status:', response.status) // 200, 401, 404, 500, etc.

  // Intentar parsear JSON (puede fallar si la respuesta no es JSON)
  let data = {}
  try {
    data = await response.json()
  } catch (e) {
    console.error('Error al parsear JSON:', e)
  }

  // Si la respuesta no es OK (status 200-299), lanzar error
  if (!response.ok) {
    const errorMessage = data.message || data.error || `Error HTTP: ${response.status}`
    throw new Error(errorMessage)
  }

  // Si todo está bien, retornar los datos
  return data
}
```

Códigos de respuesta comunes:

| Status | Significado   | ¿Qué hace handleResponse?               |
|--------|---------------|-----------------------------------------|
| 200    | OK            | Retorna data (no lanza error)           |
| 201    | Creado        | Retorna data (no lanza error)           |
| 400    | Bad Request   | Lanza error con mensaje del backend     |
| 401    | No autorizado | Lanza error (token inválido o expirado) |
| 403    | Prohibido     | Lanza error (no tiene permisos)         |
| 404    | No encontrado | Lanza error                             |
| 500    | Error interno | Lanza error                             |

## Headers que se envían en cada petición

```
javascript
// Petición SIN autenticación (login, register):
headers = {
  "Content-Type": "application/json"
}

// Petición CON autenticación (dashboard, usuarios):
headers = {
  "Content-Type": "application/json",
  "Authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

## Analogía para entenderlo

Imagina que `httpService.js` es un **servicio de mensajería**:

- `post()` → Es como enviar una carta certificada (entregas datos y esperas respuesta).
- `get()` → Es como hacer una consulta en una biblioteca (solo pides información).
- `put()` → Es como actualizar un documento existente (reemplazas todo).
- `delete()` → Es como solicitar que borren un expediente.
- `getHeaders()` → Es como poner el sello y la dirección en el sobre.
- `handleResponse()` → Es como recibir la respuesta y verificar que no venga rota.

El **token** que se agrega automáticamente es como una **credencial de identificación** que se pone en cada sobre para que el destinatario sepa quién eres.

## Puntos clave para recordar

| Concepto                                    | Explicación                                                                                          |
|---------------------------------------------|------------------------------------------------------------------------------------------------------|
| Singleton                                   | <code>export default new HttpService()</code> significa que solo UNA instancia existe en toda la app |
| Base URL                                    | Se define una sola vez en <code>constructor()</code> y se reutiliza para todas las peticiones        |
| <code>includeAuth = true</code> por defecto | La mayoría de las peticiones requieren autenticación, por eso es <code>true</code> por defecto       |

| Concepto                        | Explicación                                                                                                  |
|---------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Bearer token</b>             | Formato estándar para enviar JWT:<br><code>Authorization: Bearer &lt;token&gt;</code>                        |
| <b>try/catch en cada método</b> | Captura errores de red (servidor caído, timeout, sin internet) y los propaga                                 |
| <b>throw error</b>              | En lugar de retornar un objeto de error, <code>throw</code> permite que el modelo use <code>try/catch</code> |

## Posibles errores en este punto

| Error                                                       | Causa                                                                    | Solución                                                                 |
|-------------------------------------------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <code>Failed to fetch</code>                                | El servidor backend no está corriendo en <code>localhost:3000</code>     | Inicia el backend con <code>npm run dev</code> en la carpeta del backend |
| <code>NetworkError when attempting to fetch resource</code> | CORS no configurado en el backend                                        | El backend debe tener <code>cors()</code> habilitado                     |
| <code>401 Unauthorized</code>                               | Token inválido o expirado                                                | El usuario debe volver a iniciar sesión                                  |
| <code>Unexpected token &lt; in JSON</code>                  | El servidor devolvió HTML en lugar de JSON                               | El endpoint no existe o hay error en el servidor                         |
| El token no se envía en el header                           | <code>getToken()</code> retorna null o <code>includeAuth</code> es false | Verifica que el token exista en localStorage                             |

## Configuración de CORS en el backend (importante)

El backend DEBE tener CORS configurado para aceptar peticiones desde `localhost:5173`:

```
javascript
// En el backend (index.js o server.js)
app.use(cors({
  origin: ['http://localhost:5173', 'http://127.0.0.1:5173'],
  credentials: true
})))
```

¿Entendiste por qué `httpService.js` es el encargado de hacer las peticiones HTTP reales y agregar el token automáticamente?

Escribe "siguiente" y pasamos al ARCHIVO 9: `storageService.js` (manejo de localStorage). 

siguiente

## ARCHIVO 9: `src/services/storageService.js` (NOVENO en ejecutarse)

```
javascript
// src/services/storageService.js
class StorageService {
  constructor(storageType = 'localStorage') {
    this.storage = storageType === 'localStorage' ? localStorage : sessionStorage
```

```

    this.tokenKey = 'auth_token'
    this.userKey = 'user_data'
    this.roleKey = 'user_role'
  }

  // Guardar token
  setToken(token) {
    console.log('📁 Guardando token en storage')
    return this.setItem(this.tokenKey, token)
  }

  // Obtener token
  getToken() {
    const token = this.getItem(this.tokenKey)
    console.log('🔑 Token obtenido:', token ? 'Sí existe' : 'No existe')
    return token
  }

  // Eliminar token
  removeToken() {
    console.log('🗑️ Eliminando token')
    return this.removeItem(this.tokenKey)
  }

  // Guardar usuario
  setUser(user) {
    console.log('📁 Guardando usuario en storage:', user)
    return this.setItem(this.userKey, JSON.stringify(user))
  }

  // Obtener usuario
  getUser() {
    const user = this.getItem(this.userKey)
    return user ? JSON.parse(user) : null
  }

  // Eliminar usuario
  removeUser() {
    return this.removeItem(this.userKey)
  }

  // Guardar rol del usuario
  setUserRole(role) {
    console.log('📁 Guardando rol en storage:', role)
    return this.setItem(this.roleKey, role)
  }

  // Obtener rol del usuario
  getUserRole() {
    return this.getItem(this.roleKey)
  }

  // Eliminar rol
  removeUserRole() {
    return this.removeItem(this.roleKey)
  }

  // Limpiar toda la sesión
  clearSession() {
    this.removeToken()
    this.removeUser()
    this.removeUserRole()
  }

  // Guardar item
  setItem(key, value) {
    try {
      this.storage.setItem(key, value)
      return true
    } catch (error) {
      console.error('Error al guardar en storage:', error)
    }
  }

```



```

        return false
    }
}

// Obtener item
getItem(key) {
    try {
        return this.storage.getItem(key)
    } catch (error) {
        console.error('Error al obtener del storage:', error)
        return null
    }
}

// Eliminar item
removeItem(key) {
    try {
        this.storage.removeItem(key)
        return true
    } catch (error) {
        console.error('Error al eliminar del storage:', error)
        return false
    }
}

// Limpiar todo
clear() {
    try {
        this.storage.clear()
        return true
    } catch (error) {
        console.error('Error al limpiar storage:', error)
        return false
    }
}

// Verificar si existe
hasItem(key) {
    return this.getItem(key) !== null
}
}

export default new StorageService('localStorage')

```

## Responsabilidad en la ejecución

`storageService.js` es un **servicio singleton** que se ejecuta **cuando** `AuthModel.js` (u otros modelos) necesitan guardar o leer datos del almacenamiento local. Su responsabilidad es:

1. **Abstraer el acceso a** `localStorage` (o `sessionStorage`) para no tener que escribir `localStorage.setItem()` directamente.
2. **Proveer métodos específicos** para operaciones comunes: `setToken()`, `getToken()`, `setUser()`, `getUser()`, etc.
3. **Manejar errores** de almacenamiento (cuota excedida, cookies deshabilitadas, etc.).
4. **Centralizar las claves** usadas en el almacenamiento (`auth_token`, `user_data`, `user_role`) para evitar errores de tipeo.
5. **Automatizar la conversión JSON** al guardar/leer objetos (no necesitas hacer `JSON.stringify` manualmente).

## Explicación del patrón Singleton con constructor configurable

```

javascript

constructor(storageType = 'localStorage') {
  this.storage = storageType === 'localStorage' ? localStorage : sessionStorage
  // ...
}

// Al final:
export default new StorageService('localStorage')

```

### ¿Qué significa?

- El `constructor` recibe `storageType` (por defecto `'localStorage'`).
- Si alguien quisiera usar `sessionStorage`, podría crear otra instancia, pero por ahora usamos `localStorage`.
- Se exporta una **instancia única** ya creada.

**Diferencia entre** `localStorage` **y** `sessionStorage`:

| Característica            | localStorage                   | sessionStorage                 |
|---------------------------|--------------------------------|--------------------------------|
| Persistencia              | Hasta que se borre manualmente | Hasta que se cierra la pestaña |
| Compartido entre pestañas | ✅ Sí                           | ❌ No                           |
| Se mantiene al recargar   | ✅ Sí                           | ✅ Sí                           |
| Uso en este proyecto      | ✅ Token y usuario              | ❌ No se usa                    |

### 🔍 Orden de ejecución línea por línea (métodos clave)

`setToken(token)` y `getToken()`

| Orden | Línea                                                    | ¿Qué pasa en tiempo de ejecución?                                           |
|-------|----------------------------------------------------------|-----------------------------------------------------------------------------|
| 1     | <code>setToken(token) {</code>                           | Modelo llama a este método después de login exitoso                         |
| 2     | <code>console.log('📁 Guardando token en storage')</code> | Log de depuración                                                           |
| 3     | <code>return this.setItem(this.tokenKey, token)</code>   | Llama a <code>setItem</code> con clave <code>'auth_token'</code> y el token |
| ...   | ...                                                      | ...                                                                         |
| 8     | <code>getToken() {</code>                                | Modelo llama para leer token (ej: para agregar a headers)                   |
| 9     | <code>const token = this.getItem(this.tokenKey)</code>   | Lee de <code>localStorage</code>                                            |
| 10    | <code>console.log(...)</code>                            | Log informativo                                                             |
| 11    | <code>return token</code>                                | Retorna el token o null                                                     |

`setUser(user)` y `getUser()`

| Orden | Línea                                                                        | ¿Qué pasa en tiempo de ejecución?                      |
|-------|------------------------------------------------------------------------------|--------------------------------------------------------|
| 18    | <code>setUser(user) {</code>                                                 | Modelo guarda usuario después de login                 |
| 19    | <code>console.log(...)</code>                                                | Log de depuración                                      |
| 20    | <code>return<br/>this.setItem(this.userKey,<br/>JSON.stringify(user))</code> | Convierte objeto a string JSON y lo guarda             |
| ...   | ...                                                                          | ...                                                    |
| 23    | <code>getUser() {</code>                                                     | Modelo lee usuario (ej: para mostrar nombre en navbar) |
| 24    | <code>const user =<br/>this.getItem(this.userKey)</code>                     | Lee string JSON de localStorage                        |
| 25    | <code>return user ? JSON.parse(user)<br/>: null</code>                       | Convierte string a objeto o retorna null               |

#### `clearSession()` - limpieza completa

| Orden | Línea                              | ¿Qué pasa en tiempo de ejecución? |
|-------|------------------------------------|-----------------------------------|
| 44    | <code>clearSession() {</code>      | Modelo llama al hacer logout      |
| 45    | <code>this.removeToken()</code>    | Borra <code>auth_token</code>     |
| 46    | <code>this.removeUser()</code>     | Borra <code>user_data</code>      |
| 47    | <code>this.removeUserRole()</code> | Borra <code>user_role</code>      |

### ⚡ Escenario real: Después de un login exitoso

```
javascript

// Desde AuthModel.js, después de recibir respuesta de La API:

// 1. Guardar token
storageService.setToken("eyJhbGciOiJIUzI1NiIs...")
// → localStorage.setItem('auth_token', 'eyJhbGciOiJIUzI1NiIs...')

// 2. Guardar usuario
storageService.setUser({ id: 5, email: "oliver@stone.com", name: "Oliver", role: "admin" })
// → localStorage.setItem('user_data', '{"id":5,"email":"oliver@stone.com","name":"Oliver","role":"admin"}')

// 3. Guardar rol (para acceso rápido)
storageService.setUserRole("admin")
// → localStorage.setItem('user_role', 'admin')

// Estado final de localStorage:
localStorage = {
  "auth_token": "eyJhbGciOiJIUzI1NiIs...",
  "user_data": "{\"id\":5,\"email\":\"oliver@stone.com\",\"name\":\"Oliver\",\"role\":\"admin\"}",
  "user_role": "admin"
}
```

### ⚡ Escenario real: Recargar la página (F5)

```

javascript

// La app se reinicia, main.jsx ejecuta App.jsx, App.jsx ejecuta ProtectedRoute

// ProtectedRoute llama a useAuth() → useAuth ejecuta checkAuth()

// Dentro de checkAuth():
const authState = AuthController.getAuthState()

// Dentro de getAuthState():
return {
  isAuthenticated: AuthModel.isAuthenticated(), // ← Llama a storageService.getToken()
  currentUser: AuthModel.getCurrentUser(), // ← Llama a storageService.getUser()
  currentToken: AuthModel.getCurrentToken() // ← Llama a storageService.getToken()
}

// storageService.getToken() → Lee de LocalStorage → retorna el token
// storageService.getUser() → Lee de LocalStorage → retorna el usuario

// Si el token existe y no expiró, isAuthenticated = true
// El usuario sigue Logueado después de recargar la página ✅

```

## 📁 Manejo de errores en `setItem()`

```

javascript

setItem(key, value) {
  try {
    this.storage.setItem(key, value)
    return true
  } catch (error) {
    console.error('Error al guardar en storage:', error)
    return false
  }
}

```

Posibles errores que puede capturar:

| Error              | Causa                                                                                                 | Consecuencia                                               |
|--------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| QuotaExceededError | localStorage lleno (más de 5-10MB)                                                                    | Retorna <code>false</code> , el modelo puede mostrar error |
| SecurityError      | Cookies deshabilitadas o modo incógnito muy restrictivo                                               | Retorna <code>false</code>                                 |
| TypeError          | Intentar guardar un objeto sin stringify (no ocurre porque ya hacemos <code>JSON.stringify()</code> ) | Retorna <code>false</code>                                 |

## 💡 Analogía para entenderlo

Imagina que `storageService.js` es un **cajón de escritorio con llave**:

- `localStorage` → Es el cajón físico donde guardas papeles importantes.
- `setToken()` → Es como poner un papel con tu token de acceso en el cajón.
- `getToken()` → Es como abrir el cajón y sacar ese papel.
- `clearSession()` → Es como vaciar todo el cajón de una vez.
- `JSON.stringify()` → Es como poner los objetos dentro de un sobre (porque no puedes meter una figura 3D en un cajón, solo papeles planos).

- `JSON.parse()` → Es como abrir el sobre y sacar la figura 3D.

El **servicio** es como tener un asistente que siempre sabe dónde está cada cosa y te la trae cuando la necesitas, sin que tengas que recordar en qué cajón o carpeta la pusiste.

 **Puntos clave para recordar**

| Concepto                    | Explicación                                                                                                                               |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Claves centralizadas        | <code>tokenKey</code> , <code>userKey</code> , <code>roleKey</code> están definidas en UN solo lugar. Si cambian, solo modificas aquí     |
| Automatic JSON parsing      | <code>getUser()</code> hace <code>JSON.parse()</code> automático, no tienes que hacerlo en el modelo                                      |
| Manejo de errores           | Si <code>localStorage</code> falla, no crashea la app, solo retorna <code>false</code> o <code>null</code>                                |
| Logs útiles                 | Cada operación importante tiene <code>console.log</code> para depuración                                                                  |
| Doble almacenamiento de rol | Se guarda dentro de <code>user_data</code> y también separado en <code>user_role</code> para acceso más rápido sin tener que parsear JSON |

 **Posibles errores en este punto**

| Error                                               | Causa                                                   | Solución                                                                         |
|-----------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>JSON.parse</code> lanza error                 | El valor en <code>localStorage</code> no es JSON válido | Borrar manualmente <code>localStorage</code> desde DevTools y volver a loguearse |
| <code>getUser()</code> retorna null pero hay sesión | <code>user_data</code> no existe o fue borrado          | El usuario debe volver a iniciar sesión                                          |
| <code>setItem</code> retorna false                  | <code>localStorage</code> lleno o deshabilitado         | Mostrar mensaje "No se pudo guardar la sesión" al usuario                        |
| Los datos persisten después de logout               | <code>clearSession()</code> no se llamó                 | Verificar que <code>logout()</code> llame a <code>clearSession()</code>          |

 **Ver el estado de localStorage en el navegador**

Para depurar, abre DevTools (F12) → pestaña **Application** → **Local Storage** → `http://localhost:5173`

Allí verás:

| text                    |                                                                      |
|-------------------------|----------------------------------------------------------------------|
| Key                     | Value                                                                |
| <code>auth_token</code> | <code>eyJhbGciOiJIUzI1NiIs...</code>                                 |
| <code>user_data</code>  | <code>{"id":5,"email":"oliver@stone.com","name":"Oliver",...}</code> |
| <code>user_role</code>  | <code>admin</code>                                                   |

También puedes verlo en consola:

```
javascript
```

```
console.log(localStorage.getItem('auth_token'))
console.log(JSON.parse(localStorage.getItem('user_data')))
```

¿Entendiste por qué `storageService.js` es el encargado de abstraer el acceso a `localStorage` y simplificar las operaciones de guardado/lectura?

Escribe "siguiente" y pasamos al ARCHIVO 10: `jwtService.js` (verificación y decodificación de tokens JWT). 🗝️

siguiente

## 📄 ARCHIVO 10: `src/services/jwtService.js` (DÉCIMO en ejecutarse)

```
javascript

// src/services/jwtService.js
class JWTService {
  generateToken(payload) {
    try {
      console.log('🔑 Generando token para:', payload)

      const header = { alg: 'HS256', typ: 'JWT' }
      const encodedHeader = btoa(JSON.stringify(header))
      const encodedPayload = btoa(JSON.stringify(payload))
      const signature = btoa('fake_signature_' + Date.now())
      const token = `${encodedHeader}.${encodedPayload}.${signature}`

      console.log('✅ Token generado:', token.substring(0, 50) + '...')
      return token
    } catch (error) {
      console.error('Error al generar token:', error)
      return null
    }
  }

  verifyToken(token) {
    try {
      if (!token) return false

      const parts = token.split('.')
      if (parts.length !== 3) return false

      const payload = JSON.parse(atob(parts[1]))

      // 🚫 CORREGIDO: Verificación con tolerancia de 5 minutos
      if (payload.exp) {
        const now = Date.now()
        const expTime = payload.exp * 1000 // Convertir a milisegundos
        const tolerance = 5 * 60 * 1000 // 5 minutos de tolerancia

        if (expTime + tolerance < now) {
          console.warn('Token expirado (exp:', new Date(expTime), 'now:', new Date(now))
          return false
        }
      }

      console.log('✅ Token verificado correctamente')
      return true
    } catch (error) {
      console.error('Error al verificar token:', error)
      return false
    }
  }
}
```

```

decodeToken(token) {
  try {
    const parts = token.split('.')
    if (parts.length !== 3) return null
    const payload = JSON.parse(atob(parts[1]))
    return payload
  } catch (error) {
    console.error('Error al decodificar token:', error)
    return null
  }
}

getTokenRemainingTime(token) {
  try {
    const payload = this.decodeToken(token)
    if (!payload || !payload.exp) return 0
    const remainingTime = (payload.exp * 1000) - Date.now()
    return remainingTime > 0 ? remainingTime : 0
  } catch (error) {
    return 0
  }
}
}

export default new JWTService()

```

## Responsabilidad en la ejecución

`jwtService.js` es un **servicio singleton** que se ejecuta **cuando** `AuthModel.js` necesita verificar si un token JWT es válido o decodificar su contenido. Su responsabilidad es:

1. **Verificar que un token JWT no haya expirado** (con tolerancia de 5 minutos para evitar problemas de sincronización de reloj).
2. **Decodificar el payload** del token para extraer información como el `exp` (fecha de expiración) o el `role`.
3. **Calcular el tiempo restante** antes de que el token expire (útil para mostrar mensajes como "Tu sesión expirará en 5 minutos").
4. **Generar tokens falsos** (solo para desarrollo/demo, el backend genera los reales).

## Estructura de un token JWT

Un token JWT tiene 3 partes separadas por puntos:

text

```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NSwiZW1haWwiOiJvbG12ZXJAc3RvbmUuY29tIiwicm9sZSI6ImFkbWluIiwiaXNjaXZlbnR5b250NDAwfQ.Sf1KxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

```

| Parte | Nombre    | Contenido (decodificado)                                                                  |
|-------|-----------|-------------------------------------------------------------------------------------------|
| 1     | Header    | <code>{ "alg": "HS256", "typ": "JWT" }</code>                                             |
| 2     | Payload   | <code>{ "id": 5, "email": "oliver@stone.com", "role": "admin", "exp": 1733254400 }</code> |
| 3     | Signature | Firma digital (verifica que no fue modificado)                                            |

## Orden de ejecución línea por línea (método `verifyToken`)





```
parts[2] = "Sf1KxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c" // Signature
```

Paso 2: `payload = JSON.parse(atob(parts[1]))`  
`atob(parts[1])` decodifica Base64 → `'{"id":5,"email":"oliver@stone.com","role":"admin","exp":1733254400}'`  
`JSON.parse()` → `{ id: 5, email: "oliver@stone.com", role: "admin", exp: 1733254400 }`

Paso 3: Verificar expiración  
`expTime = 1733254400 * 1000 = 1733254400000 ms` (fecha: 3 de diciembre de 2024, 10:00:00)  
`now = Date.now() = 1733254000000 ms` (fecha: 3 de diciembre de 2024, 9:55:00)  
`tolerance = 300,000 ms` (5 minutos)  
`expTime + tolerance = 1733254400000 + 300000 = 1733254700000`  
`{1733254700000 < 1733254000000?}` → NO (es mayor)  
→ Token NO ha expirado ✓

Paso 4: Retorna `true` → `isAuthenticated = true`

## ⚡ Escenario real: Token expirado

```
javascript

// Supongamos que pasaron 6 minutos desde que expiró:
expTime = 1733254400000 (expira a las 10:00)
now = 1733254760000 (son las 10:06)
tolerance = 300,000 ms

expTime + tolerance = 1733254400000 + 300000 = 1733254700000 (10:05:00)
{1733254700000 < 1733254760000?} → SÍ (10:05 es menor que 10:06)

→ Token expirado ✗
→ Retorna false
→ AuthModel.isAuthenticated() retorna false
→ storageService.clearSession() se ejecuta
→ Usuario debe volver a iniciar sesión
```

## 📁 Método `getTokenRemainingTime()` - tiempo restante

```
javascript

getTokenRemainingTime(token) {
  const payload = this.decodeToken(token)
  if (!payload || !payload.exp) return 0

  const remainingTime = (payload.exp * 1000) - Date.now()
  return remainingTime > 0 ? remainingTime : 0
}
```

Ejemplo de uso (para mostrar "tu sesión expira en X minutos"):

```
javascript

const remainingMs = jwtService.getTokenRemainingTime(token)
const remainingMinutes = Math.floor(remainingMs / 60000)
console.log(`Sesión expira en ${remainingMinutes} minutos`)
// Output: "Sesión expira en 5 minutos"
```

## 🔍 Diferencia entre `generateToken` (fake) y el token real del backend

| Característica    | <code>generateToken()</code> (fake) | Token del backend (real)                 |
|-------------------|-------------------------------------|------------------------------------------|
| ¿Quién lo genera? | Este servicio (solo demo)           | El backend con <code>jsonwebtoken</code> |

| Característica         | <code>generateToken()</code> (fake) | Token del backend (real)                |
|------------------------|-------------------------------------|-----------------------------------------|
| ¿Tiene firma válida?   | No, es una firma falsa              | Sí, firmada con <code>JWT_SECRET</code> |
| ¿Se usa en producción? | No                                  | Sí                                      |
| Propósito              | Pruebas locales sin backend         | Autenticación real                      |

### 💡 Analogía para entenderlo

Imagina que `jwtService.js` es un **revisor de boletos en un concierto**:

- `verifyToken(token)` → Es el revisor que mira tu boleto y verifica:
  - ¿Tiene la forma correcta? (3 partes, como un boleto con talón, cuerpo y firma)
  - ¿La fecha del evento ya pasó? (fecha de expiración)
  - ¿Tiene 5 minutos de tolerancia? (por si el reloj del revisor está un poco adelantado)
- `decodeToken(token)` → Es como leer la información del boleto: "Asiento Fila 5, Número 12, Válido hasta las 22:00".
- `getTokenRemainingTime(token)` → Es calcular "Faltan 2 horas para que termine el concierto".
- `generateToken(payload)` **(fake)** → Es como imprimir un boleto falso en casa (no te dejarán entrar al concierto real, pero sirve para practicar).

### ⚠️ Puntos clave para recordar

| Concepto                | Explicación                                                                                                                                                         |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>atob()</code>     | Función nativa de JavaScript que decodifica Base64 a texto. <code>btoa()</code> hace lo opuesto (codifica a Base64)                                                 |
| Expiración en segundos  | JWT usa <code>exp</code> en <b>segundos</b> desde 1970 (Unix timestamp). JavaScript usa <code>Date.now()</code> en <b>milisegundos</b> → multiplicar por 1000       |
| Tolerancia de 5 minutos | Evita que diferencias de reloj entre cliente y servidor invaliden tokens legítimos. Si el servidor está 2 minutos adelantado, el token no se rechazará injustamente |
| No verifica la firma    | Este servicio solo verifica la expiración y estructura. En producción, el backend verifica la firma. El frontend confía en que el token no fue modificado           |
| Singleton               | <code>export default new JWTService()</code> → toda la app usa la misma instancia                                                                                   |

### 🐛 Posibles errores en este punto

| Error                            | Causa                                           | Solución                                                          |
|----------------------------------|-------------------------------------------------|-------------------------------------------------------------------|
| <code>atob is not defined</code> | Entorno que no soporta Base64 (Node.js antiguo) | Usar <code>Buffer.from(payload).toString('base64')</code> en Node |

| Error                       | Causa                                                | Solución                                                     |
|-----------------------------|------------------------------------------------------|--------------------------------------------------------------|
| InvalidCharacterError       | El token contiene caracteres no válidos para Base64  | El token está mal formado, el usuario debe reloguearse       |
| JSON.parse falla            | El payload no es JSON válido                         | El token está corrupto, limpiar localStorage                 |
| Token nunca expira          | El backend no incluye <code>exp</code> en el payload | El backend debe agregar <code>exp</code> al generar el token |
| Token expira inmediatamente | El <code>exp</code> está en el pasado                | El backend tiene la fecha mal calculada                      |

## 🔧 Formato esperado del token del backend

```
javascript
// El backend DEBE generar tokens con esta estructura:
const token = jwt.sign(
  {
    id: user.id,
    email: user.email,
    role: user.role,
    exp: Math.floor(Date.now() / 1000) + (60 * 60) // 1 hora desde ahora
  },
  process.env.JWT_SECRET
);

// El token resultante tendrá un payload como:
{
  "id": 5,
  "email": "oliver@stone.com",
  "role": "admin",
  "exp": 1733254400 // ← Este valor es EN SEGUNDOS, no milisegundos
}
```

¿Entendiste por qué `jwtService.js` es el encargado de verificar expiración y decodificar tokens JWT?

Escribe "siguiente" y pasamos al ARCHIVO 11: `config/api.js` (la configuración de la API). ⚙️

siguiente

## 📄 ARCHIVO 11: `src/config/api.js` (UNDÉCIMO en ejecutarse)

```
javascript
// src/config/api.js
const API_CONFIG = {
  // Usa la IP real de tu máquina (La que muestra ipconfig/ifconfig)
  // Si es la misma máquina, usa 'http://localhost:3000/api'
  BASE_URL: 'http://localhost:3000/api', // ← Verifica que esta IP sea correcta
  TIMEOUT: 10000,
  ENDPOINTS: {
    LOGIN: '/users/login',
    REGISTER: '/users/create',
    USERS: '/users',
    USER_BY_ID: '/users/:id',
    USER_DELETE: '/users/delete/:id'
  }
}
```

```
}  
  
export default API_CONFIG
```

## Responsabilidad en la ejecución

`api.js` es un archivo de configuración que se ejecuta cuando cualquier otro archivo lo importa (como `httpService.js` o `AuthModel.js`). Su responsabilidad es:

1. Centralizar la URL base de la API para que si cambia (ej: de localhost a una IP de red), solo se modifique en UN lugar.
2. Definir todos los endpoints de la API como constantes, evitando escribir rutas a mano.
3. Configurar el timeout para las peticiones (aunque actualmente no se usa, está preparado).
4. Servir como documentación rápida de qué endpoints están disponibles.

## Orden de ejecución línea por línea

| Orden | Línea                                                       | ¿Qué pasa en tiempo de ejecución?                                                     |
|-------|-------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 1     | <code>// src/config/api.js</code>                           | Comentario (no afecta ejecución)                                                      |
| 2     | <code>const API_CONFIG = {</code>                           | Declara un objeto constante                                                           |
| 3-5   | <code>  BASE_URL:<br/>  'http://localhost:3000/api',</code> | URL base del backend (protocolo + host + puerto + /api)                               |
| 6     | <code>  TIMEOUT: 10000,</code>                              | Límite de espera para peticiones (10 segundos)                                        |
| 7     | <code>  ENDPOINTS: {</code>                                 | Objeto anidado con todos los endpoints                                                |
| 8     | <code>    LOGIN: '/users/login',</code>                     | Endpoint para iniciar sesión                                                          |
| 9     | <code>    REGISTER: '/users/create',</code>                 | Endpoint para registrar usuario                                                       |
| 10    | <code>    USERS: '/users',</code>                           | Endpoint para listar usuarios                                                         |
| 11    | <code>    USER_BY_ID: '/users/:id',</code>                  | Endpoint para obtener/actualizar un usuario ( <code>:id</code> es parámetro dinámico) |
| 12    | <code>    USER_DELETE:<br/>    '/users/delete/:id'</code>   | Endpoint para eliminar usuario                                                        |
| 13    | <code>  }</code>                                            | Cierra ENDPOINTS                                                                      |
| 14    | <code>}</code>                                              | Cierra API_CONFIG                                                                     |
| 16    | <code>export default API_CONFIG</code>                      | Exporta el objeto para que otros archivos lo importen                                 |

## Cómo se usa este archivo en otros lugares

En `httpService.js`:

```
javascript
```

```
import API_CONFIG from '../config/api'

class HttpService {
  constructor() {
    this.baseUrl = API_CONFIG.BASE_URL // ← 'http://localhost:3000/api'
  }
}
```

En `AuthModel.js` :

```
javascript

import API_CONFIG from '../config/api'

// Para Login:
const response = await httpService.post(
  API_CONFIG.ENDPOINTS.LOGIN, // ← '/users/login'
  credentials
)

// Para registro:
const response = await httpService.post(
  API_CONFIG.ENDPOINTS.REGISTER, // ← '/users/create'
  userData
)
```

En `UserModel.js` :

```
javascript

import API_CONFIG from '../config/api'

// Obtener todos los usuarios
const response = await httpService.get(API_CONFIG.ENDPOINTS.USERS, true)

// Obtener usuario por ID (con parámetro dinámico)
const endpoint = API_CONFIG.ENDPOINTS.USER_BY_ID.replace(':id', userId)
const response = await httpService.get(endpoint, true)

// Eliminar usuario
const endpoint = API_CONFIG.ENDPOINTS.USER_DELETE.replace(':id', userId)
const response = await httpService.delete(endpoint, true)
```

## Construcción de URLs completas

Este archivo solo define **sufijos** (ej: `/users/login`). La URL completa se construye en `httpService.js` :

```
javascript

// En httpService.js:
const url = `${this.baseUrl}${endpoint}`

// Ejemplos:
BASE_URL + LOGIN    = "http://localhost:3000/api/users/login"
BASE_URL + USERS    = "http://localhost:3000/api/users"
BASE_URL + REGISTER = "http://localhost:3000/api/users/create"
```

## ¿Por qué usar `:id` en los endpoints?

```
javascript

USER_BY_ID: '/users/:id',
USER_DELETE: '/users/delete/:id'
```

El `:id` es un **parámetro dinámico** que debe ser reemplazado antes de usar:

```
javascript

// Si quieres obtener el usuario con ID = 5:
const endpoint = API_CONFIG.ENDPOINTS.USER_BY_ID.replace(':id', 5)
// Resultado: '/users/5'

// Si quieres eliminar el usuario con ID = 10:
const endpoint = API_CONFIG.ENDPOINTS.USER_DELETE.replace(':id', 10)
// Resultado: '/users/delete/10'
```

 **Posibles cambios en BASE\_URL según entorno**

| Entorno          | BASE_URL                      | ¿Cuándo se usa?                             |
|------------------|-------------------------------|---------------------------------------------|
| Desarrollo local | http://localhost:3000/api     | Backend y frontend en la misma PC           |
| Red local        | http://192.168.1.100:3000/api | Probar en celular u otra PC de la misma red |
| Producción       | https://midominio.com/api     | Servidor en la nube                         |

 **Mejora posible: Configuración por entorno**

Podrías mejorar este archivo para que detecte automáticamente el entorno:

```
javascript

// Versión mejorada (opcional)
const getBaseUrl = () => {
  // Si estamos en producción (desplegado en Vercel/Netlify)
  if (import.meta.env.PROD) {
    return 'https://api.midominio.com/api'
  }

  // Si estamos en desarrollo con IP específica
  if (import.meta.env.VITE_API_URL) {
    return import.meta.env.VITE_API_URL
  }

  // Por defecto, Localhost
  return 'http://localhost:3000/api'
}

const API_CONFIG = {
  BASE_URL: getBaseUrl(),
  TIMEOUT: 10000,
  ENDPOINTS: {
    // ...
  }
}
```

 **Resumen de endpoints y sus métodos HTTP**

| Endpoint | Ruta completa | Método | ¿Requiere token?                                                                         | ¿Qué hace?                |
|----------|---------------|--------|------------------------------------------------------------------------------------------|---------------------------|
| LOGIN    | /users/login  | POST   |  No | Autenticar usuario        |
| REGISTER | /users/create | POST   | No                                                                                       | Crear nuevo usuario       |
| USERS    | /users        | GET    |  Sí | Listar todos los usuarios |

| Endpoint    | Ruta completa     | Método | ¿Requiere token? | ¿Qué hace?            |
|-------------|-------------------|--------|------------------|-----------------------|
| USER_BY_ID  | /users/:id        | GET    | ✓ Sí             | Obtener usuario esp   |
| USER_BY_ID  | /users/:id        | PUT    | ✓ Sí             | Actualizar usuario co |
| USER_DELETE | /users/delete/:id | DELETE | ✓ Sí             | Eliminar usuario      |

💡 **Analogía para entenderlo**

Imagina que `api.js` es un **directorio telefónico de la empresa**:

- `BASE_URL` → Es el número de la centralita de la empresa: `+34 91 123 45 67`
- `ENDPOINTS.LOGIN` → Es la extensión del departamento de acceso: `ext. 101`
- Para llamar al departamento de login, marcas: `+34 91 123 45 67 + ext. 101`

Si la empresa se muda a otra ciudad y cambia el número de centralita, solo actualizas `BASE_URL`. Si el departamento de login cambia de extensión, solo actualizas `ENDPOINTS.LOGIN`.

⚠️ **Puntos clave para recordar**

| Concepto                       | Explicación                                                                                                                                         |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| URL base sin barra al final    | <code>http://localhost:3000/api</code> (sin <code>/</code> al final) porque los endpoints empiezan con <code>/</code> → <code>"/users/login"</code> |
| Endpoints con <code>:id</code> | Son placeholders que deben ser reemplazados con <code>replace()</code> antes de usar                                                                |
| TIMEOUT no implementado        | La propiedad existe pero <code>httpService.js</code> aún no la usa. Se podría agregar con <code>AbortController</code>                              |
| Cambiar BASE_URL               | Si pruebas con celular, cambia <code>localhost</code> por la IP de tu PC (ej: <code>http://192.168.1.100:3000/api</code> )                          |

🐛 **Posibles errores en este punto**

| Error                                                                 | Causa                                                  | Solución                                                                       |
|-----------------------------------------------------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>http://localhost:3000/api/users/login</code> no responde        | El backend no está corriendo                           | Ejecuta <code>npm run dev</code> en la carpeta del backend                     |
| <code>http://192.168.x.x:3000/api</code> no funciona desde el celular | Firewall bloquea el puerto 3000                        | Abrir el puerto 3000 en el firewall de Windows/Linux                           |
| Error CORS                                                            | El backend no tiene configurado el origen del frontend | Agregar <code>http://localhost:5173</code> a <code>cors()</code> en el backend |
| <code>USER_BY_ID.replace(':id', id)</code> no funciona                | El <code>id</code> es undefined o null                 | Verificar que el ID exista antes de reemplazar                                 |

¿Entendiste por qué `api.js` es la configuración centralizada de todas las rutas del backend?

Escribe "siguiente" y pasamos al ARCHIVO 12: `config/routes.js` (ya lo vimos, pero ahora en su orden de ejecución real). 📄

siguiente

📄 ARCHIVO 12: `src/config/routes.js` (DUODÉCIMO en ejecutarse)

```
javascript
// src/config/routes.js
export const ROUTES = {
  LOGIN: '/login',
  DASHBOARD: '/dashboard',
  HOME: '/'
}

export const PROTECTED_ROUTES = [ROUTES.DASHBOARD]
export const PUBLIC_ROUTES = [ROUTES.LOGIN, ROUTES.HOME]
```

🔗 Responsabilidad en la ejecución

`routes.js` es un archivo de configuración que se ejecuta cuando `App.jsx` lo importa (y potencialmente otros componentes como `Navbar.jsx` o `LoginView.jsx`). Su responsabilidad es:

- 1. Centralizar todas las rutas de navegación de la aplicación frontend (no confundir con las rutas de la API).
- 2. Clasificar rutas por tipo (públicas vs protegidas) para futuras validaciones.
- 3. Evitar errores de tipeo al escribir rutas manualmente en múltiples archivos.
- 4. Servir como fuente única de verdad para todas las URLs de la aplicación.

🔍 Diferencia entre `config/api.js` y `config/routes.js`

| Archivo                       | ¿Qué define?                             | Ejemplo                                                                |
|-------------------------------|------------------------------------------|------------------------------------------------------------------------|
| <code>config/api.js</code>    | Rutas del <b>backend</b> (API REST)      | <code>/users/login</code> , <code>/users/create</code>                 |
| <code>config/routes.js</code> | Rutas del <b>frontend</b> (React Router) | <code>/login</code> , <code>/dashboard</code> , <code>/register</code> |

Son dos cosas completamente diferentes:

- Las rutas del backend son a donde el frontend hace peticiones HTTP.
- Las rutas del frontend son las URLs que el usuario ve en el navegador.

📄 Cómo se usa este archivo en otros lugares

En `App.jsx` :

```
javascript
import { ROUTES } from './config/routes'

<Routes>
  <Route path={ROUTES.HOME} element={<Navigate to={ROUTES.LOGIN} />} />
  <Route path={ROUTES.LOGIN} element={<LoginView />} />
  <Route path={ROUTES.DASHBOARD} element={<ProtectedRoute><DashboardView /></ProtectedRoute>} />
</Routes>

// Dentro de ProtectedRoute:
```



```

if (!isAuthenticated) {
  return <Navigate to={ROUTES.LOGIN} replace />
}

```

En `Navbar.jsx` :

```

javascript

import { ROUTES } from '../config/routes'

<Link to={ROUTES.LOGIN}>Iniciar Sesión</Link>
<Link to={ROUTES.DASHBOARD}>Dashboard</Link>

```

En `LoginView.jsx` :

```

javascript

import { ROUTES } from '../config/routes'

// Después de Login exitoso:
navigate(ROUTES.DASHBOARD)

```

## Orden de ejecución en tiempo real

```

javascript

// 1. App.jsx se ejecuta
import { ROUTES } from '../config/routes' // ← En este momento se carga routes.js

// 2. routes.js se ejecuta y exporta:
ROUTES = { LOGIN: '/login', DASHBOARD: '/dashboard', HOME: '/' }
PROTECTED_ROUTES = ['/dashboard']
PUBLIC_ROUTES = ['/login', '/']

// 3. App.jsx usa estos valores para definir las rutas
<Route path={ROUTES.LOGIN} ... /> // ← path="/Login"

```

## ¿Por qué son importantes las clasificaciones?

```

javascript

export const PROTECTED_ROUTES = [ROUTES.DASHBOARD]
export const PUBLIC_ROUTES = [ROUTES.LOGIN, ROUTES.HOME]

```

Estos arrays podrían usarse para:

1. **Redirigir usuarios autenticados** fuera de rutas públicas (si ya estás logueado, no deberías ver el login).
2. **Middleware de autenticación** (similar a `ProtectedRoute` pero a nivel de configuración).
3. **Mostrar/ocultar enlaces** según autenticación.

Ejemplo de uso potencial (no implementado en el código actual):

```

javascript

// En un futuro middleware o hook:
const { isAuthenticated } = useAuth()
const location = useLocation()

// Si está autenticado y trata de ir a una ruta pública, redirigir al dashboard
if (isAuthenticated && PUBLIC_ROUTES.includes(location.pathname)) {
  return <Navigate to={ROUTES.DASHBOARD} replace />
}

```

```
// Si no está autenticado y trata de ir a una ruta protegida, redirigir al login
if (!isAuthenticated && PROTECTED_ROUTES.includes(location.pathname)) {
  return <Navigate to={ROUTES.LOGIN} replace />
}
```

## Tabla de rutas del frontend

| Constante        | Valor      | ¿Es pública? | ¿Componente que renderiza?     |
|------------------|------------|--------------|--------------------------------|
| ROUTES.HOME      | /          | Pública      | Redirige a <code>/login</code> |
| ROUTES.LOGIN     | /login     | Pública      | <code>LoginView</code>         |
| ROUTES.DASHBOARD | /dashboard | Protegida    | <code>DashboardView</code>     |

## Errores comunes al no usar este archivo

### Sin `routes.js` (mala práctica):

```
javascript

// En App.jsx
<Route path="/login" element={<LoginView />} />

// En Navbar.jsx
<Link to="/login">Iniciar Sesión</Link>

// En LoginView.jsx
navigate('/dashboard')

// Si quieres cambiar /dashboard a /panel, tienes que modificar 3+ archivos
```

### Con `routes.js` (buena práctica):

```
javascript

// En App.jsx
<Route path={ROUTES.LOGIN} element={<LoginView />} />

// En Navbar.jsx
<Link to={ROUTES.LOGIN}>Iniciar Sesión</Link>

// En LoginView.jsx
navigate(ROUTES.DASHBOARD)

// Si quieres cambiar /dashboard a /panel, solo modificas routes.js
```

## Posibles mejoras futuras

```
javascript

// Podrías expandir routes.js para incluir más información:
export const ROUTES = {
  LOGIN: { path: '/login', title: 'Iniciar Sesión', public: true },
  REGISTER: { path: '/register', title: 'Registrarse', public: true },
  DASHBOARD: { path: '/dashboard', title: 'Panel de Control', public: false, roles: ['admin', 'seller'] },
  USERS: { path: '/dashboard/usuarios', title: 'Usuarios', public: false, roles: ['admin', 'seller'] },
  HOME: { path: '/', title: 'Inicio', public: true }
}

// Función helper para obtener el título de una ruta:
export const getRouteTitle = (path) => {
  const route = Object.values(ROUTES).find(r => r.path === path)
```

```
return route?.title || 'Sistema Web'
}
```

## 💡 Analogía para entenderlo

Imagina que `routes.js` es como un **mapa de la aplicación**:

- `ROUTES.LOGIN = '/login'` → Es como decir "La recepción está en el primer piso, oficina 101"
- `ROUTES.DASHBOARD = '/dashboard'` → Es como "La oficina de administración está en el piso 5, al fondo"
- `PROTECTED_ROUTES` → Es la lista de oficinas que requieren tarjeta de acceso
- `PUBLIC_ROUTES` → Es la lista de áreas de acceso libre (baños, cafetería, recepción)

Cuando alguien quiere ir a la recepción, no dice "voy a la oficina 101", dice "voy a la recepción" y el mapa traduce. Si la recepción se muda a la oficina 202, solo actualizas el mapa, no le dices a cada persona que cambie su destino.

## ⚠️ Puntos clave para recordar

| Concepto                      | Explicación                                                                                                                                          |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rutas del frontend vs backend | <code>routes.js</code> es para URLs del navegador, <code>api.js</code> es para endpoints del servidor                                                |
| Constantes vs strings mágicos | Usar <code>ROUTES.LOGIN</code> es más seguro que <code>"/login"</code> (autocompletado, evita errores)                                               |
| Clasificación no utilizada    | <code>PROTECTED_ROUTES</code> y <code>PUBLIC_ROUTES</code> están definidos pero NO se usan en el código actual (solo como documentación/para futuro) |
| Solo rutas principales        | El dashboard tiene tabs internos, pero esas subrutas no están definidas aquí                                                                         |

¿Entendiste por qué `routes.js` centraliza las rutas del frontend para evitar errores de tipeo?

Escribe "siguiente" y pasamos al ARCHIVO 13: `LoginView.jsx` (la pantalla que ve el usuario al abrir la app). 🗝️

siguiente

## 📄 ARCHIVO 13: `src/views/auth/LoginView.jsx` (DECIMOTERCERO en ejecutarse)

```
javascript
// src/views/auth/LoginView.jsx
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import useAuth from '../../hooks/useAuth'
import AlertMessage from '../../common/AlertMessage'
import '../../styles/Login.css'

const LoginView = () => {
  const [credentials, setCredentials] = useState({
    email: '',
    password: ''
  })
}
```

```

const [loading, setLoading] = useState(false)
const [error, setError] = useState('')

const { login } = useAuth()
const navigate = useNavigate()

const handleChange = (e) => {
  const { name, value } = e.target
  setCredentials(prev => ({
    ...prev,
    [name]: value
  }))
  if (error) setError('')
}

const handleSubmit = async (e) => {
  e.preventDefault()

  console.log('🚀 Intentando login con:', credentials.email)

  if (!credentials.email || !credentials.password) {
    setError('Por favor complete todos los campos')
    return
  }

  setLoading(true)

  try {
    const result = await login(credentials)
    console.log('✅ Login exitoso:', result)
    navigate('/dashboard')
  } catch (err) {
    console.error('❌ Error en login:', err)
    setError(err.error || 'Error al iniciar sesión. Verifica tus credenciales.')
  } finally {
    setLoading(false)
  }
}

return (
  <div className="login-container">
    <div className="login-card">
      <div className="login-header">
        <h1>Sistema de Información Web</h1>
        <p>Inicie sesión para continuar</p>
      </div>

      {error && (
        <AlertMessage
          type="error"
          message={error}
          onClose={() => setError('')}
        />
      )}

      <form onSubmit={handleSubmit} className="login-form">
        <div className="form-group">
          <label htmlFor="email">Correo Electrónico</label>
          <input
            type="email"
            id="email"
            name="email"
            value={credentials.email}
            onChange={handleChange}
            placeholder="usuario@ejemplo.com"
            disabled={loading}
            autoComplete="off"
          />
        </div>

        <div className="form-group">

```

```

    <label htmlFor="password">Contraseña</label>
    <input
      type="password"
      id="password"
      name="password"
      value={credentials.password}
      onChange={handleChange}
      placeholder="*****"
      disabled={loading}
      autoComplete="off"
    />
  </div>

  <button
    type="submit"
    className="login-button"
    disabled={loading}
  >
    {loading ? 'Iniciando sesión...' : 'Iniciar Sesión'}
  </button>
</form>

<div className="login-footer">
  <p>Prueba con los usuarios de tu base de datos:</p>
  <p className="demo-credentials">
    <strong>Email:</strong> oliver@stone.com<br />
    <strong>Contraseña:</strong> [la que usaste para hashear]
  </p>
</div>
</div>
</div>
)
}

export default LoginView

```

## Responsabilidad en la ejecución

`LoginView.jsx` es un **componente de vista** que se ejecuta cuando la URL es `/login` (según lo definido en `App.jsx`). Su responsabilidad es:

1. **Mostrar el formulario de inicio de sesión** (campos de email y contraseña).
2. **Capturar los datos ingresados** por el usuario y guardarlos en el estado local.
3. **Validar que los campos no estén vacíos** antes de intentar el login.
4. **Llamar al hook** `useAuth().login()` para autenticar al usuario.
5. **Manejar estados de carga** (deshabilitar botón, mostrar "Iniciando sesión...").
6. **Mostrar errores** si el login falla (credenciales incorrectas, servidor caído).
7. **Redirigir al dashboard** si el login es exitoso.

## Orden de ejecución línea por línea

| Orden | Línea                                                                                    | ¿Qué pasa en tiempo de ejecución?          |
|-------|------------------------------------------------------------------------------------------|--------------------------------------------|
| 1-5   | <code>import ...</code>                                                                  | Importa hooks, dependencias, estilos       |
| 7     | <code>const LoginView = () =&gt; {</code>                                                | Define el componente                       |
| 8-12  | <code>const [credentials, setCredentials] = useState({ email: '', password: '' })</code> | Crea estado para los campos del formulario |

| Orden  | Línea                                                                                | ¿Qué pasa en tiempo de ejecución?                                       |
|--------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| 13     | <code>const [loading, setLoading] = useState(false)</code>                           | Estado de carga (inicialmente falso)                                    |
| 14     | <code>const [error, setError] = useState('')</code>                                  | Estado para mensajes de error                                           |
| 16     | <code>const { login } = useAuth()</code>                                             | 🔥 EJECUTA <code>useAuth()</code> y extrae la función <code>login</code> |
| 17     | <code>const navigate = useNavigate()</code>                                          | Obtiene función para cambiar la URL programáticamente                   |
| 19     | <code>const handleChange = (e) =&gt; { ... }</code>                                  | Define función (se ejecuta CADA VEZ que el usuario escribe)             |
| 20-26  | <code>setCredentials(...)</code>                                                     | Actualiza el estado del campo que cambió                                |
| 27     | <code>if (error) setError('')</code>                                                 | Borra el error cuando el usuario empieza a escribir                     |
| 30     | <code>const handleSubmit = async (e) =&gt; { ... }</code>                            | Define función (se ejecuta cuando el usuario ENVÍA el formulario)       |
| 31     | <code>e.preventDefault()</code>                                                      | Evita que el navegador recargue la página                               |
| 34-36  | <code>if (!credentials.email    !credentials.password) { setError(...) }</code>      | Valida que los campos no estén vacíos                                   |
| 38     | <code>setLoading(true)</code>                                                        | Cambia estado de carga → botón se deshabilita y muestra texto           |
| 40-46  | <code>try { const result = await login(credentials); navigate('/dashboard') }</code> | 🔥 LLAMA AL LOGIN y espera; si funciona, redirige                        |
| 47-49  | <code>catch (err) { setError(err.error) }</code>                                     | Si hay error, lo muestra al usuario                                     |
| 50     | <code>finally { setLoading(false) }</code>                                           | Restablece estado de carga (pase lo que pase)                           |
| 54-110 | <code>return (...)</code>                                                            | Renderiza el HTML del formulario                                        |

## ⚡ Escenario real: Usuario escribe en el campo email

javascript

*// El usuario escribe "o" en el campo email*

1. `onChange` se dispara
2. `handleChange(e)` se ejecuta
  - `e.target.name = "email"`
  - `e.target.value = "o"`
3. `setCredentials(prev => ({ ...prev, email: "o" })))`
4. React actualiza el estado `credentials = { email: "o", password: "" }`
5. React **RE-RENDERIZA** `LoginView` con el nuevo valor en el input
6. El usuario ve "o" en el campo email

```
// El usuario sigue escribiendo "oliver@stone.com"
// El mismo proceso ocurre por cada letra
```

## ⚡ Escenario real: Usuario hace clic en "Iniciar Sesión"

javascript

```
// El usuario completa email y contraseña y hace clic
```

1. `onSubmit` se **dispara** (por el form)
2. `handleSubmit(e)` se ejecuta
3. `e.preventDefault()` evita recarga
4. **Validación:** `credentials.email` ☒, `credentials.password`
5. `setLoading(true)` → botón muestra "Iniciando sesión..." y se deshabilita
6. `await login(credentials)` → llama a `useAuth().login()`
  - `useAuth().login()` llama a `AuthController.handleLogin()`
  - `AuthController` valida y llama a `AuthModel.login()`
  - `AuthModel` hace petición **HTTP** al backend
  - 🕒 El usuario **espera** (200-500ms)
7. Si la respuesta es exitosa:
  - `login()` resuelve la promesa
  - `navigate('/dashboard')` cambia la URL
  - El usuario ve el dashboard
8. Si hay error:
  - `catch` se ejecuta
  - `setError("Credenciales incorrectas")`
  - El usuario ve el mensaje de error en el formulario
9. `finally { setLoading(false) }` → botón vuelve a estado normal

## 📊 Evolución del estado durante el login

| Momento         | credentials                                                    | loading | error                      | UI                                        |
|-----------------|----------------------------------------------------------------|---------|----------------------------|-------------------------------------------|
| Inicial         | <code>{ email: "", password: "" }</code>                       | false   | ""                         | Botón "Iniciar Sesión"                    |
| Usuario escribe | <code>{ email: "oliver@stone.com", password: "123456" }</code> | false   | ""                         | Botón "Iniciar Sesión"                    |
| Clic en botón   | Mismo valor                                                    | true    | ""                         | Botón deshabilitado "Iniciando sesión..." |
| Login exitoso   | Mismo valor                                                    | false   | ""                         | Redirige a <code>/dashboard</code>        |
| Login fallido   | Mismo valor                                                    | false   | "Credenciales incorrectas" | Muestra error, botón "Intentar de nuevo"  |

## 💡 Por qué el formulario usa `autocomplete="off"`

javascript

```
autocomplete="off"
```

Esto evita que el navegador autocomplete los campos (útil para desarrollo). En producción, quizás quieras quitarlo para permitir que el navegador guarde contraseñas.

## 🔄 Flujo de actualización del estado en tiempo real

text

Usuario escribe "o"

↓

`handleChange` → `setCredentials({ email: "o", password: "" })`

```

↓
React re-renderiza LoginView
↓
Input muestra "o"

Usuario escribe "1"
↓
handleChange → setCredentials({ email: "ol", password: "" })
↓
React re-renderiza LoginView
↓
Input muestra "ol"

... y así sucesivamente

```

## 💡 Analogía para entenderlo

Imagina que `LoginView.jsx` es un **cajero automático**:

- `credentials` → Es la memoria temporal donde guardas el número de tarjeta y PIN que vas escribiendo.
- `handleChange` → Es el mecanismo que registra cada tecla que presionas.
- `handleSubmit` → Es el botón "Aceptar" que envía tus datos al banco.
- `loading` → Es la pantalla "Procesando..." que aparece mientras el banco verifica.
- `error` → Es el mensaje "PIN incorrecto" que ves si fallas.
- `navigate('/dashboard')` → Es la pantalla "Bienvenido" que ves después de un login exitoso.

## ⚠️ Puntos clave para recordar

| Concepto                             | Explicación                                                                                       |
|--------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>e.preventDefault()</code>      | Es CRUCIAL. Sin esto, el formulario recargaría la página y perderías el estado de React           |
| <code>loading</code>                 | No solo es visual: deshabilita el botón para evitar múltiples envíos mientras ya hay uno en curso |
| <code>finally</code>                 | Asegura que <code>setLoading(false)</code> se ejecute SIEMPRE, haya éxito o error                 |
| <code>if (error) setError('')</code> | Borra el error en cuanto el usuario empieza a escribir, buena experiencia de usuario              |
| <code>autoComplete="off"</code>      | Previene que el navegador autocomplete campos que no quieres (útil en desarrollo)                 |

## 🐛 Posibles errores en este punto

| Error                                | Causa                                                        | Solución                                                                       |
|--------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>login is not a function</code> | <code>useAuth()</code> no está retornando <code>login</code> | Verifica que <code>useAuth.js</code> exporte <code>login</code> en el return   |
| El formulario recarga la página      | Falta <code>e.preventDefault()</code>                        | Agregar <code>e.preventDefault()</code> al inicio de <code>handleSubmit</code> |
| El error no se muestra               | <code>AlertMessage</code> no recibe bien las props           | Verifica que <code>AlertMessage</code> esté correctamente                      |



| Error                                        | Causa                                                                | Solución                                                                                         |
|----------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
|                                              |                                                                      | importado                                                                                        |
| <code>navigate</code> no funciona            | No hay <code>&lt;BrowserRouter&gt;</code> arriba en el árbol         | <code>main.jsx</code> debe tener <code>&lt;BrowserRouter&gt;</code> envolviendo <code>App</code> |
| El botón se deshabilita pero no cambia texto | <code>loading</code> se actualiza pero el texto condicional está mal | Verificar <code>{loading ? 'Iniciando...' : 'Iniciar Sesión'}</code>                             |

¿Entendiste por qué `LoginView.jsx` maneja el estado local del formulario, valida campos y llama a `useAuth().login()` ?

Escribe "siguiente" y pasamos al ARCHIVO 14: `RegisterView.jsx` (similar pero para registro de nuevos usuarios). 🐛

siguiente

## 📄 ARCHIVO 14: `src/views/auth/RegisterView.jsx` (DECIMOCUARTO en ejecutarse)

```
javascript
// src/views/auth/RegisterView.jsx
import { useState } from 'react'
import { useNavigate, Link } from 'react-router-dom'
import useAuth from '../../hooks/useAuth'
import AlertMessage from '../../common/AlertMessage'
import '../../styles/Register.css'

const RegisterView = () => {
  const [formData, setFormData] = useState({
    name: '',
    lastname: '', // ← NUEVO: Apellido
    email: '',
    password: '',
    confirmPassword: '',
    phone: '', // ← NUEVO: Teléfono
    role: 'user' // ← NUEVO: Rol (por defecto 'user')
  })
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')
  const [success, setSuccess] = useState('')

  const { register } = useAuth()
  const navigate = useNavigate()

  const handleChange = (e) => {
    const { name, value } = e.target
    setFormData(prev => ({
      ...prev,
      [name]: value
    }))
    if (error) setError('')
  }

  const handleSubmit = async (e) => {
    e.preventDefault()

    console.log('🐛 Intentando registrar:', formData.email)

    // Validaciones
    if (!formData.name || !formData.lastname || !formData.email || !formData.password) {
```

```

        setError('Por favor complete todos los campos obligatorios')
        return
    }

    if (formData.password !== formData.confirmPassword) {
        setError('Las contraseñas no coinciden')
        return
    }

    if (formData.password.length < 5) {
        setError('La contraseña debe tener al menos 6 caracteres')
        return
    }

    // Validar formato de email
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
    if (!emailRegex.test(formData.email)) {
        setError('Por favor ingrese un email válido')
        return
    }

    setLoading(true)

    try {
        // Enviar todos Los campos que La API espera
        const result = await register({
            name: formData.name,
            lastname: formData.lastname,
            email: formData.email,
            password: formData.password,
            phone: formData.phone,
            role: formData.role
            // image se puede agregar después
        })

        console.log('✅ Registro exitoso:', result)
        setSuccess('Usuario registrado exitosamente. Redirigiendo al login...')

        setTimeout(() => {
            navigate('/login')
        }, 2000)

    } catch (err) {
        console.error('❌ Error en registro:', err)
        setError(err.error || 'Error al registrar usuario. Intente nuevamente.')
    } finally {
        setLoading(false)
    }
}

return (
    <div className="register-container">
        <div className="register-card">
            <div className="register-header">
                <h1>Crear Cuenta</h1>
                <p>Regístrate para acceder al sistema</p>
            </div>

            {error && (
                <AlertMessage
                    type="error"
                    message={error}
                    onClose={() => setError('')}
                />
            )}

            {success && (
                <AlertMessage
                    type="success"
                    message={success}
                    onClose={() => setSuccess('')}
            )}
        </div>
    </div>
)

```

```

    />
  })

  <form onSubmit={handleSubmit} className="register-form">
    <div className="form-row">
      <div className="form-group">
        <label htmlFor="name">Nombre *</label>
        <input
          type="text"
          id="name"
          name="name"
          value={formData.name}
          onChange={handleChange}
          placeholder="Ingrese su nombre"
          disabled={loading}
          required
        />
      </div>

      <div className="form-group">
        <label htmlFor="lastname">Apellido *</label>
        <input
          type="text"
          id="lastname"
          name="lastname"
          value={formData.lastname}
          onChange={handleChange}
          placeholder="Ingrese su apellido"
          disabled={loading}
          required
        />
      </div>
    </div>

    <div className="form-group">
      <label htmlFor="email">Correo Electrónico *</label>
      <input
        type="email"
        id="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
        placeholder="usuario@ejemplo.com"
        disabled={loading}
        required
      />
    </div>

    <div className="form-group">
      <label htmlFor="phone">Teléfono</label>
      <input
        type="tel"
        id="phone"
        name="phone"
        value={formData.phone}
        onChange={handleChange}
        placeholder="Opcional"
        disabled={loading}
      />
    </div>

    <div className="form-row">
      <div className="form-group">
        <label htmlFor="password">Contraseña *</label>
        <input
          type="password"
          id="password"
          name="password"
          value={formData.password}
          onChange={handleChange}
          placeholder="Mínimo 6 caracteres"
        />
      </div>
    </div>
  </form>

```

```

        disabled={loading}
        required
      />
    </div>

    <div className="form-group">
      <label htmlFor="confirmPassword">Confirmar Contraseña *</label>
      <input
        type="password"
        id="confirmPassword"
        name="confirmPassword"
        value={formData.confirmPassword}
        onChange={handleChange}
        placeholder="Repita su contraseña"
        disabled={loading}
        required
      />
    </div>
  </div>

  <div className="form-group">
    <label htmlFor="role">Rol</label>
    <select
      id="role"
      name="role"
      value={formData.role}
      onChange={handleChange}
      disabled={loading}
      className="form-control"
    >
      <option value="user">Usuario</option>
      <option value="customer">Cliente</option>
      <option value="seller">Vendedor</option>
      <option value="admin">Administrador</option>
    </select>
    <small className="form-hint">El rol por defecto es "usuario"</small>
  </div>

  <button
    type="submit"
    className="register-button"
    disabled={loading}
  >
    {loading ? 'Registrando...' : 'Registrarse'}
  </button>
</form>

<div className="register-footer">
  <p>¿Ya tienes cuenta? <Link to="/login">Inicia Sesión aquí</Link></p>
</div>
</div>
</div>
)
}

export default RegisterView

```

## Responsabilidad en la ejecución

`RegisterView.jsx` es un **componente de vista** que se ejecuta **cuando la URL es** `/register` (según lo definido en `App.jsx`). Su responsabilidad es:

1. **Mostrar el formulario de registro** con campos para nombre, apellido, email, contraseña, confirmación, teléfono y rol.
2. **Validar los datos del formulario** antes de enviarlos (campos obligatorios, coincidencia de contraseñas, longitud mínima, formato de email).
3. **Llamar al hook** `useAuth().register()` para crear un nuevo usuario en el backend.

4. Manejar estados de carga (deshabilitar botón, mostrar "Registrando...").
5. Mostrar mensajes de éxito o error al usuario.
6. Redirigir al login después de un registro exitoso (con un delay de 2 segundos para que el usuario vea el mensaje de éxito).

### Orden de ejecución línea por línea

| Orden  | Línea                                                          | ¿Qué pasa en tiempo de ejecución?                                                                                                                          |
|--------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-5    | <code>import ...</code>                                        | Importa hooks, dependencias, estilos                                                                                                                       |
| 7      | <code>const RegisterView = () =&gt; {</code>                   | Define el componente                                                                                                                                       |
| 8-18   | <code>const [formData, setFormData] = useState({ ... })</code> | Crea estado con todos los campos del formulario                                                                                                            |
| 19     | <code>const [loading, setLoading] = useState(false)</code>     | Estado de carga                                                                                                                                            |
| 20     | <code>const [error, setError] = useState('')</code>            | Estado para errores                                                                                                                                        |
| 21     | <code>const [success, setSuccess] = useState('')</code>        | Estado para mensaje de éxito                                                                                                                               |
| 23     | <code>const { register } = useAuth()</code>                    |  EJECUTA <code>useAuth()</code> y extrae la función <code>register</code> |
| 24     | <code>const navigate = useNavigate()</code>                    | Obtiene función para cambiar URL                                                                                                                           |
| 26     | <code>const handleChange = (e) =&gt; { ... }</code>            | Define función para cambios en inputs                                                                                                                      |
| 34     | <code>const handleSubmit = async (e) =&gt; { ... }</code>      | Define función para envío del formulario                                                                                                                   |
| 35     | <code>e.preventDefault()</code>                                | Evita recarga de página                                                                                                                                    |
| 38-73  | Validaciones secuenciales                                      | Verifican que los datos sean correctos                                                                                                                     |
| 75     | <code>setLoading(true)</code>                                  | Activa estado de carga                                                                                                                                     |
| 77-86  | <code>const result = await register({ ... })</code>            |  LLAMA AL REGISTRO                                                      |
| 88-89  | <code>setSuccess(...)</code> y <code>setTimeout(...)</code>    | Muestra éxito y programa redirección                                                                                                                       |
| 91-93  | <code>catch (err) { setError(...) }</code>                     | Captura y muestra errores                                                                                                                                  |
| 94     | <code>finally { setLoading(false) }</code>                     | Restablece estado de carga                                                                                                                                 |
| 98-188 | <code>return (...)</code>                                      | Renderiza el HTML del formulario                                                                                                                           |

### Escenario real: Usuario completa todos los campos correctamente

```
javascript
```

```
// El usuario ingresa:
// Nombre: "Oliver"
// Apellido: "Stone"
// Email: "oliver@stone.com"
// Teléfono: "123456789"
// Contraseña: "123456"
// Confirmar: "123456"
// Rol: "admin"

// Hace clic en "Registrarse"

1. handleSubmit se ejecuta
2. Validaciones:
  - name ✅ "Oliver" existe
  - lastname ✅ "Stone" existe
  - email ✅ "oliver@stone.com" existe
  - password ✅ "123456" existe
  - password === confirmPassword ✅ "123456" === "123456"
  - password.length >= 6 ✅ 6 >= 6
  - emailRegex.test() ✅ formato válido

3. setLoading(true) → botón muestra "Registrando..." y se deshabilita

4. await register({
  name: "Oliver",
  lastname: "Stone",
  email: "oliver@stone.com",
  password: "123456",
  phone: "123456789",
  role: "admin"
})

5. 🛠 El backend procesa el registro (crea usuario en BD)

6. La promesa se resuelve → setSuccess("Usuario registrado exitosamente...")

7. setTimeout de 2 segundos → navigate('/login')

8. finally { setLoading(false) }
```

## 📁 Validaciones en orden secuencial

javascript

```
// Las validaciones se ejecutan en este orden:

// 1. Campos obligatorios
if (!formData.name || !formData.lastname || !formData.email || !formData.password) {
  setError('Por favor complete todos los campos obligatorios')
  return // ← DETIENE la ejecución, no sigue
}

// 2. Coincidencia de contraseñas
if (formData.password !== formData.confirmPassword) {
  setError('Las contraseñas no coinciden')
  return // ← DETIENE
}

// 3. Longitud mínima de contraseña
if (formData.password.length < 5) { // ← Nota: La condición es < 5 (mínimo 6 caracteres)
  setError('La contraseña debe tener al menos 6 caracteres')
  return // ← DETIENE
}

// 4. Formato de email
const emailRegex = /^[^\\s@]+@[^\\s@]+\\.\\.[^\\s@]+$/
if (!emailRegex.test(formData.email)) {
```

```
setError('Por favor ingrese un email válido')
return // ← DETIENE
}

// Si todas las validaciones pasan, recién se llama a register()
```

## 💡 Diferencias clave con LoginView

| Característica      | LoginView                           | RegisterView                                                    |
|---------------------|-------------------------------------|-----------------------------------------------------------------|
| Número de campos    | 2 (email, password)                 | 7 (nombre, apellido, email, password, confirmar, teléfono, rol) |
| Función de auth     | <code>login()</code>                | <code>register()</code>                                         |
| Redirección después | Inmediata a <code>/dashboard</code> | Delay de 2 segundos a <code>/login</code>                       |
| Mensajes            | Solo error                          | Error y éxito                                                   |
| Enlace adicional    | No                                  | "¿Ya tienes cuenta? Inicia Sesión"                              |
| Selector de rol     | No                                  | Sí (con rol por defecto "user")                                 |

## 🔄 Flujo de estados durante el registro

| Momento               | <code>formData</code>        | <code>loading</code> | <code>error</code> | <code>success</code> |
|-----------------------|------------------------------|----------------------|--------------------|----------------------|
| Inicial               | Valores vacíos, role: 'user' | <code>false</code>   | <code>""</code>    | <code>""</code>      |
| Usuario escribe       | Datos parciales              | <code>false</code>   | <code>""</code>    | <code>""</code>      |
| Clic en registrar     | Datos completos              | <code>true</code>    | <code>""</code>    | <code>""</code>      |
| Registro exitoso      | Datos completos              | <code>false</code>   | <code>""</code>    | "Usuario n"          |
| Después de 2 segundos | -                            | -                    | -                  | Redirige a           |

## 💡 Analogía para entenderlo

Imagina que `RegisterView.jsx` es un **formulario de registro en una biblioteca**:

- `formData` → Es la hoja de papel donde vas escribiendo tus datos (nombre, email, teléfono).
- `handleChange` → Es el lápiz con el que vas llenando cada campo.
- Validaciones** → Es el bibliotecario que revisa que hayas llenado todos los campos obligatorios y que la contraseña tenga al menos 6 caracteres.
- `register()` → Es el proceso de crear tu cuenta en el sistema.
- `success` → Es el mensaje "¡Registro exitoso! Revisa tu correo para activar tu cuenta".
- `navigate('/login')` → Es la persona que te dice "Ahora ve a la entrada principal para iniciar sesión con tus nuevas credenciales".

## ⚠️ Puntos clave para recordar

| Concepto                     | Explicación                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------|
| <code>confirmPassword</code> | No se envía al backend, solo se usa para validar que el usuario escribió bien su contraseña |

| Concepto                                               | Explicación                                                                                                  |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Rol por defecto 'user'                                 | Si el usuario no selecciona un rol, se envía 'user' (seguridad: evita que cualquiera se registre como admin) |
| Selector de rol                                        | En producción, deberías ocultar esto para usuarios normales (solo admin puede asignar roles)                 |
| <code>setTimeout</code> antes de <code>navigate</code> | Da tiempo al usuario para leer el mensaje "Registro exitoso" antes de redirigir                              |
| Validación de email                                    | Usa una expresión regular básica (no cubre todos los casos, pero es suficiente para la mayoría)              |

## Posibles errores en este punto

| Error                                                                                | Causa                                                                | Solución                                                                                 |
|--------------------------------------------------------------------------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>register is not a function</code>                                              | <code>useAuth()</code> no retorna <code>register</code>              | Verificar que <code>useAuth.js</code> exporte <code>register</code>                      |
| El registro falla con error 500                                                      | El backend no tiene la tabla <code>users</code> o campos incorrectos | Verificar la estructura de la base de datos                                              |
| La validación de contraseña dice "mínimo 6" pero la condición es <code>&lt; 5</code> | El mensaje y la condición no coinciden                               | Cambiar condición a <code>&lt; 6</code> o mensaje a "al menos 5"                         |
| El rol siempre se guarda como 'user' aunque selecciones otro                         | El backend o el modelo ignoran el campo <code>role</code>            | Verificar que <code>AuthModel.register()</code> incluya <code>role</code> en la petición |
| El teléfono no se guarda                                                             | El backend no tiene columna <code>phone</code>                       | Agregar columna <code>phone</code> a la tabla <code>users</code>                         |

## Nota sobre la validación de contraseña

```
javascript
if (formData.password.length < 5) {
  setError('La contraseña debe tener al menos 6 caracteres')
  return
}
```

Aquí hay una **inconsistencia**: la condición dice `< 5` (menor a 5, es decir 4 caracteres o menos) pero el mensaje dice "al menos 6 caracteres". El backend también debe tener la misma regla (bcrypt generalmente no tiene problema con contraseñas cortas, pero es buena práctica pedir mínimo 6).

¿Entendiste por qué `RegisterView.jsx` valida múltiples campos antes de llamar a `register()` y muestra mensajes de éxito antes de redirigir?

Escribe "siguiente" y pasamos al ARCHIVO 15: `DashboardView.jsx` (la pantalla principal después del login). 

[siguiente](#)



## ARCHIVO 15: src/views/dashboard/DashboardView.jsx

### (DECIMOQUINTO en ejecutarse)

javascript

```
// src/views/dashboard/DashboardView.jsx
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import useAuth from '../../../hooks/useAuth'
import DashboardHeader from './DashboardHeader'
import DashboardStats from './DashboardStats'
import UsersView from './UsersView'
import AlertMessage from '../../../common/AlertMessage'
import LoadingSpinner from '../../../common/LoadingSpinner'
// IMPORTANTE: Importar los estilos
import '../../../styles/Dashboard.css'
import '../../../styles/Users.css'

const DashboardView = () => {
  const { currentUser, logout } = useAuth()
  const [activeTab, setActiveTab] = useState('dashboard')
  const [logoutMessage, setLogoutMessage] = useState('')
  const navigate = useNavigate()

  const handleLogout = async () => {
    try {
      await logout()
      setLogoutMessage('Sesión cerrada exitosamente')
      setTimeout(() => {
        navigate('/login')
      }, 1500)
    } catch (error) {
      console.error('Error al cerrar sesión:', error)
    }
  }

  const renderContent = () => {
    switch (activeTab) {
      case 'dashboard':
        return (
          <>
            <div className="dashboard-welcome">
              <h2>Bienvenido, {currentUser?.name || currentUser?.email?.split('@')[0] || 'Usuario'}!</h2>
              <p>Este es tu panel de administración</p>
            </div>

            <DashboardStats stats={{
              activeUsers: 150,
              todayVisits: 45,
              activeSessions: 23,
              successRate: 95
            }} />

            <div className="dashboard-info">
              <div className="info-card">
                <h3>Información del Sistema</h3>
                <p>Este sistema utiliza un patrón arquitectónico MVC con React.</p>
              </div>

              <ul>
                <li>✔ Autenticación JWT simulada</li>
                <li>✔ Almacenamiento en localStorage</li>
                <li>✔ Rutas protegidas</li>
                <li>✔ CRUD de usuarios completo</li>
              </ul>
            </div>

            <div className="info-card">
              <h3>Estado de Autenticación</h3>
              <p><strong>Token almacenado:</strong> {localStorage.getItem('auth_
```

```

token') ? '✓ Activo' : '✗ No encontrado']}</p>
    <p><strong>Usuario actual:</strong> {currentUser?.email}</p>
    <p><strong>Rol:</strong> Administrador</p>
  </div>
</div>
</>
)
case 'users':
  return <UsersView />
default:
  return null
}
}

return (
  <div className="dashboard-container">
    <DashboardHeader
      user={currentUser}
      onLogout={handleLogout}
      activeTab={activeTab}
      onTabChange={setActiveTab}
    />

    {logoutMessage && (
      <AlertMessage
        type="success"
        message={logoutMessage}
        onClose={() => setLogoutMessage('')}
      />
    )}

    <main className="dashboard-main">
      {renderContent()}
    </main>
  </div>
)
}

export default DashboardView

```

## Responsabilidad en la ejecución

`DashboardView.jsx` es un componente de vista protegido que se ejecuta cuando la URL es `/dashboard` Y el usuario está autenticado (después de pasar por `ProtectedRoute`). Su responsabilidad es:

1. **Mostrar el panel principal** con tabs para cambiar entre diferentes secciones (Dashboard principal y Gestión de Usuarios).
2. **Gestionar la pestaña activa** mediante estado local (`activeTab`).
3. **Renderizar contenido diferente** según la pestaña seleccionada usando `renderContent()`.
4. **Proveer el header del dashboard** (`DashboardHeader`) con información del usuario y botón de logout.
5. **Manejar el cierre de sesión** con un mensaje de éxito y redirección después de 1.5 segundos.
6. **Mostrar estadísticas estáticas** (simuladas) en la pestaña de dashboard.

## Orden de ejecución línea por línea

| Orden | Línea                                                                                                                                                  | ¿Qué pasa en tiempo de ejecución?                                             |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| 1-10  | <code>import ...</code>                                                                                                                                | Importa hooks, componentes, estilos                                           |
| 12    | <code>const DashboardView = () =&gt; {</code>                                                                                                          | Define el componente                                                          |
| 13    | <code>const { currentUser, logout }<br/>= useAuth()</code>                                                                                             | 🔥 <b>EJECUTA useAuth()</b> para obtener usuario y función logout              |
| 14    | <code>const [activeTab,<br/>setActiveTab] =<br/>useState('dashboard')</code>                                                                           | Estado para saber qué pestaña está activa (inicia en 'dashboard')             |
| 15    | <code>const [logoutMessage,<br/>setLogoutMessage] =<br/>useState('')</code>                                                                            | Estado para mensaje de éxito al cerrar sesión                                 |
| 16    | <code>const navigate = useNavigate()</code>                                                                                                            | Función para cambiar URL                                                      |
| 18    | <code>const handleLogout = async ()<br/>=&gt; { ... }</code>                                                                                           | Define función para cerrar sesión                                             |
| 19    | <code>try {</code>                                                                                                                                     | Inicia bloque de manejo de errores                                            |
| 20    | <code>await logout()</code>                                                                                                                            | 🔥 <b>LLAMA A logout()</b> del hook                                            |
| 21    | <code>setLogoutMessage('Sesión<br/>cerrada exitosamente')</code>                                                                                       | Muestra mensaje de éxito                                                      |
| 22-24 | <code>setTimeout(() =&gt; {<br/>navigate('/login') }, 1500)</code>                                                                                     | Espera 1.5 segundos y redirige al login                                       |
| 25-27 | <code>catch (error) {<br/>console.error(...) }</code>                                                                                                  | Captura errores                                                               |
| 30    | <code>const renderContent = () =&gt; {<br/>... }</code>                                                                                                | Define función que retorna diferente contenido según la pestaña               |
| 31    | <code>switch (activeTab) {</code>                                                                                                                      | Evalúa qué pestaña está activa                                                |
| 32-65 | <code>case 'dashboard': return ( ...<br/>)</code>                                                                                                      | Si es 'dashboard', muestra bienvenida, estadísticas y tarjetas de información |
| 66-67 | <code>case 'users': return<br/>&lt;UsersView /&gt;</code>                                                                                              | Si es 'users', 🔥 <b>EJECUTA UsersView</b>                                     |
| 70    | <code>return (</code>                                                                                                                                  | Renderiza el componente completo                                              |
| 71    | <code>&lt;div className="dashboard-<br/>container"&gt;</code>                                                                                          | Contenedor principal                                                          |
| 72-76 | <code>&lt;DashboardHeader user=<br/>{currentUser} onLogout=<br/>{handleLogout} activeTab=<br/>{activeTab} onTabChange=<br/>{setActiveTab} /&gt;</code> | 🔥 <b>EJECUTA DashboardHeader</b> y le pasa props                              |
| 78-81 | <code>{logoutMessage &amp;&amp;<br/>&lt;AlertMessage ... /&gt;}</code>                                                                                 | Muestra mensaje de éxito si existe                                            |

| Orden | Línea                                                                              | ¿Qué pasa en tiempo de ejecución? |
|-------|------------------------------------------------------------------------------------|-----------------------------------|
| 83-85 | <code>&lt;main className="dashboard-main"&gt;{renderContent()}&lt;/main&gt;</code> | Renderiza el contenido dinámico   |
| 86-87 | <code>&lt;/div&gt;&lt;/&gt;</code>                                                 | Cierra contenedores               |
| 89    | <code>export default DashboardView</code>                                          | Exporta el componente             |

## ⚡ Escenario real: Usuario abre el dashboard por primera vez

javascript

*// Usuario acaba de iniciar sesión y es redirigido a /dashboard*

1. DashboardView se ejecuta
2. `useAuth()` → `currentUser = { name: "Oliver", email: "oliver@stone.com", role: "admin" }`
3. `activeTab = 'dashboard'` (estado inicial)
4. Se renderiza DashboardHeader con:
  - `user: { name: "Oliver", ... }`
  - `activeTab: "dashboard"`
  - `onTabChange`: función que puede cambiar activeTab
5. Se renderiza el contenido principal:
  - `renderContent()` ve que `activeTab = 'dashboard'`
  - Muestra "Bienvenido, Oliver!"
  - Muestra DashboardStats con stats simulados
  - Muestra las tarjetas de información
6. El usuario ve el panel principal

## ⚡ Escenario real: Usuario hace clic en pestaña "Usuarios"

javascript

*// El usuario hace clic en el botón "Usuarios" en DashboardHeader*

1. DashboardHeader ejecuta `onTabChange('users')`
2. `setActiveTab('users')` cambia el estado
3. React **RE-RENDERIZA** DashboardView
4. `renderContent()` se ejecuta nuevamente
5. `switch(activeTab)` ahora es `'users'`
6. `return <UsersView />` → **SE EJECUTA UsersView**
7. El usuario ve la tabla de usuarios

## 🔄 Flujo de renderizado según activeTab

text

```
activeTab = 'dashboard'
↓
renderContent() → case 'dashboard'
↓
Muestra:
- Mensaje de bienvenida
- DashboardStats (4 tarjetas)
- 2 tarjetas de información (Sistema y Autenticación)

activeTab = 'users'
↓
renderContent() → case 'users'
↓
Muestra:
- <UsersView /> (tabla de usuarios con CRUD)
```

## 📺 Componentes hijos que se ejecutan

| Componente      | ¿Cuándo se ejecuta?                                   | Props que recibe                                                                              |
|-----------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| DashboardHeader | Siempre (está fuera de <code>renderContent()</code> ) | <code>user</code> , <code>onLogout</code> , <code>activeTab</code> , <code>onTabChange</code> |
| DashboardStats  | Solo cuando <code>activeTab = 'dashboard'</code>      | <code>stats</code> (datos simulados)                                                          |
| UsersView       | Solo cuando <code>activeTab = 'users'</code>          | Ninguna (usa sus propios hooks)                                                               |
| AlertMessage    | Solo cuando <code>logoutMessage</code> tiene valor    | <code>type</code> , <code>message</code> , <code>onClose</code>                               |

### 💡 Por qué se usa `renderContent()` en lugar de condicionales inline

Opción A (actual - más limpia):

```
javascript

const renderContent = () => {
  switch (activeTab) {
    case 'dashboard': return <DashboardStats />
    case 'users': return <UsersView />
  }
}

return (
  <main>
    {renderContent()}
  </main>
)
```

Opción B (menos limpia):

```
javascript

return (
  <main>
    {activeTab === 'dashboard' && <DashboardStats />}
    {activeTab === 'users' && <UsersView />}
  </main>
)
```

Ambas funcionan, pero `renderContent()` es más organizada cuando hay muchas opciones.

### 💡 Analogía para entenderlo

Imagina que `DashboardView.jsx` es una **tableta con pestañas**:

- `activeTab` → Es la pestaña que está resaltada actualmente (como "Inicio" o "Configuración" en tu celular).
- `setActiveTab` → Es tu dedo tocando otra pestaña para cambiar de vista.
- `DashboardHeader` → Es la barra superior con las pestañas y tu nombre de perfil.
- `renderContent()` → Es el área principal que cambia según qué pestaña hayas seleccionado.
- `handleLogout` → Es el botón "Cerrar sesión" que te saca de la app.

## ⚠️ Puntos clave para recordar

| Concepto                                        | Explicación                                                                                          |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>activeTab</code> es estado local          | No se guarda en <code>localStorage</code> ni en el hook. Si recargas la página, vuelve a 'dashboard' |
| <code>DashboardStats</code> usa datos simulados | No son datos reales de la API. Son valores estáticos para demostración                               |
| <code>localStorage.getItem('auth_token')</code> | Se lee directamente para mostrar si el token existe (solo visual, no lógico)                         |
| El mensaje "Rol: Administrador" está hardcodedo | Debería usar <code>currentUser?.role</code> en lugar de texto fijo                                   |
| <code>setTimeout</code> en logout               | Da tiempo a que el usuario vea el mensaje "Sesión cerrada exitosamente" antes de redirigir           |

## 🐛 Posibles errores en este punto

| Error                                                                            | Causa                                                                              | Solución                                                                         |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>currentUser</code> es null y se intenta leer <code>currentUser.name</code> | El usuario no está bien cargado                                                    | Usar <code>currentUser?.name</code> (optional chaining) ya está implementado ✅   |
| <code>DashboardHeader</code> no recibe <code>onTabChange</code>                  | La prop no se pasa correctamente                                                   | Verificar que <code>DashboardHeader</code> acepte y use <code>onTabChange</code> |
| Al hacer logout, el mensaje no aparece                                           | <code>logoutMessage</code> se setea pero <code>AlertMessage</code> no se renderiza | Verificar la condición <code>{logoutMessage &amp;&amp; ...}</code>               |
| La pestaña "Usuarios" no carga                                                   | <code>UsersView</code> tiene error o no está importado                             | Revisar la consola para errores en <code>UsersView</code>                        |
| Las estadísticas no cambian                                                      | Son datos simulados estáticos                                                      | Para datos reales, habría que conectar a <code>useDashboard</code> hook          |

## 🔧 Mejora potencial: Conectar estadísticas a API real

Actualmente `DashboardStats` recibe datos hardcodedos:

```
javascript

<DashboardStats stats={{
  activeUsers: 150,    // ← Hardcodeado
  todayVisits: 45,     // ← Hardcodeado
  activeSessions: 23,  // ← Hardcodeado
  successRate: 95     // ← Hardcodeado
}} />
```

Para conectarlo a una API real, se usaría el hook `useDashboard` :

```
javascript

import useDashboard from '../hooks/useDashboard'

const { stats, loading, loadStats } = useDashboard()
```

```
useEffect(() => {
  loadStats() // Carga datos reales desde la API
}, [])

// Luego: <DashboardStats stats={stats} />
```

¿Entendiste por qué `DashboardView.jsx` actúa como un contenedor con pestañas y renderiza contenido diferente según la pestaña activa?

Escribe "siguiente" y pasamos al ARCHIVO 16: `DashboardHeader.jsx` (la barra superior con las pestañas). 📄

siguiente

## 📄 ARCHIVO 16: `src/views/dashboard/DashboardHeader.jsx` (DECIMOSEXTO en ejecutarse)

```
javascript

// src/views/dashboard/DashboardHeader.jsx
const DashboardHeader = ({ user, onLogout, activeTab, onTabChange }) => {
  // Estilos inline para asegurar que se vea bien

  const headerStyles = {
    background: 'white',
    boxShadow: '0 2px 10px rgba(0,0,0,0.1)',
    padding: '0 30px',
    display: 'flex',
    justifyContent: 'space-between',
    alignItems: 'center',
    position: 'sticky',
    top: 0,
    zIndex: 100,
    flexWrap: 'wrap',
    gap: '15px'
  }

  const headerLeftStyles = {
    display: 'flex',
    alignItems: 'center',
    gap: '15px',
    padding: '15px 0'
  }

  const headerRightStyles = {
    display: 'flex',
    alignItems: 'center',
    gap: '20px'
  }

  const badgeStyles = {
    background: 'linear-gradient(135deg, #667eea 0%, #764ba2 100%)',
    color: 'white',
    padding: '4px 12px',
    borderRadius: '20px',
    fontSize: '12px',
    fontWeight: 600
  }

  const tabStyles = {
    display: 'flex',
    gap: '5px',
    background: '#f0f0f0',
    padding: '4px',
    borderRadius: '12px'
  }
}
```

```

const tabBtnStyles = (isActive) => ({
  padding: '8px 20px',
  border: 'none',
  background: isActive ? 'linear-gradient(135deg, #667eea 0%, #764ba2 100%)' :
'transparent',
  cursor: 'pointer',
  fontSize: '14px',
  fontWeight: 500,
  borderRadius: '8px',
  transition: 'all 0.3s ease',
  color: isActive ? 'white' : '#666'
})

const logoutBtnStyles = {
  background: '#ef4444',
  color: 'white',
  border: 'none',
  padding: '8px 16px',
  borderRadius: '8px',
  cursor: 'pointer',
  fontSize: '14px',
  fontWeight: 500
}

return (
  <header style={headerStyles}>
    <div style={headerLeftStyles}>
      <h1 style={{ fontSize: '20px', color: '#333', margin: 0 }}>Sistema de Info
rmación Web</h1>
      <span style={badgeStyles}>Admin Panel</span>
    </div>

    <div style={tabStyles}>
      <button
        style={tabBtnStyles(activeTab === 'dashboard')}
        onClick={() => onTabChange('dashboard')}
      >
        Dashboard
      </button>
      <button
        style={tabBtnStyles(activeTab === 'users')}
        onClick={() => onTabChange('users')}
      >
        Usuarios
      </button>
    </div>

    <div style={headerRightStyles}>
      <div style={{ textAlign: 'right' }}>
        <div style={{ fontWeight: 600, color: '#333', fontSize: '14px' }}>
          {user?.name || user?.email?.split('@')[0] || 'Usuario'}
        </div>
        <div style={{ fontSize: '12px', color: '#666' }}>{user?.email}</div>
      </div>
      <button onClick={onLogout} style={logoutBtnStyles}>
        Cerrar Sesión
      </button>
    </div>
  </header>
)
}

export default DashboardHeader

```

## Responsabilidad en la ejecución

`DashboardHeader.jsx` es un **componente presentacional** que se ejecuta **cuando** `DashboardView.jsx` **lo renderiza** (siempre, independientemente de la pestaña activa). Su responsabilidad es:



- 1. **Mostrar la barra superior** del dashboard con el título de la aplicación.
- 2. **Mostrar un badge "Admin Panel"** para indicar el tipo de acceso.
- 3. **Renderizar las pestañas de navegación** (Dashboard y Usuarios) con estilos visuales que indican cuál está activa.
- 4. **Mostrar la información del usuario** (nombre y email) que viene del hook `useAuth`.
- 5. **Proveer el botón de "Cerrar Sesión"** que ejecuta la función `onLogout` recibida por props.
- 6. **Comunicar cambios de pestaña** al componente padre (`DashboardView`) mediante `onTabChange`.

 **Orden de ejecución línea por línea**

| Orden | Línea                                                                                                                   | ¿Qué pasa en tiempo de ejecución?                                       |
|-------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| 1     | <code>const DashboardHeader = ({ user, onLogout, activeTab, onTabChange }) =&gt; {</code>                               | Define el componente y recibe 4 props                                   |
| 3-15  | <code>const headerStyles = { ... }</code>                                                                               | Define objeto con estilos CSS para el header principal                  |
| 17-23 | <code>const headerLeftStyles = { ... }</code>                                                                           | Estilos para la sección izquierda (título y badge)                      |
| 25-30 | <code>const headerRightStyles = { ... }</code>                                                                          | Estilos para la sección derecha (usuario y botón)                       |
| 32-38 | <code>const badgeStyles = { ... }</code>                                                                                | Estilos para el badge "Admin Panel"                                     |
| 40-47 | <code>const tabStyles = { ... }</code>                                                                                  | Estilos para el contenedor de las pestañas                              |
| 49-59 | <code>const tabBtnStyles = (isActive) =&gt; ({ ... })</code>                                                            | <b>FUNCIÓN</b> que retorna estilos diferentes si la pestaña está activa |
| 61-70 | <code>const logoutBtnStyles = { ... }</code>                                                                            | Estilos para el botón de cerrar sesión (rojo)                           |
| 72    | <code>return (</code>                                                                                                   | Comienza a renderizar el header                                         |
| 73    | <code>&lt;header style={headerStyles}&gt;</code>                                                                        | Elemento <code>&lt;header&gt;</code> con estilos inline                 |
| 74-79 | <code>&lt;div style={headerLeftStyles}&gt;</code>                                                                       | Sección izquierda con título y badge                                    |
| 75    | <code>&lt;h1 style={{ ... }}&gt;Sistema de Información Web&lt;/h1&gt;</code>                                            | Título de la aplicación                                                 |
| 76-78 | <code>&lt;span style={badgeStyles}&gt;Admin Panel&lt;/span&gt;</code>                                                   | Badge morado con gradiente                                              |
| 81-91 | <code>&lt;div style={tabStyles}&gt;</code>                                                                              | Contenedor de las pestañas                                              |
| 82-86 | <code>&lt;button style={tabBtnStyles(activeTab === 'dashboard')} onClick={() =&gt; onTabChange('dashboard')}&gt;</code> | Botón "Dashboard" - cambia estilo si está activo                        |

| Orden  | Línea                                                                                                                      | ¿Qué pasa en tiempo de ejecución?                  |
|--------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| 87-90  | <pre>&lt;button style={   {tabBtnStyles(activeTab ===     'users')}} onClick={() =&gt;     onTabChange('users')}&gt;</pre> | Botón "Usuarios" - cambia estilo si está activo    |
| 93-101 | <pre>&lt;div style={   {headerRightStyles}}&gt;</pre>                                                                      | Sección derecha con información del usuario        |
| 94-98  | <pre>&lt;div style={{ textAlign:   'right' }}&gt;</pre>                                                                    | Contenedor del nombre y email                      |
| 95-96  | <pre>&lt;div style={{ ... }}&gt;   {user?.name      user?.email?.split('@')[0]      'Usuario'}&lt;/div&gt;</pre>           | Muestra el nombre o la parte del email antes del @ |
| 97     | <pre>&lt;div style={{ ... }}&gt;   {user?.email}&lt;/div&gt;</pre>                                                         | Muestra el email completo                          |
| 99-101 | <pre>&lt;button onClick={onLogout}   style={logoutBtnStyles}&gt;Cerrar   Sesión&lt;/button&gt;</pre>                       | Botón rojo que ejecuta <code>onLogout</code>       |
| 102    | <pre>&lt;/header&gt;</pre>                                                                                                 | Cierra el header                                   |
| 104    | <pre>export default DashboardHeader</pre>                                                                                  | Exporta el componente                              |

## ⚡ Escenario real: Usuario admin ve el dashboard

```
javascript

// DashboardView pasa estas props:
<DashboardHeader
  user={{ name: "Oliver", email: "oliver@stone.com", role: "admin" }}
  onLogout={handleLogout}
  activeTab="dashboard"
  onTabChange={setActiveTab}
/>

// Dentro de DashboardHeader:

1. user?.name = "Oliver" → se muestra "Oliver"
2. user?.email = "oliver@stone.com" → se muestra debajo
3. activeTab = "dashboard" →
   - tabBtnStyles(true) para botón Dashboard → background gradiente, color blanco
   - tabBtnStyles(false) para botón Usuarios → background transparente, color #666
4. onLogout es la función handleLogout del padre
5. Se renderiza todo con estilos inline
```

## 🎨 Estilos dinámicos con `tabBtnStyles`

```
javascript

const tabBtnStyles = (isActive) => ({
  padding: '8px 20px',
  border: 'none',
  background: isActive
    ? 'linear-gradient(135deg, #667eea 0%, #764ba2 100%)' // Activo: morado con g
    : 'transparent', // Inactivo: transpare
  cursor: 'pointer',
  fontSize: '14px',
  fontWeight: 500,
```

```
borderRadius: '8px',
transition: 'all 0.3s ease',
color: isActive ? 'white' : '#666' // Activo: texto blanco | Inactivo: texto gr
is
})
```

#### Resultado visual:

- Pestaña **Dashboard** activa → fondo morado, texto blanco
- Pestaña **Usuarios** inactiva → fondo transparente, texto gris

Cuando el usuario hace clic en "Usuarios":

- `onTabChange('users')` se ejecuta
- `activeTab` en el padre cambia a 'users'
- El padre re-renderiza `DashboardHeader` con la nueva prop
- `tabBtnStyles(activeTab === 'users')` ahora es `true` para Usuarios y `false` para Dashboard
- Los estilos se intercambian visualmente

### 💡 ¿Por qué estilos inline en lugar de CSS modules?

El componente usa **estilos inline** (objetos de JavaScript) en lugar de archivos CSS separados. Esto tiene ventajas y desventajas:

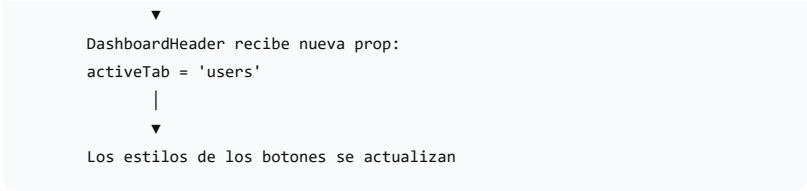
| Ventajas                                                      | Desventajas                                                                        |
|---------------------------------------------------------------|------------------------------------------------------------------------------------|
| Los estilos son <b>dinámicos</b> (pueden cambiar según props) | Menos reutilizables                                                                |
| No hay conflictos de nombres de clases                        | Más verboso                                                                        |
| Fáciles de depurar (están en el mismo archivo)                | No soporta pseudo-clases ( <code>:hover</code> , <code>:active</code> ) fácilmente |
| No requieren imports adicionales                              | Difíciles de mantener si crecen                                                    |

En este caso, como los estilos son relativamente simples y dependen de `isActive` , tiene sentido usar estilos inline.

### 🔄 Flujo de comunicación padre-hijo

```
text

DashboardView (padre)
|
| state: activeTab = 'dashboard'
|
| → renderiza DashboardHeader con:
|   - activeTab = 'dashboard'
|   - onTabChange = setActiveTab
|
| → Usuario hace clic en "Usuarios"
|
|   ▼
|   DashboardHeader ejecuta:
|   onClick={() => onTabChange('users')}
|
|   ▼
|   setActiveTab('users') en el padre
|
|   ▼
|   React re-renderiza DashboardView
|
```



## 💡 Analogía para entenderlo

Imagina que `DashboardHeader.jsx` es la **barra de herramientas de una aplicación de edición de fotos**:

- **Título y badge** → Es como el nombre de la app ("Photoshop") y la versión ("Pro 2024").
- **Pestañas** → Son como las herramientas principales ("Ajustes", "Filtros", "Recortar").
- **Información del usuario** → Es como el avatar y nombre de la cuenta de Adobe.
- **Botón "Cerrar Sesión"** → Es como el botón "Salir" que te desconecta.

Cuando haces clic en una herramienta diferente, la interfaz completa cambia (el área principal muestra las opciones de esa herramienta). La barra de herramientas se mantiene igual, solo resalta la herramienta seleccionada.

## ⚠️ Puntos clave para recordar

| Concepto                                                         | Explicación                                                                                                        |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Es un componente "presentacional"                                | No tiene estado propio, solo recibe props y renderiza. Toda la lógica está en el padre                             |
| Estilos inline                                                   | Se usan por simplicidad y para estilos dinámicos (cambian según <code>activeTab</code> )                           |
| <pre>user?.name    user?.email?.split('@')[0]    'Usuario'</pre> | Tiene 3 niveles de fallback: 1) nombre, 2) parte del email antes del @, 3) "Usuario" por defecto                   |
| <code>onTabChange</code> es una función                          | El padre puede pasar CUALQUIER función. Aquí pasa <code>setActiveTab</code> directamente                           |
| El badge dice "Admin Panel"                                      | Está hardcoded. Si un seller entra, también vería "Admin Panel" (podría mejorarse para que sea dinámico según rol) |

## 🐛 Posibles errores en este punto

| Error                                                              | Causa                                                  | Solución                                                               |
|--------------------------------------------------------------------|--------------------------------------------------------|------------------------------------------------------------------------|
| <pre>onTabChange is not a function</pre>                           | El padre no pasó la prop correctamente                 | Verificar que DashboardView pase <pre>onTabChange={setActiveTab}</pre> |
| <pre>user</pre> es undefined y rompe al leer <pre>user.email</pre> | El padre pasó <pre>user={null}</pre> o no pasó la prop | Usar optional chaining <pre>user?.email</pre> (ya está implementado)   |

| Error                                           | Causa                                                                           | Solución                                                                           |
|-------------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Los estilos no se aplican                       | Los objetos de estilo tienen errores de sintaxis                                | Verificar comas, llaves, y nombres de propiedades CSS (usar camelCase)             |
| Al hacer clic en pestaña no cambia el contenido | <code>onTabChange</code> no se ejecuta o <code>activeTab</code> no se actualiza | Verificar que <code>onTabChange</code> esté correctamente conectado                |
| El botón logout no funciona                     | <code>onLogout</code> no está definido o tiene error                            | Verificar que <code>DashboardView</code> pase <code>onLogout={handleLogout}</code> |

## Mejora potencial: Hacer el badge dinámico según rol

```
javascript

// En Lugar de hardcodear "Admin Panel":
<span style={badgeStyles}>Admin Panel</span>

// Podría ser dinámico:
const getBadgeText = (role) => {
  switch(role) {
    case 'admin': return 'Admin Panel'
    case 'seller': return 'Seller Panel'
    default: return 'User Panel'
  }
}

<span style={badgeStyles}>{getBadgeText(user?.role)}</span>
```

¿Entendiste por qué `DashboardHeader.jsx` es un componente presentacional que recibe props y renderiza la barra superior con pestañas dinámicas?

Escribe "siguiente" y pasamos al ARCHIVO 17: `DashboardStats.jsx` (las tarjetas de estadísticas). 🇵🇪

siguiente

## ARCHIVO 17: `src/views/dashboard/DashboardStats.jsx` (DECIMOSEPTIMO en ejecutarse)

```
javascript

// src/views/dashboard/DashboardStats.jsx
const DashboardStats = ({ stats }) => {
  const statItems = [
    { label: 'Usuarios Activos', value: stats?.activeUsers || 0, icon: '👤' },
    { label: 'Visitas Hoy', value: stats?.todayVisits || 0, icon: '📊' },
    { label: 'Sesiones Activas', value: stats?.activeSessions || 0, icon: '🟢' },
    { label: 'Tasa de Éxito', value: `${stats?.successRate || 0}%`, icon: '📈' }
  ]

  const gridStyles = {
    display: 'grid',
    gridTemplateColumns: 'repeat(auto-fit, minmax(250px, 1fr))',
    gap: '20px',
    marginBottom: '40px'
  }

  const cardStyles = {
    background: 'white',
    borderRadius: '15px',
  }
```

```
padding: '20px',
display: 'flex',
alignItems: 'center',
gap: '15px',
boxShadow: '0 2px 10px rgba(0, 0, 0, 0.05)',
transition: 'transform 0.3s ease'
}

return (
  <div style={gridStyles}>
    {statItems.map((item, index) => (
      <div key={index} style={cardStyles}>
        <div style={{ fontSize: '40px' }}>{item.icon}</div>
        <div style={{ flex: 1 }}>
          <h3 style={{ fontSize: '28px', fontWeight: 700, color: '#333', margin:
'0 0 5px 0' }}>
            {item.value}
          </h3>
          <p style={{ color: '#666', fontSize: '14px', margin: 0 }}>{item.label}
        </p>
      </div>
    ))}
  </div>
)
}

export default DashboardStats
```

## Responsabilidad en la ejecución

`DashboardStats.jsx` es un **componente presentacional** que se ejecuta **cuando** `DashboardView.jsx` **lo renderiza dentro de la pestaña 'dashboard'**. Su responsabilidad es:

1. **Mostrar tarjetas de estadísticas** en formato grid responsive.
2. **Recibir datos de estadísticas** a través de la prop `stats`.
3. **Renderizar 4 tarjetas** con iconos, valores y etiquetas.
4. **Proveer valores por defecto** (0) si los datos no llegan.
5. **Aplicar estilos visuales** (tarjetas blancas, sombras, hover con transformación).

## Orden de ejecución línea por línea

| Orden | Línea                                                                                  | ¿Qué pasa en tiempo de ejecución?                        |
|-------|----------------------------------------------------------------------------------------|----------------------------------------------------------|
| 1     | <pre>const DashboardStats = ({ stats }) =&gt; {</pre>                                  | Define el componente y recibe la prop <code>stats</code> |
| 2-7   | <pre>const statItems = [ ... ]</pre>                                                   | Define un array con 4 objetos (label, value, icon)       |
| 3     | <pre>{ label: 'Usuarios Activos', value: stats?.activeUsers    0, icon: '👤' }</pre>    | Toma <code>stats.activeUsers</code> o usa 0 por defecto  |
| 4     | <pre>{ label: 'Visitas Hoy', value: stats?.todayVisits    0, icon: '📊' }</pre>         | Toma <code>stats.todayVisits</code> o usa 0              |
| 5     | <pre>{ label: 'Sesiones Activas', value: stats?.activeSessions    0, icon: '🟢' }</pre> | Toma <code>stats.activeSessions</code> o usa 0           |

| Orden | Línea                                                                      | ¿Qué pasa en tiempo de ejecución?                                                   |
|-------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 6     | <pre>{ label: 'Tasa de Éxito', value: \ \${stats?.successRate</pre>        | <pre>0}%`, icon: '📈' }`</pre> <p>Toma <code>stats.suc</code><br/>agrega símbolo</p> |
| 9-14  | <pre>const gridStyles = { ... }</pre>                                      | Estilos para el contenedor grid (responsive)                                        |
| 10    | <pre>display: 'grid'</pre>                                                 | Activa CSS Grid                                                                     |
| 11    | <pre>gridTemplateColumns: 'repeat(auto-fit, minmax(250px, 1fr))'</pre>     | Columnas responsivas: mínimo 250px, máximo automático                               |
| 12    | <pre>gap: '20px'</pre>                                                     | Espacio entre tarjetas                                                              |
| 13    | <pre>marginBottom: '40px'</pre>                                            | Margen inferior para separar del contenido siguiente                                |
| 16-25 | <pre>const cardStyles = { ... }</pre>                                      | Estilos para cada tarjeta individual                                                |
| 17    | <pre>background: 'white'</pre>                                             | Fondo blanco                                                                        |
| 18    | <pre>borderRadius: '15px'</pre>                                            | Bordes redondeados                                                                  |
| 19    | <pre>padding: '20px'</pre>                                                 | Espaciado interno                                                                   |
| 20    | <pre>display: 'flex'</pre>                                                 | Flexbox para alinear icono y texto horizontalmente                                  |
| 21    | <pre>alignItems: 'center'</pre>                                            | Centrado vertical                                                                   |
| 22    | <pre>gap: '15px'</pre>                                                     | Espacio entre icono y texto                                                         |
| 23    | <pre>boxShadow: '0 2px 10px rgba(0, 0, 0, 0.05)'</pre>                     | Sombra suave                                                                        |
| 24    | <pre>transition: 'transform 0.3s ease'</pre>                               | Animación suave para efecto hover                                                   |
| 27    | <pre>return (</pre>                                                        | Comienza a renderizar                                                               |
| 28    | <pre>&lt;div style={gridStyles}&gt;</pre>                                  | Contenedor grid                                                                     |
| 29    | <pre>{statItems.map((item, index) =&gt; (</pre>                            | Itera sobre las 4 tarjetas                                                          |
| 30    | <pre>&lt;div key={index} style= {cardStyles}&gt;</pre>                     | Cada tarjeta (usando índice como key)                                               |
| 31    | <pre>&lt;div style={{ fontSize: '40px' }}&gt;{item.icon}&lt;/div&gt;</pre> | Icono grande de 40px (emoji o texto)                                                |
| 32    | <pre>&lt;div style={{ flex: 1 }}&gt;</pre>                                 | Contenedor del texto (toma el espacio restante)                                     |
| 33-35 | <pre>&lt;h3 style={{ ... }}&gt;{item.value} &lt;/h3&gt;</pre>              | Título con el valor numérico                                                        |
| 36    | <pre>&lt;p style={{ ... }}&gt;{item.label} &lt;/p&gt;</pre>                | Etiqueta descriptiva debajo del valor                                               |
| 37    | <pre>&lt;/div&gt;&lt;/div&gt;</pre>                                        | Cierra contenedores de texto y tarjeta                                              |

| Orden | Línea                                      | ¿Qué pasa en tiempo de ejecución? |
|-------|--------------------------------------------|-----------------------------------|
| 38    | <code>}}}</code>                           | Cierra map                        |
| 39    | <code>&lt;/div&gt;</code>                  | Cierra contenedor grid            |
| 40    | <code>)</code>                             | Cierra return                     |
| 42    | <code>export default DashboardStats</code> | Exporta el componente             |

## ⚡ Escenario real: DashboardView pasa stats simulados

```

javascript

// Desde DashboardView.jsx:
<DashboardStats stats={{
  activeUsers: 150,
  todayVisits: 45,
  activeSessions: 23,
  successRate: 95
}} />

// Dentro de DashboardStats:

const statItems = [
  { label: 'Usuarios Activos', value: 150, icon: '👤' },
  { label: 'Visitas Hoy', value: 45, icon: '📊' },
  { label: 'Sesiones Activas', value: 23, icon: '🟢' },
  { label: 'Tasa de Éxito', value: '95%', icon: '📈' }
]

// Se renderizan 4 tarjetas con estos valores

```

## 📱 Grid responsive explicado

```

css

gridTemplateColumns: 'repeat(auto-fit, minmax(250px, 1fr))'

```

Esta es una de las propiedades CSS Grid más potentes:

| Tamaño de pantalla      | Ancho disponible | ¿Cuántas tarjetas?               |
|-------------------------|------------------|----------------------------------|
| Móvil (320px)           | 320px            | 1 tarjeta (250px + se expande)   |
| Tablet (600px)          | 600px            | 2 tarjetas (250px c/u + espacio) |
| Desktop (900px)         | 900px            | 3 tarjetas (250px c/u)           |
| Desktop grande (1200px) | 1200px           | 4 tarjetas (250px c/u + espacio) |

### Cómo funciona:

- `minmax(250px, 1fr)` → Cada tarjeta tiene mínimo 250px y máximo 1 fracción del espacio disponible
- `auto-fit` → Ajusta automáticamente el número de columnas según el espacio
- Si hay espacio para 4 tarjetas de 250px, muestra 4. Si no, muestra menos y las expande

## 🎨 Efecto hover (aunque no se ve en el código)



El componente tiene `transition: 'transform 0.3s ease'` en `cardStyles`, pero no tiene el `hover` definido. Para que funcione, necesitarías agregar CSS o una clase:

```
javascript

// Si quisieras agregar efecto hover, podrías hacer:
const cardStyles = {
  // ... estilos existentes
  transition: 'transform 0.3s ease',
  ':hover': { // ← Esto NO funciona con estilos inline!
    transform: 'translateY(-5px)'
  }
}
```

**Nota:** Los estilos inline NO soportan pseudo-clases (`:hover`, `:active`, etc.). Para que el `hover` funcione, se necesitaría CSS tradicional o una librería como `styled-components`. Actualmente la propiedad `transition` está presente pero no tiene efecto porque no hay un cambio de estado.

### 💡 ¿Por qué usar `stats?.activeUsers || 0`?

```
javascript

value: stats?.activeUsers || 0
```

Esto maneja dos escenarios:

1. `stats` es `null` o `undefined` → `stats?.activeUsers` es `undefined` → `|| 0` devuelve 0
2. `stats.activeUsers` es 0 → `0 || 0` devuelve 0
3. `stats.activeUsers` tiene valor → devuelve ese valor

### 🔄 Renderizado de las 4 tarjetas

```
javascript

statItems.map((item, index) => (
  <div key={index} style={cardStyles}>
    <div style={{ fontSize: '40px' }}>{item.icon}</div>
    <div style={{ flex: 1 }}>
      <h3 style={{ ... }}>{item.value}</h3>
      <p style={{ ... }}>{item.label}</p>
    </div>
  </div>
))
```

**Uso de `key={index}`:** Es aceptable porque el array es estático (nunca cambia de orden o tamaño). Si fuera dinámico, se recomendaría usar un ID único.

### 💡 Analogía para entenderlo

Imagina que `DashboardStats.jsx` es un **tablero de control en un estadio de fútbol**:

- `gridStyles` → Es como la pantalla gigante dividida en 4 cuadrantes.
- **Cada tarjeta** → Es como cada cuadrante mostrando una estadística del partido.
- **Icono** (👤, 📊, 🟢, 📉) → Es como el ícono que representa cada métrica (goles, posesión, tarjetas, faltas).
- **Valor** (150, 45, 23, 95%) → Es como el número grande que ves (goles, porcentaje de posesión).
- **Etiqueta** ("Usuarios Activos", etc.) → Es como el texto pequeño que explica qué significa ese número.

## ⚠️ Puntos clave para recordar

| Concepto                                  | Explicación                                                                                 |
|-------------------------------------------|---------------------------------------------------------------------------------------------|
| Componente puramente presentacional       | No tiene estado, no hace peticiones, solo recibe <code>stats</code> y renderiza             |
| Valores por defecto con <code>   0</code> | Previene que se muestre "undefined" o "NaN" si los datos no llegan                          |
| Grid responsivo                           | Las tarjetas se reorganizan automáticamente según el tamaño de pantalla                     |
| Iconos emoji                              | Usa emojis como iconos (👤, 📊, 🟢, 🔄). Son universales y no requieren librerías               |
| <code>flex: 1</code> en el texto          | Hace que el contenedor del texto ocupe todo el espacio restante después del icono           |
| El hover no funciona con estilos inline   | La propiedad <code>transition</code> está presente pero no hay <code>:hover</code> definido |

## 🐛 Posibles errores en este punto

| Error                                                    | Causa                                                                     | Solución                                                                       |
|----------------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>stats is undefined</code> y se muestra "undefined" | El padre no pasó la prop <code>stats</code>                               | DashboardView debe pasar <code>stats={{...}}</code> o usar valores por defecto |
| Las tarjetas no son responsivas                          | <code>gridStyles</code> tiene errores                                     | Verificar que <code>gridTemplateColumns</code> esté correctamente escrito      |
| Los números no se actualizan                             | Los stats son estáticos en DashboardView                                  | Para datos reales, usar hook <code>useDashboard</code>                         |
| El emoji no se ve en algunos navegadores                 | Navegador muy antiguo                                                     | Los emojis son estándar en navegadores modernos                                |
| El layout se rompe en móvil                              | <code>minmax(250px, 1fr)</code> puede ser muy ancho para móviles pequeños | Cambiar a <code>minmax(200px, 1fr)</code> o usar media queries                 |

## 📄 Mejora potencial: Conectar a API real

Actualmente los stats son simulados. Para conectarlos a una API:

```
javascript

// En DashboardView.jsx (mejorado):
import useDashboard from '../hooks/useDashboard'

const DashboardView = () => {
  const { stats, loading, loadStats } = useDashboard()

  useEffect(() => {
    loadStats() // Carga datos reales desde la API
  }, [])

  if (loading) return <LoadingSpinner />
}
```

```
    return <DashboardStats stats={stats} />
  }
}
```

¿Entendiste por qué `DashboardStats.jsx` es un componente simple que solo muestra tarjetas con datos que recibe por props?

Escribe "siguiente" y pasamos al ARCHIVO 18: `UsersView.jsx` (la tabla de gestión de usuarios). 🧑

siguiente

## 📄 ARCHIVO 18: `src/views/dashboard/UsersView.jsx` (DECIMOCTAVO en ejecutarse)

```
javascript

// src/views/dashboard/UsersView.jsx
import { useState } from 'react'
import useUsers from '../../../hooks/useUsers'
import useAuth from '../../../hooks/useAuth' // ← NUEVO: para saber el rol del usuario actual
import AlertMessage from '../../../common/AlertMessage'
import UserForm from './UserForm'
import UserDetails from './UserDetails'
import '../../../styles/Users.css'

const UsersView = () => {
  const { users, loading, error, deleteUser, loadUsers } = useUsers()
  const { currentUser } = useAuth() // ← NUEVO: usuario logueado
  const [showForm, setShowForm] = useState(false)
  const [showDetails, setShowDetails] = useState(false)
  const [selectedUser, setSelectedUser] = useState(null)
  const [editMode, setEditMode] = useState(false)
  const [message, setMessage] = useState(null)
  const [searchTerm, setSearchTerm] = useState('')

  // Filtrar usuarios por búsqueda (ahora incluye nombre y rol)
  const filteredUsers = users.filter(user =>
    user.email?.toLowerCase().includes(searchTerm.toLowerCase()) ||
    user.id?.toString().includes(searchTerm) ||
    user.name?.toLowerCase().includes(searchTerm.toLowerCase()) ||
    user.role?.toLowerCase().includes(searchTerm.toLowerCase())
  )

  // NUEVA FUNCIÓN: Verificar si el usuario actual puede EDITAR
  const canEdit = (user) => {
    if (!currentUser) return false
    // Admin puede editar todo
    if (currentUser.role === 'admin') return true
    // Seller puede editar pero no eliminar
    if (currentUser.role === 'seller') return true
    return false
  }

  // NUEVA FUNCIÓN: Verificar si el usuario actual puede ELIMINAR
  const canDelete = (user) => {
    if (!currentUser) return false
    // Solo admin puede eliminar
    if (currentUser.role !== 'admin') return false
    // No puede eliminarse a sí mismo
    if (currentUser.id === user.id) return false
    return true
  }

  // NUEVA FUNCIÓN: Obtener clase CSS para el rol (color según rol)
  const getRoleBadgeClass = (role) => {
    switch (role) {

```

```

    case 'admin':
      return 'role-badge role-admin'
    case 'seller':
      return 'role-badge role-seller'
    case 'customer':
      return 'role-badge role-customer'
    default:
      return 'role-badge role-user'
  }
}

// NUEVA FUNCIÓN: Texto legible del rol
const getRoleText = (role) => {
  switch (role) {
    case 'admin': return '👑 Administrador'
    case 'seller': return '🛒 Vendedor'
    case 'customer': return '👤 Cliente'
    default: return '👤 Usuario'
  }
}

// Manejar eliminación (ahora con verificación de permisos)
const handleDelete = async (user) => {
  if (!canDelete(user)) {
    setMessage({ type: 'error', text: 'No tiene permisos para eliminar este usuario' })
    setTimeout(() => setMessage(null), 3000)
    return
  }

  if (window.confirm(`¿Estás seguro de eliminar al usuario "${user.email}"?`)) {
    try {
      await deleteUser(user.id)
      setMessage({ type: 'success', text: 'Usuario eliminado exitosamente' })
      setTimeout(() => setMessage(null), 3000)
    } catch (err) {
      setMessage({ type: 'error', text: err.error || 'Error al eliminar usuario' })
    }
  }
}

// Manejar edición (ahora con verificación de permisos)
const handleEdit = (user) => {
  if (!canEdit(user)) {
    setMessage({ type: 'error', text: 'No tiene permisos para editar este usuario' })
    setTimeout(() => setMessage(null), 3000)
    return
  }
  setSelectedUser(user)
  setEditMode(true)
  setShowForm(true)
  setShowDetails(false)
}

// Manejar ver detalles (siempre permitido para admin/seller)
const handleViewDetails = (user) => {
  setSelectedUser(user)
  setShowDetails(true)
  setShowForm(false)
  setEditMode(false)
}

// Manejar éxito del formulario
const handleFormSuccess = () => {
  setShowForm(false)
  setEditMode(false)
  setSelectedUser(null)
  loadUsers()
  setMessage({ type: 'success', text: 'Operación completada exitosamente' })
}

```

```

        setTimeout(() => setMessage(null), 3000)
    }

    const handleCloseForm = () => {
        setShowForm(false)
        setEditMode(false)
        setSelectedUser(null)
    }

    const handleCloseDetails = () => {
        setShowDetails(false)
        setSelectedUser(null)
    }

    // NUEVA FUNCIÓN: Verificar si puede crear usuarios
    const canCreateUser = () => {
        return currentUser && (currentUser.role === 'admin' || currentUser.role === 'seller')
    }

    if (loading && users.length === 0) {
        return <div className="loading-container">Cargando usuarios...</div>
    }

    // NUEVO: Si hay error de permisos (403), mostrar pantalla de acceso denegado
    if (error && (error.includes('No tiene permisos') || error.includes('403'))) {
        return (
            <div className="error-container">
                <div className="error-icon">🔒</div>
                <h3>Acceso Denegado</h3>
                <p>No tiene permisos para ver la lista de usuarios.</p>
                <p className="error-hint">Contacte al administrador si necesita acceso.</p>
            </div>
        )
    }

    return (
        <div className="users-container">
            <div className="users-header">
                <h2>Gestión de Usuarios</h2>
                { /* NUEVO: Solo mostrar botón si tiene permisos */ }
                { canCreateUser() && (
                    <button
                        className="btn-primary"
                        onClick={() => {
                            setSelectedUser(null)
                            setEditMode(false)
                            setShowForm(true)
                            setShowDetails(false)
                        }}
                    >
                        + Nuevo Usuario
                    </button>
                ) }
            </div>

            {message && (
                <AlertMessage
                    type={message.type}
                    message={message.text}
                    onClose={() => setMessage(null)}
                />
            )}

            <div className="users-search">
                <input
                    type="text"
                    placeholder="Buscar por email, nombre, ID o rol..."
                    value={searchTerm}
                    onChange={(e) => setSearchTerm(e.target.value)}
                />
            </div>
        </div>
    )

```

```

        className="search-input"
      />
    </div>

    {error && !error.includes('No tiene permisos')} && (
      <div className="error-message">
        Error: {error}
        <button onClick={loadUsers}>Reintentar</button>
      </div>
    )}

    <div className="users-table-container">
      <table className="users-table">
        <thead>
          <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Email</th>
            <th>Rol</th>
            <th>Teléfono</th>
            <th>Acciones</th>
          </tr>
        </thead>
        <tbody>
          {filteredUsers.length === 0 ? (
            <tr>
              <td colspan="6" className="no-data">
                No hay usuarios registrados
              </td>
            </tr>
          ) : (
            filteredUsers.map((user) => (
              <tr key={user.id} className={currentUser?.id === user.id ? 'current-user-row' : ''}>
                <td>{user.id}</td>
                <td>{user.name} {user.lastname} <span>||</span></td>
                <td>{user.email}</td>
                <td>
                  <span className={getRoleBadgeClass(user.role)}>
                    {getRoleText(user.role)}
                  </span>
                </td>
                <td>{user.phone} <span>||</span> '-'</td>
                <td className="actions">
                  <button
                    className="btn-view"
                    onClick={() => handleViewDetails(user)}
                    title="Ver detalles"
                  >

                  </button>
                  <button
                    className="btn-edit"
                    onClick={() => handleEdit(user)}
                    title="Editar"
                    disabled={!canEdit(user)}
                  >

                  </button>
                  <button
                    className="btn-delete"
                    onClick={() => handleDelete(user)}
                    title="Eliminar"
                    disabled={!canDelete(user)}
                  >

                  </button>
                </td>
              </tr>
            ))
          )}
        </tbody>
      </table>
    </div>
  )}

```

```

        </tbody>
      </table>
    </div>

    {showForm && (
      <UserForm
        user={selectedUser}
        isEdit={editMode}
        onSuccess={handleFormSuccess}
        onClose={handleCloseForm}
      />
    )}

    {showDetails && selectedUser && (
      <UserDetails
        user={selectedUser}
        onClose={handleCloseDetails}
        onEdit={() => {
          handleCloseDetails()
          handleEdit(selectedUser)
        }}
      />
    )}
  </div>
)
}

export default UsersView

```

## Responsabilidad en la ejecución

`UsersView.jsx` es un **componente de vista complejo** que se ejecuta **cuando el usuario hace clic en la pestaña "Usuarios" dentro del dashboard**. Su responsabilidad es:

1. **Mostrar una tabla con todos los usuarios** obtenidos de la API.
2. **Permitir búsqueda en tiempo real** por email, nombre, ID o rol.
3. **Gestionar permisos según el rol** del usuario actual (admin puede todo, seller solo editar, customer/user solo ver).
4. **Proveer acciones** para ver detalles, editar y eliminar usuarios.
5. **Abrir modales** ( `UserForm` y `UserDetails` ) para operaciones CRUD.
6. **Manejar estados de carga y error** con mensajes apropiados.
7. **Mostrar un mensaje de acceso denegado** si el usuario no tiene permisos.

## Orden de ejecución línea por línea (partes clave)

| Orden | Línea                                                                                            | ¿Qué pasa en tiempo de ejecución?                                                                                                                 |
|-------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-9   | <code>import ...</code>                                                                          | Importa hooks, componentes, estilos                                                                                                               |
| 11    | <code>const UsersView = () =&gt; {</code>                                                        | Define el componente                                                                                                                              |
| 12    | <code>const { users, loading, error, deleteUser, loadUsers } = useUsers()</code>                 |  <b>EJECUTA useUsers()</b> (carga usuarios automáticamente)    |
| 13    | <code>const { currentUser } = useAuth()</code>                                                   |  <b>EJECUTA useAuth()</b> para saber el rol del usuario actual |
| 14-20 | Estados locales ( <code>showForm</code> , <code>showDetails</code> , <code>selectedUser</code> , | Controlan qué modales se muestran                                                                                                                 |

| Orden   | Línea                                                                                                             | ¿Qué pasa en tiempo de ejecución?               |
|---------|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
|         | etc.)                                                                                                             |                                                 |
| 21      | <code>const [searchTerm, setSearchTerm] = useState('')</code>                                                     | Estado para búsqueda en vivo                    |
| 24-31   | <code>const filteredUsers = users.filter(...)</code>                                                              | Filtra usuarios según texto de búsqueda         |
| 34-49   | <code>canEdit()</code> , <code>canDelete()</code> , <code>getRoleBadgeClass()</code> , <code>getRoleText()</code> | Funciones auxiliares para permisos y estilos    |
| 52-68   | <code>handleDelete()</code>                                                                                       | Maneja eliminación con confirmación y permisos  |
| 71-81   | <code>handleEdit()</code>                                                                                         | Maneja edición con verificación de permisos     |
| 84-89   | <code>handleViewDetails()</code>                                                                                  | Muestra modal de detalles                       |
| 92-99   | <code>handleFormSuccess()</code>                                                                                  | Recarga la tabla y muestra mensaje de éxito     |
| 112-114 | <code>if (loading &amp;&amp; users.length === 0)</code>                                                           | Muestra spinner mientras carga                  |
| 117-127 | <code>if (error &amp;&amp; error.includes('No tiene permisos'))</code>                                            | Muestra pantalla de acceso denegado             |
| 130-221 | <code>return (...)</code>                                                                                         | Renderiza la tabla, búsqueda, botones y modales |

## ⚡ Escenario real: Usuario admin ve la tabla de usuarios

javascript

```
// currentUser = { id: 1, role: 'admin', name: 'Admin' }
```

- `useUsers()` carga todos los usuarios desde la API
- `users = [ { id: 1, name: 'Admin', role: 'admin', ... }, { id: 2, name: 'Oliver', role: 'admin', ... }, { id: 3, name: 'Vendedor', role: 'seller', ... }, ... ]`
- `canEdit(cualquierUsuario) → true` (admin puede editar todo)
- `canDelete(usuario) → true` si `usuario.id !== currentUser.id`
- `canCreateUser() → true` (admin puede crear)
- Se renderiza tabla con todos los usuarios
- La fila del usuario **actual** (`id=1`) tiene clase `'current-user-row'` (fondo amarillo)
- Los botones de editar y eliminar están **habilitados** (excepto eliminar su propio usuario)

## ⚡ Escenario real: Usuario seller ve la tabla de usuarios

javascript

```
// currentUser = { id: 5, role: 'seller', name: 'Seller' }
```

- `useUsers()` carga todos los usuarios
- `canEdit(cualquierUsuario) → true` (seller puede editar)



3. `canDelete(cualquierUsuario)` → `false` (seller NO puede eliminar)
4. `canCreateUser()` → `true` (seller puede crear)
5. Botones de editar: habilitados 
6. Botones de eliminar: `deshabilitados` (opacos, no clickeables) 
7. Botón "Nuevo Usuario": visible 

## ⚡ Escenario real: Usuario normal (customer/user) intenta acceder

javascript

```
// currentUser = { id: 10, role: 'customer', name: 'Cliente' }
```

1. `useUsers()` intenta cargar usuarios desde la API
2. El backend responde con `403 Forbidden` (no tiene permisos)
3. `error = "No tiene permisos para ver la lista de usuarios"`
4. La condición ``error.includes('No tiene permisos')`` es `true`
5. Se muestra pantalla de acceso denegado con candado 
6. No se ve la tabla de usuarios

## Filtrado en tiempo real ( `filteredUsers` )

javascript

```
const filteredUsers = users.filter(user =>
  user.email?.toLowerCase().includes(searchTerm.toLowerCase()) ||
  user.id?.toString().includes(searchTerm) ||
  user.name?.toLowerCase().includes(searchTerm.toLowerCase()) ||
  user.role?.toLowerCase().includes(searchTerm.toLowerCase())
)
```

Ejemplo: Usuario escribe "oli" en el buscador

| Campo del usuario                            | ¿Coincide?                                                                                                 |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>user.email = "oliver@stone.com"</code> |  "oliver" incluye "oli" |
| <code>user.name = "Oliver"</code>            | "Oliver" incluye "oli" (case-insensitive)                                                                  |
| <code>user.role = "admin"</code>             |  no incluye "oli"       |
| <code>user.id = 5</code>                     |  "5" no incluye "oli"   |

Resultado: El usuario Oliver aparece en la tabla filtrada.

## Flujo de eliminación de usuario

text

```
Usuario hace clic en 
↓
canDelete(user) verifica permisos
↓ (true para admin, no es su propio usuario)
window.confirm() pregunta "¿Estás seguro?"
↓ (usuario confirma)
deleteUser(id) se ejecuta
↓
UserController.deleteUser() → UserModel.deleteUser() → DELETE a /api/users/delete/:id
↓
Backend elimina usuario de la base de datos
↓
loadUsers() recarga la lista actualizada
↓
```

```
setMessage({ type: 'success', text: 'Usuario eliminado exitosamente' })
↓
El usuario ve el mensaje de éxito por 3 segundos
```

## 💡 Analogía para entenderlo

Imagina que `UsersView.jsx` es una **aplicación de gestión de empleados en una oficina**:

- `useUsers()` → Es el archivo maestro de empleados que trae la lista completa.
- `currentUser` → Es tu propia tarjeta de identificación (muestra si eres admin, vendedor o empleado normal).
- `searchTerm` → Es el cuadro de búsqueda donde escribes el nombre de un empleado.
- **Tabla de usuarios** → Es el listado con todos los empleados, sus cargos y teléfonos.
- **Botón "🔍 Ver detalles"** → Es como abrir el expediente de un empleado (todos pueden ver).
- **Botón "✏️ Editar"** → Solo admin y vendedores pueden modificar datos.
- **Botón "🗑️ Eliminar"** → Solo admin puede despedir empleados (y no puede despedirse a sí mismo).
- **Pantalla de "Acceso Denegado 🚫"** → Es lo que ve un empleado normal si intenta entrar a la oficina de RRHH.

## ⚠️ Puntos clave para recordar

| Concepto                                             | Explicación                                                                                                |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>useUsers()</code> <b>carga automáticamente</b> | Dentro del hook hay un <code>useEffect</code> que llama a <code>loadUsers()</code> al montar el componente |
| <b>Filtrado en el frontend</b>                       | La búsqueda NO hace una nueva petición a la API, solo filtra los usuarios ya cargados                      |
| <code>filteredUsers.map()</code>                     | Cada <code>tr</code> tiene <code>key={user.id}</code> (buena práctica, ID único en lugar de índice)        |
| <code>currentUser-row</code>                         | La fila del usuario actual tiene fondo amarillo para identificarse fácilmente                              |
| <code>disabled={!canEdit(user)}</code>               | El botón de editar se deshabilita visualmente (no solo lógicamente)                                        |
| <b>Mensajes temporales</b>                           | Los mensajes de éxito/error desaparecen después de 3 segundos con <code>setTimeout</code>                  |
| <b>Modales</b>                                       | <code>UserForm</code> y <code>UserDetails</code> se renderizan condicionalmente al final del componente    |

## 🐛 Posibles errores en este punto

| Error                                          | Causa                                          | Solución                                                                           |
|------------------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------|
| <code>users</code> está vacío aunque hay datos | <code>useUsers()</code> no cargó correctamente | Verificar que el endpoint <code>/users</code> devuelva datos y el token sea válido |
| El buscador no filtra por nombre               | <code>user.name</code> es null o undefined     | Usar <code>user.name?.toLowerCase()</code>                                         |

| Error                                        | Causa                                                                           | Solución                                                                                                |
|----------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
|                                              |                                                                                 | (optional chaining) ya está implementado                                                                |
| <code>canEdit()</code> siempre retorna false | <code>currentUser</code> es null o no tiene <code>role</code>                   | Verificar que <code>useAuth()</code> esté trayendo el usuario correctamente                             |
| El botón eliminar no hace nada               | <code>deleteUser</code> no está definido o <code>canDelete</code> retorna false | Verificar que <code>useUsers()</code> exporte <code>deleteUser</code> y que el rol sea admin            |
| Los modales no se abren                      | <code>showForm</code> o <code>showDetails</code> no se actualizan               | Verificar que <code>handleEdit</code> y <code>handleViewDetails</code> seteen los estados correctamente |

¿Entendiste por qué `UsersView.jsx` es el componente más complejo del dashboard, manejando permisos, búsqueda, modales y operaciones CRUD? Escribe "siguiente" y pasamos al ARCHIVO 19: `UserForm.jsx` (modal para crear/editar usuarios). 🚀

siguiente

## 📄 ARCHIVO 19: `src/views/dashboard/UserForm.jsx` (DECIMONOVENO en ejecutarse)

```
javascript

// src/views/dashboard/UserForm.jsx
import { useState, useEffect } from 'react'
import useUsers from '../../../hooks/useUsers'
import useAuth from '../../../hooks/useAuth'

const UserForm = ({ user, isEdit, onSuccess, onClose }) => {
  const { createUser, updateUser, patchUser } = useUsers()
  const { currentUser } = useAuth()
  const [formData, setFormData] = useState({
    name: '',
    lastname: '',
    email: '',
    password: '',
    confirmPassword: '',
    phone: '',
    role: 'user'
  })
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')
  const [patchMode, setPatchMode] = useState(false)
  const [patchField, setPatchField] = useState('')
  const [patchValue, setPatchValue] = useState('')

  // Cargar datos del usuario si es edición
  useEffect(() => {
    if (user && isEdit) {
      setFormData({
        name: user.name || '',
        lastname: user.lastname || '',
        email: user.email || '',
        password: '',
        confirmPassword: '',
        phone: user.phone || '',
        role: user.role || 'user'
      })
    }
  })
}
```

```

}, [user, isEdit])

const handleChange = (e) => {
  const { name, value } = e.target
  setFormData(prev => ({ ...prev, [name]: value }))
  if (error) setError('')
}

// Verificar si el usuario actual puede asignar roles (solo admin)
const canAssignRole = () => {
  return currentUser?.role === 'admin'
}

const handleSubmit = async (e) => {
  e.preventDefault()

  // Validaciones
  if (!formData.email) {
    setError('El email es requerido')
    return
  }

  if (!isEdit && formData.password !== formData.confirmPassword) {
    setError('Las contraseñas no coinciden')
    return
  }

  if (!isEdit && formData.password.length < 5) {
    setError('La contraseña debe tener al menos 6 caracteres')
    return
  }

  if (!isEdit && !formData.name) {
    setError('El nombre es requerido')
    return
  }

  setLoading(true)
  setError('')

  try {
    if (isEdit) {
      if (patchMode && patchField) {
        // Modo PATCH - Actualizar solo un campo
        const patchData = {}
        patchData[patchField] = patchValue
        await patchUser(user.id, patchData)
      } else {
        // Modo PUT - Actualizar todos los datos
        const updateData = {
          name: formData.name,
          lastname: formData.lastname,
          email: formData.email,
          phone: formData.phone
        }
        // Solo incluir rol si el usuario actual es admin
        if (canAssignRole()) {
          updateData.role = formData.role
        }
        if (formData.password) {
          updateData.password = formData.password
        }
        await updateUser(user.id, updateData)
      }
    } else {
      // Modo POST - Crear nuevo usuario
      const newUserData = {
        name: formData.name,
        lastname: formData.lastname,
        email: formData.email,
        password: formData.password,

```

```

        phone: formData.phone,
        role: canAssignRole() ? formData.role : 'user'
      }
      await createUser(newUserData)
    }
    onSuccess()
  } catch (err) {
    setError(err.error || 'Error al guardar usuario')
  } finally {
    setLoading(false)
  }
}

const handlePatchSubmit = async (e) => {
  e.preventDefault()
  if (!patchField || !patchValue) {
    setError('Seleccione un campo y un valor')
    return
  }

  setLoading(true)
  try {
    const patchData = {}
    patchData[patchField] = patchValue
    await patchUser(user.id, patchData)
    onSuccess()
  } catch (err) {
    setError(err.error || 'Error al actualizar campo')
  } finally {
    setLoading(false)
  }
}

return (
  <div className="modal-overlay" onClick={onClose}>
    <div className="modal-content" onClick={(e) => e.stopPropagation()}>
      <div className="modal-header">
        <h3>{isEdit ? 'Editar Usuario' : 'Nuevo Usuario'}</h3>
        <button className="modal-close" onClick={onClose}>x</button>
      </div>

      {error && <div className="form-error">{error}</div>}

      {/* Mostrar toggle de PATCH solo para admin en modo edición */}
      {isEdit && canAssignRole() && (
        <div className="patch-toggle">
          <label>
            <input
              type="radio"
              checked={!patchMode}
              onChange={() => setPatchMode(false)}
            />
            Actualizar todos los datos (PUT)
          </label>
          <label>
            <input
              type="radio"
              checked={patchMode}
              onChange={() => setPatchMode(true)}
            />
            Actualizar campo específico (PATCH)
          </label>
        </div>
      )}

      {patchMode && isEdit ? (
        // Formulario PATCH - Actualizar un solo campo
        <form onSubmit={handlePatchSubmit} className="user-form">
          <div className="form-group">
            <label>Campo a actualizar</label>
            <select

```

```

        value={patchField}
        onChange={(e) => setPatchField(e.target.value)}
        className="form-control"
        required
      >
      <option value="">Seleccionar campo</option>
      <option value="name">Nombre</option>
      <option value="lastname">Apellido</option>
      <option value="email">Email</option>
      <option value="phone">Teléfono</option>
      <option value="password">Contraseña</option>
      {canAssignRole() && <option value="role">Rol</option>}
    </select>
  </div>

  <div className="form-group">
    <label>Nuevo valor</label>
    {patchField === 'role' ? (
      <select
        value={patchValue}
        onChange={(e) => setPatchValue(e.target.value)}
        className="form-control"
        required
      >
        <option value="">Seleccionar rol</option>
        <option value="admin">Administrador</option>
        <option value="seller">Vendedor</option>
        <option value="customer">Cliente</option>
        <option value="user">Usuario</option>
      </select>
    ) : (
      <input
        type={patchField === 'email' ? 'email' : (patchField === 'password' ? 'password' : 'text')}
        value={patchValue}
        onChange={(e) => setPatchValue(e.target.value)}
        placeholder={`Nuevo valor para ${patchField}`}
        className="form-control"
        required
      />
    )}
  </div>

  <div className="form-actions">
    <button type="submit" className="btn-primary" disabled={loading}>
      {loading ? 'Actualizando...' : 'Actualizar Campo'}
    </button>
    <button type="button" className="btn-secondary" onClick={onClose}>
      Cancelar
    </button>
  </div>
</form>

) : (
  // Formulario completo (PUT para edición, POST para creación)
  <form onSubmit={handleSubmit} className="user-form">
    <div className="form-row">
      <div className="form-group">
        <label>Nombre {!isEdit && '*'}</label>
        <input
          type="text"
          name="name"
          value={formData.name}
          onChange={handleChange}
          placeholder="Nombre"
          className="form-control"
          required={!isEdit}
          disabled={loading}
        />
      </div>

      <div className="form-group">

```

```

<label>Apellido {!isEdit && '*'}</label>
<input
  type="text"
  name="lastname"
  value={formData.lastname}
  onChange={handleChange}
  placeholder="Apellido"
  className="form-control"
  required={!isEdit}
  disabled={loading}
/>
</div>
</div>

```

```

<div className="form-group">
  <label>Email *</label>
  <input
    type="email"
    name="email"
    value={formData.email}
    onChange={handleChange}
    placeholder="usuario@ejemplo.com"
    className="form-control"
    required
    disabled={loading}
  />
</div>

```

```

<div className="form-group">
  <label>Teléfono</label>
  <input
    type="tel"
    name="phone"
    value={formData.phone}
    onChange={handleChange}
    placeholder="Teléfono (opcional)"
    className="form-control"
    disabled={loading}
  />
</div>

```

```

{!isEdit && (
  <>
    <div className="form-row">
      <div className="form-group">
        <label>Contraseña *</label>
        <input
          type="password"
          name="password"
          value={formData.password}
          onChange={handleChange}
          placeholder="Mínimo 6 caracteres"
          className="form-control"
          required
          disabled={loading}
        />
      </div>

      <div className="form-group">
        <label>Confirmar Contraseña *</label>
        <input
          type="password"
          name="confirmPassword"
          value={formData.confirmPassword}
          onChange={handleChange}
          placeholder="Repita la contraseña"
          className="form-control"
          required
          disabled={loading}
        />
      </div>
    </div>
  </>
)
}

```

```

        </div>
      </>
    )}

    {isEdit && !patchMode && (
      <div className="form-group">
        <label>Nueva Contraseña (opcional)</label>
        <input
          type="password"
          name="password"
          value={formData.password}
          onChange={handleChange}
          placeholder="Dejar en blanco para mantener actual"
          className="form-control"
          disabled={loading}
        />
      </div>
    )}

    {/* Selector de rol - Solo visible para admin */}
    {canAssignRole() && (
      <div className="form-group">
        <label>Rol</label>
        <select
          name="role"
          value={formData.role}
          onChange={handleChange}
          className="form-control"
          disabled={loading}
        >
          <option value="user">Usuario</option>
          <option value="customer">Cliente</option>
          <option value="seller">Vendedor</option>
          <option value="admin">Administrador</option>
        </select>
        <small className="form-hint">Define los permisos del usuario</small>
      </div>
    )}

    <div className="form-actions">
      <button type="submit" className="btn-primary" disabled={loading}>
        {loading ? 'Guardando...' : (isEdit ? 'Actualizar' : 'Crear')}
      </button>
      <button type="button" className="btn-secondary" onClick={onClose}>
        Cancelar
      </button>
    </div>
  </form>
)
}

export default UserForm

```

## Responsabilidad en la ejecución

`UserForm.jsx` es un **componente modal complejo** que se ejecuta **cuando** `UsersView.jsx` **muestra el formulario de creación o edición**. Su responsabilidad es:

1. Proveer un formulario para crear nuevos usuarios (POST a `/users/create`).
2. Proveer un formulario para editar usuarios existentes (PUT a `/users/:id`).
3. Soportar dos modos de edición:
  - PUT: Actualizar todos los datos del usuario.



- **PATCH:** Actualizar un solo campo específico.
4. **Validar datos del formulario** (campos requeridos, contraseñas coincidentes, longitud mínima).
  5. **Gestionar permisos** (solo admin puede cambiar roles).
  6. **Cargar datos del usuario** cuando se abre en modo edición.
  7. **Comunicar éxito al padre** ( `onSuccess` ) para recargar la tabla y cerrar el modal.

### Orden de ejecución línea por línea (partes clave)

| Orden   | Línea                                                                                                 | ¿Qué pasa en tiempo de ejecución?                                                                                               |
|---------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 1-4     | <code>import ...</code>                                                                               | Importa hooks y dependencias                                                                                                    |
| 6       | <code>const UserForm = ({ user, isEdit, onSuccess, onClose }) =&gt; {</code>                          | Recibe 4 props del padre                                                                                                        |
| 7       | <code>const { createUser, updateUser, patchUser } = useUsers()</code>                                 |  <b>Obtiene funciones CRUD</b>                 |
| 8       | <code>const { currentUser } = useAuth()</code>                                                        |  <b>Obtiene usuario actual (para permisos)</b> |
| 9-18    | <code>const [formData, setFormData] = useState(...)</code>                                            | Estado con todos los campos del formulario                                                                                      |
| 19      | <code>const [loading, setLoading] = useState(false)</code>                                            | Estado de carga (deshabilita botones)                                                                                           |
| 20      | <code>const [error, setError] = useState('')</code>                                                   | Estado para mensajes de error                                                                                                   |
| 21      | <code>const [patchMode, setPatchMode] = useState(false)</code>                                        | ¿Modo PATCH activado? (solo para admin)                                                                                         |
| 22      | <code>const [patchField, setPatchField] = useState('')</code>                                         | Campo seleccionado para PATCH                                                                                                   |
| 23      | <code>const [patchValue, setPatchValue] = useState('')</code>                                         | Nuevo valor para el campo PATCH                                                                                                 |
| 25-37   | <code>useEffect(() =&gt; { if (user &amp;&amp; isEdit) { setFormData(...) } }, [user, isEdit])</code> | Carga datos del usuario al abrir en edición                                                                                     |
| 39-43   | <code>handleChange()</code>                                                                           | Actualiza estado cuando el usuario escribe                                                                                      |
| 46-48   | <code>canAssignRole()</code>                                                                          | Retorna true si el usuario actual es admin                                                                                      |
| 50-133  | <code>handleSubmit()</code>                                                                           | <b>Envío del formulario completo (PUT/POST)</b>                                                                                 |
| 135-152 | <code>handlePatchSubmit()</code>                                                                      | <b>Envío del formulario PATCH</b>                                                                                               |
| 154-308 | <code>return (...)</code>                                                                             | Renderiza el modal con el formulario correspondiente                                                                            |

### Escenario real: Crear un nuevo usuario (POST)

javascript

```
// Usuario admin hace clic en "Nuevo Usuario"  
// Props: isEdit = false, user = null
```

1. useEffect NO carga datos (user es null)
2. formData tiene valores vacíos, role = 'user'
3. Se renderiza formulario completo con todos los campos obligatorios (\*)

4. Usuario completa:
  - Nombre: "Nuevo"
  - Apellido: "Usuario"
  - Email: "nuevo@ejemplo.com"
  - Contraseña: "123456"
  - Confirmar: "123456"
  - Teléfono: "987654321"
  - Rol: "seller"

5. Hace clic en "Crear"

6. handleSubmit() valida:
  - email ☒ existe
  - password === confirmPassword ☒
  - password.length >= 6 ☒ (6 >= 6)
  - name ☒ existe

7. setLoading(true)

8. createUser({  
 name: "Nuevo",  
 lastname: "Usuario",  
 email: "nuevo@ejemplo.com",  
 password: "123456",  
 phone: "987654321",  
 role: "seller"  
})

9. POST a /api/users/create

10. Backend crea usuario en BD

11. onSuccess() se ejecuta → cierra modal y recarga tabla

## ⚡ Escenario real: Editar usuario con PUT

javascript

```
// Usuario admin edita a un usuario existente  
// Props: isEdit = true, user = { id: 5, name: "Oliver", role: "admin", ... }
```

1. useEffect carga los datos del usuario en formData
2. patchMode = false (por defecto, modo PUT)
3. Se muestra selector de rol (admin puede cambiar roles)

4. Admin cambia el rol de "admin" a "seller"
5. Admin hace clic en "Actualizar"

6. handleSubmit() detecta isEdit = true y patchMode = false
7. Construye updateData con name, lastname, email, phone, role (nuevo rol "seller")
8. No incluye password porque el campo está vacío

9. updateUser(5, updateData) → PUT a /api/users/5

10. Backend actualiza usuario en BD

11. onSuccess() → cierra modal y recarga tabla

## ⚡ Escenario real: Editar usuario con PATCH (solo un campo)

javascript

```
// Usuario admin quiere cambiar SOLO el teléfono de un usuario
// Props: isEdit = true, user = { id: 5, name: "Oliver", phone: "123456789", ... }
```

1. useEffect carga datos del usuario
2. Admin activa "Actualizar campo específico (PATCH)" → `setPatchMode(true)`
3. Selecciona campo "Teléfono" del dropdown
4. Escribe nuevo valor: "999888777"
5. Hace clic en "Actualizar Campo"
6. `handlePatchSubmit()` se ejecuta
7. `patchData = { phone: "999888777" }`
8. `patchUser(5, patchData)` → PATCH a `/api/users/5`
9. Backend actualiza SOLO el campo phone
10. `onSuccess()` → cierra modal y recarga tabla

## 📊 Comparación: PUT vs PATCH

| Característica   | PUT                         | PATCH                                       |
|------------------|-----------------------------|---------------------------------------------|
| Actualiza        | Todos los campos            | Un solo campo específico                    |
| Envía            | Objeto completo             | Objeto con solo el campo cambiado           |
| Endpoint         | PUT <code>/users/:id</code> | PATCH (usa PUT en este proyecto)            |
| ¿Cuándo usarlo?  | Cambiar múltiples campos    | Cambiar un dato puntual (ej: solo teléfono) |
| En este proyecto | Por defecto                 | Opcional (solo admin)                       |

## 🔄 Flujo de validaciones

text

```
Crear usuario (isEdit = false):
├─ ¿Email existe? → NO → error "El email es requerido"
├─ ¿Password = ConfirmPassword? → NO → error "Las contraseñas no coinciden"
├─ ¿Password.length >= 6? → NO → error "Mínimo 6 caracteres"
├─ ¿Nombre existe? → NO → error "El nombre es requerido"
└─ Todo OK → createUser()

Editar usuario (isEdit = true):
├─ ¿Email existe? → NO → error "El email es requerido"
└─ Todo OK → updateUser() o patchUser()
```

## 💡 Analogía para entenderlo

Imagina que `UserForm.jsx` es un **formulario de registro en un hospital**:

- **Modo "Nuevo Usuario"** → Es cuando un paciente nuevo llega por primera vez: llenas todos los campos (nombre, apellido, email, teléfono, contraseña para su cuenta digital).
- **Modo "Editar Usuario (PUT)"** → Es cuando un paciente actualiza TODOS sus datos: cambia dirección, teléfono, contacto de emergencia, todo en una sola hoja.

- **Modo "Editar Usuario (PATCH)"** → Es cuando SOLO quieres cambiar el número de teléfono: no rellenas todo el formulario, solo una hoja pequeña con el campo modificado.
- **Selector de rol (solo admin)** → Es como el doctor que puede cambiar el tipo de paciente (particular, obra social, VIP), mientras que el paciente no puede cambiar eso por sí mismo.

 **Puntos clave para recordar**

| Concepto                                                               | Explicación                                                                                  |
|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>useEffect</code> con dependencias<br><code>[user, isEdit]</code> | Cuando el modal se abre con un usuario diferente, el formulario se actualiza automáticamente |
| <code>canAssignRole()</code>                                           | Solo admin puede ver y cambiar el selector de rol (seguridad en UI)                          |
| <code>patchMode</code> solo para admin                                 | El toggle PUT/PATCH solo aparece si <code>isEdit &amp;&amp; canAssignRole()</code>           |
| <code>handlePatchSubmit</code> separado                                | Es un flujo completamente diferente al <code>handleSubmit</code> normal                      |
| Contraseña opcional en edición                                         | En edición, si el campo password está vacío, NO se envía al backend (mantiene la actual)     |
| <code>onClick={onClose}</code> en el overlay                           | Hacer clic fuera del modal lo cierra (por el <code>modal-overlay</code> )                    |
| <code>e.stopPropagation()</code>                                       | Evita que hacer clic DENTRO del modal cierre el modal (el evento no sube al overlay)         |

 **Posibles errores en este punto**

| Error                                         | Causa                                                                  | Solución                                                                            |
|-----------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>createUser</code> is not a function     | <code>useUsers()</code> no exporta <code>createUser</code>             | Verificar que <code>useUsers.js</code> retorne <code>createUser</code>              |
| Al editar, los datos no cargan                | <code>useEffect</code> no se ejecuta porque <code>user</code> es null  | Asegurar que <code>UsersView</code> pase <code>selectedUser</code> correctamente    |
| El rol no se actualiza aunque admin lo cambie | <code>canAssignRole()</code> retorna false                             | Verificar que <code>currentUser.role === 'admin'</code>                             |
| PATCH envía PUT a la API                      | El backend no tiene endpoint PATCH separado                            | En <code>UserModel.patchUser</code> se usa <code>httpService.put</code> (es normal) |
| El modal no se cierra después de éxito        | <code>onSuccess</code> no se ejecuta o no llama a <code>onClose</code> | Verificar que <code>onSuccess()</code> cierre el modal en <code>UsersView</code>    |

¿Entendiste por qué `UserForm.jsx` es un componente modal que soporta tres operaciones diferentes (crear, editar completo, editar campo específico) y

gestiona permisos según el rol del usuario actual?

Escribes "siguiente" y pasamos al ARCHIVO 20: `UserDetails.jsx` (modal para ver detalles del usuario). 🕒

siguiente

## 📄 ARCHIVO 20: `src/views/dashboard/UserDetails.jsx` (VIGÉSIMO en ejecutarse)

```
javascript

// src/views/dashboard/UserDetails.jsx
import useAuth from '../../../hooks/useAuth'

const UserDetails = ({ user, onClose, onEdit }) => {
  const { currentUser } = useAuth()

  // Verificar si el usuario actual puede editar (admin o seller)
  const canEdit = () => {
    return currentUser && (currentUser.role === 'admin' || currentUser.role === 'seller')
  }

  // Obtener texto legible del rol
  const getRoleText = (role) => {
    switch (role) {
      case 'admin': return '👑 Administrador'
      case 'seller': return '🛒 Vendedor'
      case 'customer': return '👤 Cliente'
      default: return '👤 Usuario'
    }
  }

  // Obtener clase CSS para el badge del rol
  const getRoleBadgeClass = (role) => {
    switch (role) {
      case 'admin': return 'role-badge role-admin'
      case 'seller': return 'role-badge role-seller'
      case 'customer': return 'role-badge role-customer'
      default: return 'role-badge role-user'
    }
  }

  // Formatear fecha si existe
  const formatDate = (dateString) => {
    if (!dateString) return 'N/A'
    try {
      return new Date(dateString).toLocaleString()
    } catch {
      return 'N/A'
    }
  }

  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal-content" onClick={(e) => e.stopPropagation()}>
        <div className="modal-header">
          <h3>Detalles del Usuario</h3>
          <button className="modal-close" onClick={onClose}>x</button>
        </div>

        <div className="user-details">
          <div className="detail-row">
            <span className="detail-label">ID:</span>
            <span className="detail-value">{user.id}</span>
          </div>

          <div className="detail-row">
            <span className="detail-label">Nombre completo:</span>

```

```

        <span className="detail-value">
          {user.name} {user.lastname || ''}
        </span>
      </div>

      <div className="detail-row">
        <span className="detail-label">Email:</span>
        <span className="detail-value">{user.email}</span>
      </div>

      <div className="detail-row">
        <span className="detail-label">Rol:</span>
        <span className="detail-value">
          <span className={getRoleBadgeClass(user.role)}>
            {getRoleText(user.role)}
          </span>
        </span>
      </div>

      <div className="detail-row">
        <span className="detail-label">Teléfono:</span>
        <span className="detail-value">{user.phone || 'No registrado'}</span>
      </div>

      <div className="detail-row">
        <span className="detail-label">Fecha registro:</span>
        <span className="detail-value">{formatDate(user.created_at)}</span>
      </div>

      <div className="detail-row">
        <span className="detail-label">Última actualización:</span>
        <span className="detail-value">{formatDate(user.updated_at)}</span>
      </div>
    </div>

    <div className="modal-footer">
      {canEdit() && (
        <button className="btn-primary" onClick={onEdit}>
          Editar Usuario
        </button>
      )}
      <button className="btn-secondary" onClick={onClose}>
        Cerrar
      </button>
    </div>
  </div>
</div>

export default UserDetails

```

## 🔗 Responsabilidad en la ejecución

`UserDetails.jsx` es un **componente modal** que se ejecuta cuando el usuario hace clic en el botón "🔍 Ver detalles" en la tabla de `UsersView.jsx`. Su responsabilidad es:

1. **Mostrar toda la información de un usuario** en un formato legible y organizado.
2. **Presentar el rol del usuario con un badge coloreado** (diferente color para admin, seller, customer, user).
3. **Formatear fechas** (created\_at, updated\_at) para mostrar en formato local legible.
4. **Proveer un botón "Editar Usuario"** que aparece solo si el usuario actual tiene permisos (admin o seller).
5. **Permitir cerrar el modal** con botón de cierre o haciendo clic fuera del modal.

6. Comunicar la acción de editar al padre ( `onEdit` ) para que abra el `UserForm` con los datos del usuario.

 Orden de ejecución línea por línea

| Orden | Línea                                                                                                     | ¿Qué pasa en tiempo de ejecución?                                                                                                    |
|-------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 1     | <code>import useAuth from<br/>'../../hooks/useAuth'</code>                                                | Importa hook para obtener usuario actual                                                                                             |
| 3     | <code>const UserDetails = ({ user,<br/>onClose, onEdit }) =&gt; {</code>                                  | Define componente y recibe 3 props                                                                                                   |
| 4     | <code>const { currentUser } =<br/>useAuth()</code>                                                        |  <b>Obtiene usuario logueado</b><br>(para permisos) |
| 7-10  | <code>const canEdit = () =&gt; { ... }</code>                                                             | Verifica si el usuario actual puede editar (admin o seller)                                                                          |
| 13-20 | <code>const getRoleText = (role) =&gt;<br/>{ ... }</code>                                                 | Retorna texto legible con emoji según rol                                                                                            |
| 23-30 | <code>const getRoleBadgeClass =<br/>(role) =&gt; { ... }</code>                                           | Retorna clase CSS para el color del badge                                                                                            |
| 33-39 | <code>const formatDate =<br/>(dateString) =&gt; { ... }</code>                                            | Convierte fecha ISO a formato local legible                                                                                          |
| 42-99 | <code>return (...)</code>                                                                                 | Renderiza el modal con todos los detalles                                                                                            |
| 43    | <code>&lt;div className="modal-overlay"<br/>onClick={onClose}&gt;</code>                                  | Fondo oscuro que cierra al hacer clic                                                                                                |
| 44    | <code>&lt;div className="modal-content"<br/>onClick={(e) =&gt;<br/>e.stopPropagation()}&gt;</code>        | Contenedor blanco que NO cierra al hacer clic dentro                                                                                 |
| 45-48 | <code>&lt;div className="modal-header"&gt;</code>                                                         | Cabecera con título y botón de cierre                                                                                                |
| 50-79 | <code>&lt;div className="user-details"&gt;</code>                                                         | Cuerpo con todos los campos del usuario                                                                                              |
| 51-77 | 8 <code>detail-row</code>                                                                                 | Cada fila tiene una etiqueta y un valor                                                                                              |
| 81-90 | <code>&lt;div className="modal-footer"&gt;</code>                                                         | Pie con botones de acción                                                                                                            |
| 83-87 | <code>{canEdit() &amp;&amp; &lt;button onClick=<br/>{onEdit}&gt;Editar<br/>Usuario&lt;/button&gt;}</code> | Botón de edición (solo si tiene permisos)                                                                                            |
| 88-90 | <code>&lt;button onClick=<br/>{onClose}&gt;Cerrar&lt;/button&gt;</code>                                   | Botón para cerrar modal                                                                                                              |

 Escenario real: Usuario admin ve detalles de otro usuario

```
javascript
// currentUser = { id: 1, role: 'admin', name: 'Admin' }
// user = { id: 5, name: 'Oliver', lastname: 'Stone', role: 'admin',
//         email: 'oliver@stone.com', phone: '123456789',
//         created_at: '2024-01-15T10:30:00Z', updated_at: '2024-03-20T14:45:00Z' }
```

```
}

1. canEdit() → currentUser.role === 'admin' → true
2. getRoleText('admin') → '👑 Administrador'
3. getRoleBadgeClass('admin') → 'role-badge role-admin' (color rojo)
4. formatDate('2024-01-15T10:30:00Z') → '15/1/2024 10:30:00' (depende de la zona horaria)
5. Se muestra el modal con:
   - ID: 5
   - Nombre completo: Oliver Stone
   - Email: oliver@stone.com
   - Rol: [badge rojo] 👑 Administrador
   - Teléfono: 123456789
   - Fecha registro: 15/1/2024 10:30:00
   - Última actualización: 20/3/2024 14:45:00
6. El botón "Editar Usuario" es visible (canEdit = true)
```

## ⚡ Escenario real: Usuario seller ve detalles de otro usuario

```
javascript

// currentUser = { id: 10, role: 'seller', name: 'Seller' }
// user = { id: 5, name: 'Oliver', role: 'admin', ... }

1. canEdit() → currentUser.role === 'seller' → true (seller puede editar)
2. El botón "Editar Usuario" es visible
3. Si hace clic en "Editar", se ejecuta onEdit() y se abre UserForm
```

## ⚡ Escenario real: Usuario normal (customer/user) ve detalles

```
javascript

// currentUser = { id: 10, role: 'customer', name: 'Cliente' }

1. canEdit() → currentUser.role !== 'admin' y !== 'seller' → false
2. El botón "Editar Usuario" NO se muestra
3. Solo puede ver los detalles y cerrar el modal
```

## 🏷️ Badges de roles y sus colores

| Rol            | Clase CSS     | Color     | Texto           |
|----------------|---------------|-----------|-----------------|
| admin          | role-admin    | 🔴 Rojo    | 👑 Administrador |
| seller         | role-seller   | 🟠 Naranja | 🛒 Vendedor      |
| customer       | role-customer | 🟢 Verde   | 👤 Cliente       |
| user (default) | role-user     | ⬛ Gris    | 👤 Usuario       |

## 🔄 Flujo de formateo de fechas

```
javascript

const formatDate = (dateString) => {
  if (!dateString) return 'N/A'
  try {
    return new Date(dateString).toLocaleString()
  } catch {
    return 'N/A'
  }
}
```

Ejemplos de formato en diferentes zonas horarias:



| Entrada (ISO)        | Salida en España (UTC+1) | Salida en México (UTC-6) |
|----------------------|--------------------------|--------------------------|
| 2024-01-15T10:30:00Z | 15/1/2024 11:30:00       | 15/1/2024 4:30:00        |
| 2024-12-25T00:00:00Z | 25/12/2024 1:00:00       | 24/12/2024 18:00:00      |
| null                 | N/A                      | N/A                      |

### 💡 Analogía para entenderlo

Imagina que `UserDetails.jsx` es una **ficha personal de empleado en una empresa**:

- **Modal overlay** → Es la ventana emergente que cubre la pantalla para enfocar la atención.
- `user.name` y `user.lastname` → Es el nombre completo del empleado.
- **Badge de rol** → Es la credencial de color: rojo para directores, naranja para supervisores, verde para personal normal.
- `user.email` → Es su correo corporativo.
- `user.phone` → Es su número de contacto.
- `user.created_at` → Es la fecha de contratación.
- `user.updated_at` → Es la última vez que actualizaron sus datos.
- **Botón "Editar Usuario"** → Es la acción que solo managers y RRHH pueden hacer (los empleados normales solo pueden ver su propia ficha).

### ⚠️ Puntos clave para recordar

| Concepto                                             | Explicación                                                                                                   |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>modal-overlay</code>                           | Cubre toda la pantalla con fondo semitransparente oscuro                                                      |
| <code>modal-content</code>                           | Contenedor blanco centrado con el contenido del modal                                                         |
| <code>e.stopPropagation()</code>                     | <b>CRUCIAL:</b> Sin esto, hacer clic dentro del modal cerraría el modal (porque el evento subiría al overlay) |
| <code>canEdit()</code>                               | Usa <code>currentUser.role</code> para determinar si mostrar el botón de edición                              |
| <code>formatDate()</code> con <code>try/catch</code> | Previene que fechas mal formateadas rompan la aplicación                                                      |
| <code>user.lastname    ''</code>                     | Si no hay apellido, muestra string vacío (no "null" o "undefined")                                            |
| <code>user.phone    'No registrado'</code>           | Mensaje amigable cuando no hay teléfono                                                                       |

### 🐛 Posibles errores en este punto

| Error                          | Causa                                                               | Solución                                                                                          |
|--------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>user is undefined</code> | <code>UsersView</code> pasa <code>selectedUser</code> null al modal | Verificar que <code>handleViewDetails</code> setee <code>selectedUser</code> antes de abrir modal |

| Error                                            | Causa                                                                | Solución                                                                                               |
|--------------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>user.created_at</code> no se formatea      | La fecha viene en formato diferente                                  | Agregar más lógica de parsing en <code>formatDate</code>                                               |
| El modal se cierra al hacer clic dentro          | Falta <code>e.stopPropagation()</code> en <code>modal-content</code> | Agregar <code>onClick={(e) =&gt; e.stopPropagation()}</code>                                           |
| El badge no tiene el color correcto              | La clase CSS no está definida o importada                            | Verificar que <code>Users.css</code> tenga <code>.role-admin</code> , <code>.role-seller</code> , etc. |
| El botón "Editar Usuario" no aparece para seller | <code>canEdit()</code> solo verifica <code>role === 'admin'</code>   | Ya incluye <code>   currentUser.role === 'seller'</code><br>✅                                          |

## 📄 Mejora potencial: Mostrar avatar del usuario

Actualmente no se muestra imagen. Se podría agregar:

```
javascript
<div className="detail-row">
  <span className="detail-label">Avatar:</span>
  <span className="detail-value">
    {user.image ? (
      <img src={user.image} alt="Avatar" style={{ width: '50px', borderRadius: '50%' }} />
    ) : (
      'Sin imagen'
    )}
  </span>
</div>
```

¿Entendiste por qué `UserDetails.jsx` es un modal informativo que muestra todos los datos de un usuario y permite editar solo si el usuario actual tiene permisos?

Escribe "siguiente" y pasamos al ARCHIVO 21: `AlertMessage.jsx` y

`LoadingSpinner.jsx` (componentes auxiliares). 🛎

siguiente

## 📄 ARCHIVO 21: `src/views/common/AlertMessage.jsx` (VIGESIMOPRIMERO en ejecutarse)

```
javascript
// src/views/common/AlertMessage.jsx
import { useEffect, useState } from 'react'

const AlertMessage = ({ type, message, onClose, duration = 5000 }) => {
  const [isVisible, setIsVisible] = useState(true)

  useEffect(() => {
    if (duration > 0) {
      const timer = setTimeout(() => {
        setIsVisible(false)
        if (onClose) setTimeout(onClose, 300)
      }, duration)

      return () => clearTimeout(timer)
    }
  })
}
```

```

    }, [duration, onClose])

    if (!isVisible) return null

    return (
      <div className={`alert alert-${type}`}>
        <span className="alert-message">{message}</span>
        {onClose && (
          <button className="alert-close" onClick={() => {
            setIsVisible(false)
            setTimeout(onClose, 300)
          }}>
            x
          </button>
        )}
      </div>
    )
  }
}

export default AlertMessage

```

## 🔧 Responsabilidad en la ejecución

`AlertMessage.jsx` es un **componente de notificación temporal** que se ejecuta **cada vez que se necesita mostrar un mensaje de éxito, error o información al usuario**. Su responsabilidad es:

1. **Mostrar un mensaje** en formato de alerta (éxito o error) con estilo visual.
2. **Desaparecer automáticamente** después de un tiempo configurable (por defecto 5 segundos).
3. **Permitir cierre manual** mediante un botón "x".
4. **Ejecutar un callback** ( `onClose` ) cuando la alerta se cierra (automática o manualmente).
5. **Animar la desaparición** (con un pequeño delay de 300ms entre ocultar visualmente y ejecutar `onClose` ).

## 🔍 Orden de ejecución línea por línea

| Orden | Línea                                                                                   | ¿Qué pasa en tiempo de ejecución?                                        |
|-------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| 1     | <code>import { useEffect, useState } from 'react'</code>                                | Importa hooks de React                                                   |
| 3     | <code>const AlertMessage = ({ type, message, onClose, duration = 5000 }) =&gt; {</code> | Define componente y recibe 4 props (duration tiene valor por defecto)    |
| 4     | <code>const [isVisible, setIsVisible] = useState(true)</code>                           | Estado que controla si la alerta es visible (inicialmente true)          |
| 6-15  | <code>useEffect(() =&gt; { ... }, [duration, onClose])</code>                           | 🔥 <b>EFEECTO PRINCIPAL</b> que programa el cierre automático             |
| 7     | <code>if (duration &gt; 0) {</code>                                                     | Solo programa cierre si duration es mayor a 0                            |
| 8     | <code>const timer = setTimeout(() =&gt; {</code>                                        | Programa una función para ejecutarse después de <code>duration</code> ms |
| 9     | <code>setIsVisible(false)</code>                                                        | Oculto la alerta (cambia estado a false)                                 |

| Orden | Línea                                                                           | ¿Qué pasa en tiempo de ejecución?                                                         |
|-------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 10-11 | <pre>if (onClose)   setTimeout(onClose, 300)</pre>                              | Después de 300ms, ejecuta <code>onClose</code> (para dar tiempo a la animación de salida) |
| 12    | <pre>}, duration)</pre>                                                         | Tiempo de espera principal                                                                |
| 14    | <pre>return () =&gt; clearTimeout(timer)</pre>                                  | <b>CLEANUP:</b> Si el componente se desmonta antes, cancela el timer                      |
| 17    | <pre>if (!isVisible) return null</pre>                                          | Si no es visible, no renderiza nada                                                       |
| 19-28 | <pre>return (...)</pre>                                                         | Renderiza la alerta visible                                                               |
| 20    | <pre>&lt;div className={alert alert-\${type}} &gt;&gt;</pre>                    | Contenedor con clase dinámica: <code>alert-success</code> o <code>alert-error</code>      |
| 21    | <pre>&lt;span className="alert- message"&gt;{message}&lt;/span&gt;</pre>        | El texto del mensaje                                                                      |
| 22-27 | <pre>{onClose &amp;&amp; (&lt;button ...&gt;&lt;/button&gt;)}</pre>             | Botón de cierre manual (solo si <code>onClose</code> existe)                              |
| 23    | <pre>onClick={() =&gt; { setIsVisible(false); setTimeout(onClose, 300) }}</pre> | Al hacer clic: oculta alerta, luego ejecuta <code>onClose</code>                          |

## ⚡ Escenario real: Mostrar mensaje de éxito

```
javascript

// Desde UsersView.jsx:
setMessage({ type: 'success', text: 'Usuario eliminado exitosamente' })

// En el return:
{message && <AlertMessage type={message.type} message={message.text} onClose={() =
> setMessage(null)} />}

// DENTRO DE AlertMessage:

1. type = "success", message = "Usuario eliminado exitosamente", duration = 5000
   (por defecto)

2. Se renderiza: <div className="alert alert-success"> ... </div>

3. useEffect se ejecuta y programa un timer por 5000ms (5 segundos)

4. El usuario ve el mensaje verde de éxito

5. Después de 5 segundos:
   - setIsVisible(false) → alerta desaparece visualmente
   - 300ms después → onClose() se ejecuta → setMessage(null)

6. El mensaje ya no está en el DOM
```

## ⚡ Escenario real: Usuario cierra manualmente la alerta

```
javascript

// Misma alerta, pero el usuario hace clic en "x" antes de que pasen 5 segundos

1. Usuario hace clic en el botón "x"
```

2. onClick ejecuta: `setVisible(false)` → alerta desaparece
3. 300ms después: `onClose()` se ejecuta → `setMessage(null)`
4. El timer programado por `useEffect` NO se ejecuta (o se ejecuta pero no hace nada porque el componente ya no está montado)

## 🎨 Clases CSS generadas según el tipo

| type prop | Clase CSS           | Estilo típico                                        |
|-----------|---------------------|------------------------------------------------------|
| "success" | alert alert-success | Fondo verde, texto blanco                            |
| "error"   | alert alert-error   | Fondo rojo, texto blanco                             |
| "info"    | alert alert-info    | Fondo azul, texto blanco (no usado en código actual) |
| "warning" | alert alert-warning | Fondo amarillo, texto oscuro (no usado)              |

## 🔄 Flujo de timers y cleanup

```
javascript

useEffect(() => {
  if (duration > 0) {
    const timer = setTimeout(() => {
      setVisible(false)
      if (onClose) setTimeout(onClose, 300)
    }, duration)

    return () => clearTimeout(timer)
  }
}, [duration, onClose])
```

¿Por qué `setTimeout(onClose, 300)` dentro del timer principal?

Para dar tiempo a que la animación de salida (fade-out, slide-up, etc.) se complete antes de eliminar el componente del DOM. Si `onClose` eliminara el componente inmediatamente, no se vería la animación.

## 🎨 Comparación con otros sistemas de notificaciones

| Sistema        | ¿Cómo funciona?                                   | Uso en este proyecto                    |
|----------------|---------------------------------------------------|-----------------------------------------|
| AlertMessage   | Componente que se monta/desmonta con estado local | Actual                                  |
| Toast          | Sistema global con cola de notificaciones         | ❌ No implementado                       |
| window.alert() | Modal nativo bloqueante                           | ❌ Solo para confirmación de eliminación |
| Snackbar       | Notificación inferior estilo Material Design      | ❌ No implementado                       |

## 💡 Analogía para entenderlo

Imagina que `AlertMessage.jsx` es un cartel temporal en una estación de tren:

- `type="success"` → Es un cartel verde que dice "Tren en hora ✅".

- `type="error"` → Es un cartel rojo que dice "Tren cancelado ❌".
- `duration=5000` → El cartel se muestra por 5 segundos y luego desaparece.
- **Botón "x"** → Es el botón "Cerrar" que un pasajero puede presionar si ya leyó el mensaje.
- `onClose` → Es la persona que retira el cartel después de que desaparece.
- `setTimeout(onClose, 300)` → Espera 300ms después de que el cartel se desvanece para retirarlo completamente.

### ⚠ Puntos clave para recordar

| Concepto                                                              | Explicación                                                                                  |
|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>duration</code> con valor por defecto                           | Si no se pasa <code>duration</code> , usa 5000ms (5 segundos)                                |
| <code>duration = 0</code>                                             | Si se pasa 0, NO se programa cierre automático (alerta permanente hasta cierre manual)       |
| <b>Doble</b> <code>setTimeout</code>                                  | Uno para ocultar, otro para llamar <code>onClose</code> . Permite animaciones de salida      |
| <b>Cleanup</b> <code>clearTimeout</code>                              | Si el componente se desmonta antes de que termine el timer, lo cancela (evita memory leaks)  |
| <b>Render condicional</b><br><code>if (!isVisible) return null</code> | Cuando <code>isVisible = false</code> , el componente no renderiza nada (se elimina del DOM) |
| <b>Botón de cierre condicional</b>                                    | Solo aparece si <code>onClose</code> existe (alerta no cerrable si no se provee callback)    |

### 🐛 Posibles errores en este punto

| Error                                                 | Causa                                                                      | Solución                                                                                                                                           |
|-------------------------------------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| La alerta no desaparece                               | <code>duration</code> es 0 o negativo                                      | Pasar <code>duration</code> positivo o usar valor por defecto                                                                                      |
| <code>onClose</code> se ejecuta antes de la animación | No hay delay entre <code>setIsVisible(false)</code> y <code>onClose</code> | Ya está implementado con <code>setTimeout(onClose, 300)</code>  |
| La alerta parpadea al montarse                        | No hay estado inicial consistente                                          | <code>isVisible</code> inicia en <code>true</code> correctamente                                                                                   |
| El mensaje se corta                                   | El CSS no maneja texto largo                                               | Agregar <code>word-break: break-word</code> a <code>.alert-message</code>                                                                          |
| Múltiples alertas aparecen al mismo tiempo            | Se renderizan varios <code>AlertMessage</code>                             | Gestionar con un array de mensajes en lugar de estado único                                                                                        |

📄 **ARCHIVO 22:** `src/views/common/LoadingSpinner.jsx`  
(VIGESIMOSEGUNDO en ejecutarse)

```
// src/views/common/LoadingSpinner.jsx
const LoadingSpinner = () => {
  return (
    <div className="spinner-container">
      <div className="spinner"></div>
      <p>Cargando...</p>
    </div>
  )
}

export default LoadingSpinner
```

## Responsabilidad en la ejecución

`LoadingSpinner.jsx` es un **componente de carga** que se ejecuta **cuando la aplicación está esperando una operación asíncrona** (cargando usuarios, verificando autenticación, etc.). Su responsabilidad es:

1. **Mostrar un indicador visual** de que el sistema está procesando una operación.
2. **Acompañar con texto "Cargando..."** para claridad.
3. **Ser reutilizable** en cualquier parte de la aplicación.

## ¿Dónde se usa en el código?

| Archivo                     | Línea                                                                           | Propósito                                                    |
|-----------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------|
| App.jsx<br>(ProtectedRoute) | if (loading) return<br><LoadingSpinner />                                       | Mientras verifica autenticación                              |
| UsersView.jsx               | if (loading && users.length<br>=== 0) return <div>Cargando<br>usuarios...</div> | Mientras carga la lista (usa texto simple, no el componente) |

## CSS necesario para que funcione (debe estar en common.css o similar)

```
css

.spinner-container {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  padding: 50px;
}

.spinner {
  width: 40px;
  height: 40px;
  border: 4px solid #f3f3f3;
  border-top: 4px solid #667eea;
  border-radius: 50%;
  animation: spin 1s linear infinite;
}

@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}
```

## Analogía para entenderlo

Imagina que `LoadingSpinner.jsx` es un reloj de arena:

- Mientras la operación se completa, ves la arena cayendo.
- No puedes hacer nada hasta que termine.
- Es visualmente claro que "algo está pasando".

¿Entendiste cómo funcionan `AlertMessage` (notificaciones temporales) y `LoadingSpinner` (indicador de carga)?

Escribe "siguiente" y pasamos al ARCHIVO 23: `Navbar.css` (estilos de la barra de navegación) y finalmente a la puesta en marcha del frontend. 🚀

siguiente

## 📄 ARCHIVO 23: `src/styles/Navbar.css` (VIGESIMOTERCERO en ejecutarse)

```
css

/* src/styles/Navbar.css */
.navbar {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  position: sticky;
  top: 0;
  z-index: 1000;
}

.navbar-container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 1rem 2rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.navbar-logo a {
  color: white;
  text-decoration: none;
}

.navbar-logo h2 {
  font-size: 1.5rem;
  margin: 0;
}

.navbar-menu {
  display: flex;
  gap: 1.5rem;
  align-items: center;
}

.nav-link {
  color: white;
  text-decoration: none;
  padding: 0.5rem 1rem;
  border-radius: 8px;
  transition: all 0.3s ease;
  font-weight: 500;
}

.nav-link:hover {
  background: rgba(255, 255, 255, 0.2);
  transform: translateY(-2px);
}
```



```

.btn-primary {
  background: white;
  color: #667eea !important;
}

.btn-primary:hover {
  background: #f0f0f0;
  transform: translateY(-2px);
}

.btn-logout {
  background: #ef4444;
  border: none;
  cursor: pointer;
  font-size: 1rem;
}

.btn-logout:hover {
  background: #dc2626;
}

.user-welcome {
  color: white;
  font-weight: 500;
  padding: 0.5rem 1rem;
  background: rgba(255, 255, 255, 0.2);
  border-radius: 8px;
}

@media (max-width: 768px) {
  .navbar-container {
    flex-direction: column;
    gap: 1rem;
  }

  .navbar-menu {
    flex-wrap: wrap;
    justify-content: center;
  }
}

```

## Responsabilidad en la ejecución

`Navbar.css` es un **archivo de estilos** que se ejecuta **cuando** `Navbar.jsx` **lo importa** (`import './../styles/Navbar.css'`). Su responsabilidad es:

1. Definir la **apariciencia visual** de la barra de navegación.
2. **Aplicar un gradiente morado** como fondo (coincide con el fondo del login).
3. **Hacer que la barra sea "sticky"** (se queda fija en la parte superior al hacer scroll).
4. **Diseñar los enlaces** (`nav-link`) con efectos hover (translúcido y elevación).
5. **Estilizar el botón de logout** de color rojo.
6. **Aplicar estilos responsivos** para pantallas pequeñas (menos de 768px).

## Explicación CSS línea por línea

Barra principal `.navbar`

```

css

.navbar {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  position: sticky;
  top: 0;
}

```

```
z-index: 1000;
}
```

| Propiedad  | Valor                                             | Efecto                                             |
|------------|---------------------------------------------------|----------------------------------------------------|
| background | linear-gradient(135deg, #667eea 0%, #764ba2 100%) | Gradiente diagonal de morado claro a morado oscuro |
| box-shadow | 0 2px 10px rgba(0,0,0,0.1)                        | Sombra suave debajo de la barra                    |
| position   | sticky                                            | Barra fija en la parte superior al hacer scroll    |
| top        | 0                                                 | Se pega al borde superior de la ventana            |
| z-index    | 1000                                              | Aparece por encima de otros elementos              |

#### Contenedor .navbar-container

```
css

.navbar-container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 1rem 2rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

| Propiedad       | Valor         | Efecto                                 |
|-----------------|---------------|----------------------------------------|
| max-width       | 1200px        | Ancho máximo del contenido             |
| margin          | 0 auto        | Centra horizontalmente el contenedor   |
| padding         | 1rem 2rem     | Espaciado interno vertical/horizontal  |
| display         | flex          | Activa Flexbox para alinear elementos  |
| justify-content | space-between | Logo a la izquierda, menú a la derecha |
| align-items     | center        | Centra verticalmente los elementos     |

#### Enlaces .nav-link

```
css

.nav-link {
  color: white;
  text-decoration: none;
  padding: 0.5rem 1rem;
  border-radius: 8px;
  transition: all 0.3s ease;
  font-weight: 500;
}

.nav-link:hover {
  background: rgba(255, 255, 255, 0.2);
  transform: translateY(-2px);
}
```

| Propiedad                          | Efecto                                                 |
|------------------------------------|--------------------------------------------------------|
| color: white                       | Texto blanco                                           |
| padding: 0.5rem 1rem               | Espacio alrededor del texto (clickeable)               |
| border-radius: 8px                 | Bordes redondeados                                     |
| transition: all 0.3s ease          | Animación suave de 0.3 segundos al cambiar propiedades |
| :hover background                  | Fondo blanco semitransparente al pasar el mouse        |
| :hover transform: translateY(-2px) | El botón se eleva 2px al pasar el mouse                |

Botón de registro `.btn-primary`

```
css

.btn-primary {
  background: white;
  color: #667eea !important;
}

.btn-primary:hover {
  background: #f0f0f0;
  transform: translateY(-2px);
}
```

| Propiedad                  | Efecto                                                      |
|----------------------------|-------------------------------------------------------------|
| background: white          | Fondo blanco (destaca sobre el gradiente morado)            |
| color: #667eea !important  | Texto morado (fuerza el color con <code>!important</code> ) |
| :hover background: #f0f0f0 | Fondo gris claro al pasar el mouse                          |

Botón de logout `.btn-logout`

```
css

.btn-logout {
  background: #ef4444;
  border: none;
  cursor: pointer;
  font-size: 1rem;
}

.btn-logout:hover {
  background: #dc2626;
}
```

| Propiedad                  | Efecto                                       |
|----------------------------|----------------------------------------------|
| background: #ef4444        | Fondo rojo (destaca como acción destructiva) |
| border: none               | Sin borde                                    |
| cursor: pointer            | Manita al pasar el mouse                     |
| :hover background: #dc2626 | Rojo más oscuro al pasar el mouse            |

Bienvenida del usuario `.user-welcome`

```
css
```

```
.user-welcome {
  color: white;
  font-weight: 500;
  padding: 0.5rem 1rem;
  background: rgba(255, 255, 255, 0.2);
  border-radius: 8px;
}
```

| Propiedad                         | Efecto                                       |
|-----------------------------------|----------------------------------------------|
| background: rgba(255,255,255,0.2) | Fondo blanco semitransparente (20% opacidad) |
| border-radius: 8px                | Bordes redondeados como los botones          |

## Responsive (pantallas pequeñas)

```
css

@media (max-width: 768px) {
  .navbar-container {
    flex-direction: column;
    gap: 1rem;
  }

  .navbar-menu {
    flex-wrap: wrap;
    justify-content: center;
  }
}
```

| Regla                   | Efecto en móviles (ancho < 768px)                 |
|-------------------------|---------------------------------------------------|
| flex-direction: column  | Logo arriba, menú abajo (vertical)                |
| gap: 1rem               | Espacio entre logo y menú                         |
| flex-wrap: wrap         | Si hay muchos botones, pasan a la línea siguiente |
| justify-content: center | Botones centrados horizontalmente                 |

## Efectos visuales en acción

### Estado normal de un enlace:

```
text



Iniciar Sesión

 ← Texto blanco, fondo transparente
```

### Hover sobre un enlace:

```
text



Iniciar Sesión

 ← Texto blanco, fondo blanco 20%, elevado 2px
```

### Botón "Registrarse" (btn-primary):

```
text



Registrarse

 ← Texto morado, fondo blanco
```

### Botón "Cerrar Sesión" (btn-logout):

text

Cerrar Sesión ← Texto blanco, fondo rojo

### 💡 Analogía para entenderlo

Imagina que `Navbar.css` es el **diseñador de interiores** de una tienda:

- **Gradiente morado** → Es como pintar la pared de un color que llama la atención.
- `position: sticky` → Es como un letrero que siempre ves aunque camines por la tienda.
- `box-shadow` → Es como una sombra debajo del letrero para que resalte.
- `transition` → Es como un botón que se ilumina suavemente cuando pasas el dedo.
- `transform: translateY(-2px)` → Es como si el botón "saltara" un poco cuando lo tocas.
- `@media (max-width: 768px)` → Es como cambiar la distribución de la tienda cuando llegan muchos clientes (formato móvil).

### ⚠️ Puntos clave para recordar

| Concepto                                             | Explicación                                                                                                           |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>!important</code> en <code>.btn-primary</code> | Fuerza el color morado aunque haya otras reglas que lo sobrescriban                                                   |
| <code>rgba(255,255,255,0.2)</code>                   | Color blanco con 20% de opacidad (fondo semitransparente)                                                             |
| <code>position: sticky</code> vs <code>fixed</code>  | <code>sticky</code> se comporta como <code>relative</code> hasta que haces scroll, luego se vuelve <code>fixed</code> |
| Gradiente                                            | <code>linear-gradient(135deg, ...)</code> crea una transición suave entre dos colores en diagonal                     |
| Breakpoint 768px                                     | Es el punto típico donde una pantalla pasa de "tablet" a "móvil"                                                      |

### 🚀 PUESTA EN MARCHA DEL FRONTEND (REACT)

Ahora que hemos recorrido todos los archivos, te explico **cómo poner en marcha el frontend** paso a paso.

### 📋 Requisitos previos

| Requisito         | Verificación                                              |
|-------------------|-----------------------------------------------------------|
| Node.js instalado | <code>node --version</code> (debe mostrar v18 o superior) |
| npm instalado     | <code>npm --version</code>                                |
| Backend corriendo | <code>http://localhost:3000/api</code> debe responder     |

## Paso 1: Abrir terminal en la carpeta del frontend

```
bash

cd frontend_web/react_node_express
```

## Paso 2: Instalar dependencias

```
bash

npm install
```

### ¿Qué hace?

- Lee `package.json`
- Descarga todas las dependencias en `node_modules/`
- Genera `package-lock.json` (bloquea versiones exactas)

Tiempo estimado: 1-2 minutos

## Paso 3: Verificar configuración de API

Edita `src/config/api.js` y asegúrate de que `BASE_URL` apunte a tu backend:

```
javascript

BASE_URL: 'http://localhost:3000/api', // Si backend está en la misma PC
// o
BASE_URL: 'http://192.168.1.100:3000/api', // Si pruebas desde celular en la misma red
```

## Paso 4: Iniciar el servidor de desarrollo

```
bash

npm run dev
```

### Salida esperada:

```
text

VITE v8.0.4 ready in 300 ms

→ Local:   http://localhost:5173/
→ Network: http://192.168.1.100:5173/
```

## Paso 5: Abrir el navegador

Ve a `http://localhost:5173`

Deberías ver la pantalla de login.

## Paso 6: Probar la aplicación

| Acción                                  | Resultado esperado                  |
|-----------------------------------------|-------------------------------------|
| Ir a <code>http://localhost:5173</code> | Redirige a <code>/login</code>      |
| Ingresa credenciales válidas            | Redirige a <code>/dashboard</code>  |
| Hacer clic en "Usuarios"                | Muestra tabla con lista de usuarios |

| Acción                         | Resultado esperado               |
|--------------------------------|----------------------------------|
| Hacer clic en "👁 Ver detalles" | Modal con datos del usuario      |
| Hacer clic en "✏ Editar"       | Modal para editar (admin/seller) |
| Hacer clic en "Cerrar Sesión"  | Vuelve a <code>/login</code>     |

### 🔧 Comandos útiles

| Comando                      | Descripción                                        |
|------------------------------|----------------------------------------------------|
| <code>npm run dev</code>     | Inicia servidor de desarrollo                      |
| <code>npm run build</code>   | Genera versión de producción en <code>dist/</code> |
| <code>npm run preview</code> | Previsualiza la versión de producción              |
| <code>npm run lint</code>    | Ejecuta ESLint para verificar calidad del código   |
| <code>Ctrl + C</code>        | Detiene el servidor                                |

### 🦖 Solución de problemas comunes

| Problema                  | Posible causa                                | Solución                                     |
|---------------------------|----------------------------------------------|----------------------------------------------|
| Error: Cannot find module | Dependencias no instaladas                   | Ejecutar <code>npm install</code>            |
| ECONNREFUSED 3000         | Backend no está corriendo                    | Iniciar backend con <code>npm run dev</code> |
| Login falla con 401       | Credenciales incorrectas o usuario no existe | Verificar en base de datos                   |
| Los estilos no se ven     | CSS no importado                             | Verificar imports en <code>main.jsx</code>   |
| El dashboard no carga     | Token inválido o expirado                    | Limpiar localStorage y reloguearse           |

### 📁 Estructura final después de `npm run build`

```
text

dist/
├─ index.html
├─ assets/
│   └─ index-abc123.css  (CSS minificado)
│   └─ index-def456.js  (JS minificado)
└─ vite.svg
```

¡FELICITACIONES! 🎉 Has completado el recorrido completo del frontend React, desde `index.html` hasta `LoadingSpinner.jsx`, pasando por todos los controladores, modelos, servicios y vistas.

¿Hay algo más específico que quieras que explique del frontend o prefieres que pasemos al backend?